

Kubernetes详细教程

1. Kubernetes介绍

1.1 应用部署方式演变

在部署应用程序的方式上，主要经历了三个时代：

- **传统部署**：互联网早期，会直接将应用程序部署在物理机上

优点：简单，不需要其它技术的参与

缺点：不能为应用程序定义资源使用边界，很难合理地分配计算资源，而且程序之间容易产生影响

- **虚拟化部署**：可以在一台物理机上运行多个虚拟机，每个虚拟机都是独立的一个环境

优点：程序环境不会相互产生影响，提供了一定程度的安全性

缺点：增加了操作系统，浪费了部分资源

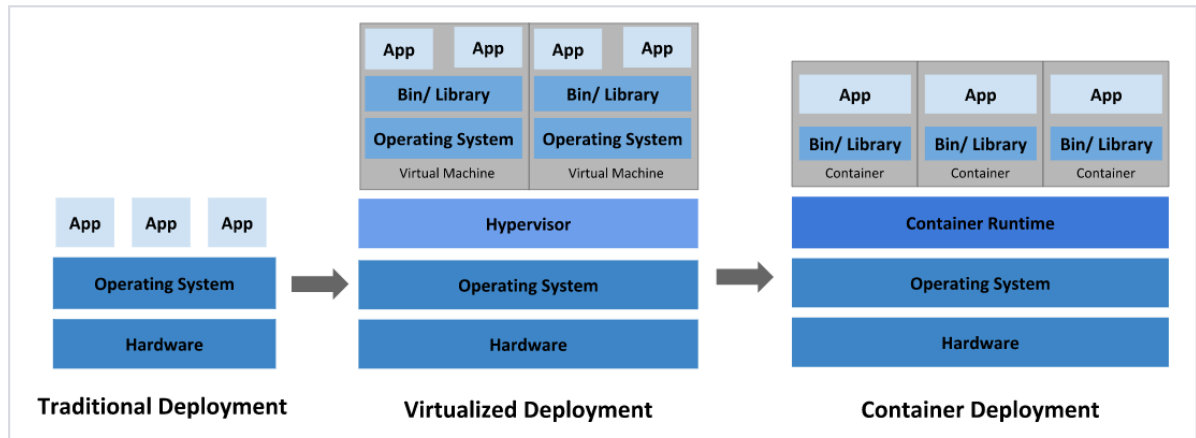
- **容器化部署**：与虚拟化类似，但是共享了操作系统

优点：

可以保证每个容器拥有自己的文件系统、CPU、内存、进程空间等

运行应用程序所需要的资源都被容器包装，并和底层基础架构解耦

容器化的应用程序可以跨云服务商、跨Linux操作系统发行版进行部署

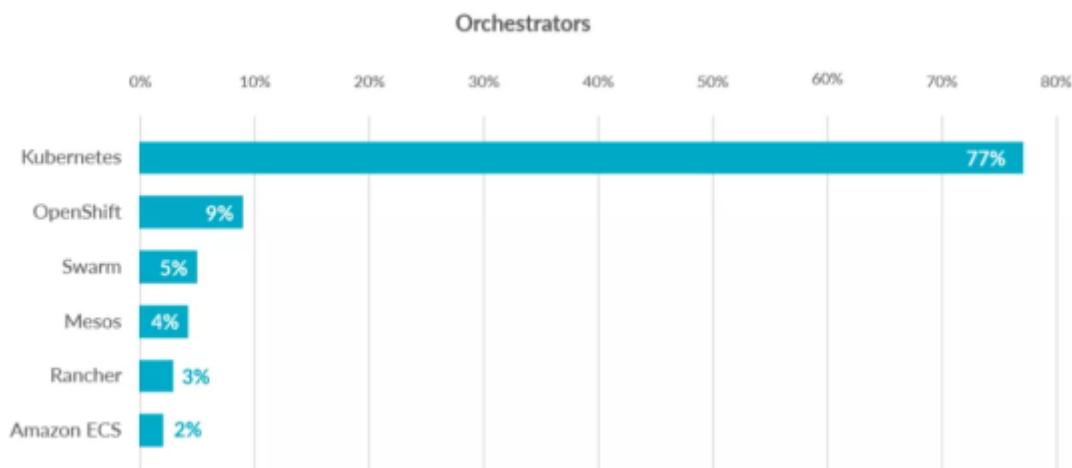


容器化部署方式给带来很多的便利，但是也会出现一些问题，比如说：

- 一个容器故障停机了，怎么样让另外一个容器立刻启动去替补停机的容器
- 当并发访问量变大的时候，怎么样做到横向扩展容器数量

这些容器管理的问题统称为**容器编排**问题，为了解决这些容器编排问题，就产生了一些容器编排的软件：

- **Swarm**：Docker自己的容器编排工具
- **Mesos**：Apache的一个资源统一管控的工具，需要和Marathon结合使用
- **Kubernetes**：Google开源的的容器编排工具



1.2 kubernetes简介



kubernetes，是一个全新的基于容器技术的分布式架构领先方案，是谷歌严格保密十几年的秘密武器----Borg系统的一个开源版本，于2014年9月发布第一个版本，2015年7月发布第一个正式版本。

kubernetes的本质是一组**服务器集群**，它可以在集群的每个节点上运行特定的程序，来对节点中的容器进行管理。目的是实现资源管理的自动化，主要提供了如下的主要功能：

- **自我修复**：一旦某一个容器崩溃，能够在1秒中左右迅速启动新的容器
- **弹性伸缩**：可以根据需要，自动对集群中正在运行的容器数量进行调整
- **服务发现**：服务可以通过自动发现的形式找到它所依赖的服务
- **负载均衡**：如果一个服务启动了多个容器，能够自动实现请求的负载均衡
- **版本回退**：如果发现新发布的程序版本有问题，可以立即回退到原来的版本
- **存储编排**：可以根据容器自身的需求自动创建存储卷

1.3 kubernetes组件

一个kubernetes集群主要是由**控制节点(master)**、**工作节点(node)**构成，每个节点上都会安装不同的组件。

master：集群的控制平面，负责集群的决策（管理）

ApiServer : 资源操作的唯一入口, 接收用户输入的命令, 提供认证、授权、API注册和发现等机制

Scheduler : 负责集群资源调度, 按照预定的调度策略将Pod调度到相应的node节点上

ControllerManager : 负责维护集群的状态, 比如程序部署安排、故障检测、自动扩展、滚动更新等

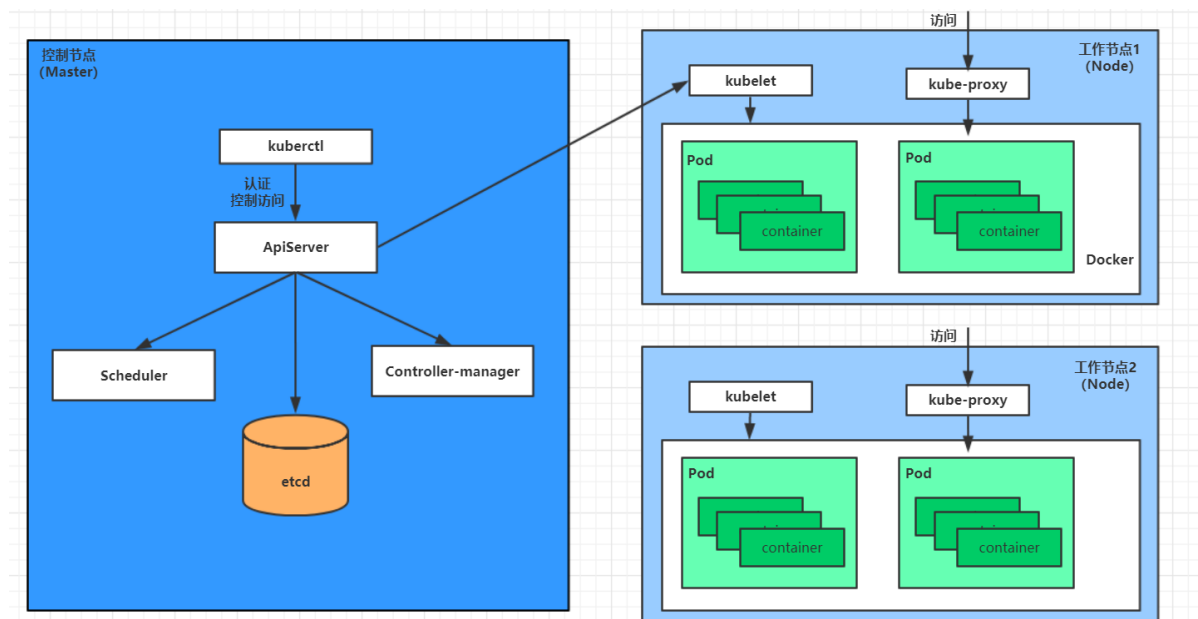
Etcd : 负责存储集群中各种资源对象的信息

node: 集群的数据平面, 负责为容器提供运行环境 (干活)

Kubelet : 负责维护容器的生命周期, 即通过控制docker, 来创建、更新、销毁容器

KubeProxy : 负责提供集群内部的服务发现和负载均衡

Docker : 负责节点上容器的各种操作



下面, 以部署一个nginx服务来说明kubernetes系统各个组件调用关系:

1. 首先要明确, 一旦kubernetes环境启动之后, master和node都会将自身的信息存储到etcd数据库中
2. 一个nginx服务的安装请求会首先被发送到master节点的apiServer组件
3. apiServer组件会调用scheduler组件来决定到底应该把这个服务安装到哪个node节点上
在此时, 它会从etcd中读取各个node节点的信息, 然后按照一定的算法进行选择, 并将结果告知apiServer
4. apiServer调用controller-manager去调度Node节点安装nginx服务
5. kubelet接收到指令后, 会通知docker, 然后由docker来启动一个nginx的pod
pod是kubernetes的最小操作单元, 容器必须跑在pod中至此,
6. 一个nginx服务就运行了, 如果需要访问nginx, 就需要通过kube-proxy来对pod产生访问的代理
这样, 外界用户就可以访问集群中的nginx服务了

1.4 kubernetes概念

Master: 集群控制节点, 每个集群需要至少一个master节点负责集群的管控

Node: 工作负载节点, 由master分配容器到这些node工作节点上, 然后node节点上的docker负责容器的运行

Pod: kubernetes的最小控制单元，容器都是运行在pod中的，一个pod中可以有1个或者多个容器

Controller: 控制器，通过它来实现对pod的管理，比如启动pod、停止pod、伸缩pod的数量等等

Service: pod对外服务的统一入口，下面可以维护者同一类的多个pod

Label: 标签，用于对pod进行分类，同一类pod会拥有相同的标签

NameSpace: 命名空间，用来隔离pod的运行环境

2. kubernetes集群环境搭建

2.1 前置知识点

目前生产部署Kubernetes 集群主要有两种方式：

kubeadm

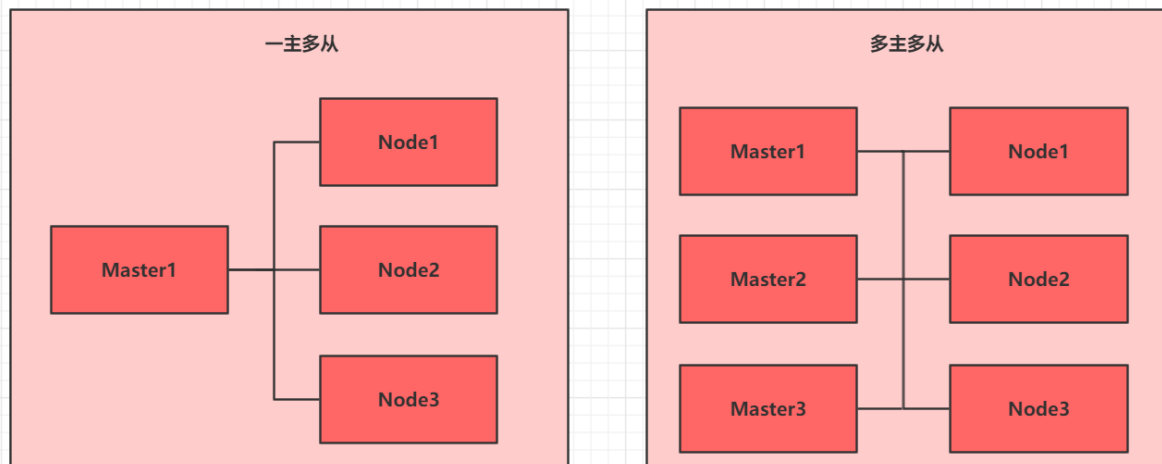
Kubeadm 是一个K8s 部署工具，提供kubeadm init 和kubeadm join，用于快速部署Kubernetes 集群。

官方地址: <https://kubernetes.io/docs/reference/setup-tools/kubeadm/kubeadm/>

二进制包

从github 下载发行版的二进制包，手动部署每个组件，组成Kubernetes 集群。

Kubeadm 降低部署门槛，但屏蔽了很多细节，遇到问题很难排查。如果想更容易可控，推荐使用二进制包部署Kubernetes 集群，虽然手动部署麻烦点，期间可以学习很多工作原理，也利于后期维护。



2.2 kubeadm 部署方式介绍

kubeadm 是官方社区推出的一个用于快速部署kubernetes 集群的工具，这个工具能通过两条指令完成一个kubernetes 集群的部署：

- 创建一个Master 节点kubeadm init
- 将Node 节点加入到当前集群中\$ kubeadm join <Master 节点的IP 和端口>

2.3 安装要求

在开始之前，部署Kubernetes 集群机器需要满足以下几个条件：

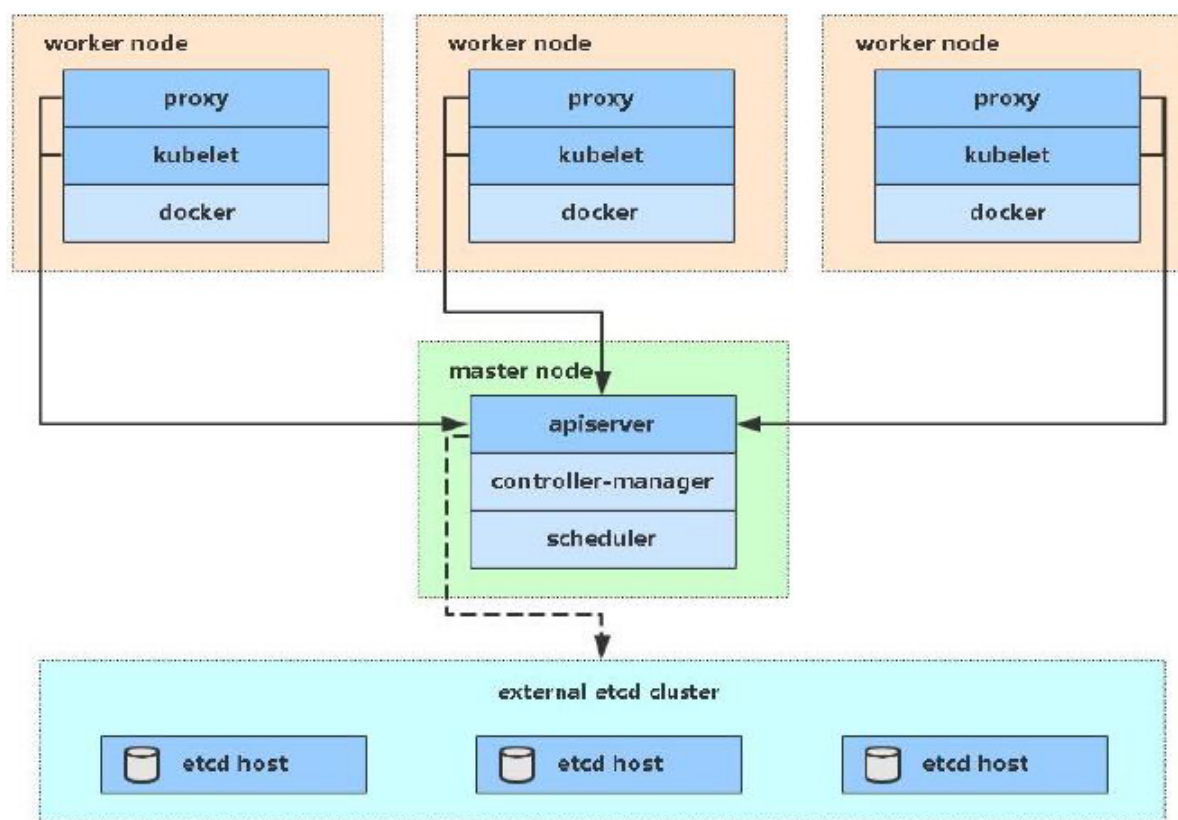
- 一台或多台机器，操作系统CentOS7.x-86_x64
- 硬件配置：2GB 或更多RAM，2 个CPU 或更多CPU，硬盘30GB 或更多
- 集群中所有机器之间网络互通

- 可以访问外网，需要拉取镜像
- 禁止swap 分区

2.4 最终目标

- 在所有节点上安装Docker 和kubeadm
- 部署Kubernetes Master
- 部署容器网络插件
- 部署Kubernetes Node，将节点加入Kubernetes 集群中
- 部署Dashboard Web 页面，可视化查看Kubernetes 资源

2.5 准备环境



角色	IP地址	组件
master01	192.168.5.100	docker, kubectl, kubeadm, kubelet
node01	192.168.5.101	docker, kubectl, kubeadm, kubelet
node02	192.168.5.102	docker, kubectl, kubeadm, kubelet

2.6 环境初始化

2.6.1 检查操作系统的版本

```
# 此方式下安装kubernetes集群要求Centos版本要在7.5或之上
[root@master ~]# cat /etc/redhat-release
Centos Linux 7.5.1804 (Core)
```

```
#查看虚拟网卡ip网段 vm8 192.168.5.1
默认网关: 192.168.5.2
dns: 223.5.5.5 ps: 阿里云申请也行
子网掩码: 255.255.255.0
type: static
```

2.6.2 主机名解析

为了方便集群节点间的直接调用，在这个配置一下主机名解析，企业中推荐使用内部DNS服务器

```
# 主机名成解析 编辑三台服务器的/etc/hosts文件，添加下面内容
192.168.5.100 master
192.168.5.101 node1
192.168.5.102 node2
```

2.6.3 时间同步

kubernetes要求集群中的节点时间必须精确一直，这里使用chronyd服务从网络同步时间

企业中建议配置内部的会见同步服务器

```
# 启动chronyd服务
[root@master ~]# systemctl start chronyd
[root@master ~]# systemctl enable chronyd
[root@master ~]# date
```

2.6.4 禁用iptables和firewalld服务

kubernetes和docker 在运行的中会产生大量的iptables规则，为了不让系统规则跟它们混淆，直接关闭系统的规则

```
# 1 关闭firewalld服务
[root@master ~]# systemctl stop firewalld
[root@master ~]# systemctl disable firewalld
# 2 关闭iptables服务
[root@master ~]# systemctl stop iptables
[root@master ~]# systemctl disable iptables
```

2.6.5 禁用selinux

selinux是linux系统下的一个安全服务，如果不关闭它，在安装集群中会产生各种各样的奇葩问题

```
# 编辑 /etc/selinux/config 文件，修改SELINUX的值为disable
# 注意修改完毕之后需要重启linux服务
SELINUX=disabled
```

2.6.6 禁用swap分区

swap分区指的是虚拟内存分区，它的作用是物理内存使用完，之后将磁盘空间虚拟成内存来使用，启用swap设备会对系统的性能产生非常负面的影响，因此kubernetes要求每个节点都要禁用swap设备，但是如果因为某些原因确实不能关闭swap分区，就需要在集群安装过程中通过明确的参数进行配置说明

```
# 编辑分区配置文件/etc/fstab，注释掉swap分区一行
# 注意修改完毕之后需要重启linux服务
vim /etc/fstab
注释掉 /dev/mapper/centos-swap swap
# /dev/mapper/centos-swap swap
```

2.6.7 修改linux的内核参数

```
# 修改linux的内核参数，添加网桥过滤和地址转发功能
# 编辑/etc/sysctl.d/kubernetes.conf文件，添加如下配置：
net.bridge.bridge-nf-call-ip6tables = 1
net.bridge.bridge-nf-call-iptables = 1
net.ipv4.ip_forward = 1

# 重新加载配置
[root@master ~]# sysctl -p
# 加载网桥过滤模块
[root@master ~]# modprobe br_netfilter
# 查看网桥过滤模块是否加载成功
[root@master ~]# lsmod | grep br_netfilter
```

2.6.8 配置ipvs功能

在Kubernetes中Service有两种带来模型，一种是基于iptables的，一种是基于ipvs的两者比较的话，ipvs的性能明显要高一些，但是如果使用它，需要手动载入ipvs模块

```
# 1. 安装ipset和ipvsadm
[root@master ~]# yum install ipset ipvsadm -y
# 2. 添加需要加载的模块写入脚本文件
[root@master ~]# cat <<EOF> /etc/sysconfig/modules/ipvs.modules
#!/bin/bash
modprobe -- ip_vs
modprobe -- ip_vs_rr
modprobe -- ip_vs_wrr
modprobe -- ip_vs_sh
modprobe -- nf_conntrack_ipv4
EOF
# 3. 为脚本添加执行权限
[root@master ~]# chmod +x /etc/sysconfig/modules/ipvs.modules
# 4. 执行脚本文件
[root@master ~]# /bin/bash /etc/sysconfig/modules/ipvs.modules
# 5. 查看对应的模块是否加载成功
[root@master ~]# lsmod | grep -e ip_vs -e nf_conntrack_ipv4
```

2.6.9 安装docker

```
# 1、切换镜像源
[root@master ~]# wget https://mirrors.aliyun.com/docker-ce/linux/centos/docker-ce.repo -O /etc/yum.repos.d/docker-ce.repo

# 2、查看当前镜像源中支持的docker版本
[root@master ~]# yum list docker-ce --showduplicates

# 3、安装特定版本的docker-ce
```

```
# 必须制定--setopt=obsoletes=0, 否则yum会自动安装更高版本
[root@master ~]# yum install --setopt=obsoletes=0 docker-ce-18.06.3.ce-3.el7 -y

# 4、添加一个配置文件
#Docker 在默认情况下使用Vgroup Driver为cgroupfs, 而Kubernetes推荐使用systemd来替代cgroupfs
[root@master ~]# mkdir /etc/docker
[root@master ~]# cat <<EOF> /etc/docker/daemon.json
{
    "exec-opts": ["native.cgroupdriver=systemd"],
    "registry-mirrors": ["https://kn0t2bca.mirror.aliyuncs.com"]
}
EOF

# 5、启动doker
[root@master ~]# systemctl restart docker
[root@master ~]# systemctl enable docker
```

2.6.10 安装Kubernetes组件

```
# 1、由于kubernetes的镜像在国外, 速度比较慢, 这里切换成国内的镜像源
# 2、编辑/etc/yum.repos.d/kubernetes.repo, 添加下面的配置
[kubernetes]
name=Kubernetes
baseurl=http://mirrors.aliyun.com/kubernetes/yum/repos/kubernetes-el7-x86_64
enabled=1
gpgcheck=0
repo_gpgcheck=0
gpgkey=http://mirrors.aliyun.com/kubernetes/yum/doc/yum-key.gpg
        http://mirrors.aliyun.com/kubernetes/yum/doc/rpm-package-key.gpg

# 3、安装kubeadm、kubenet和kubectl
[root@master ~]# yum install --setopt=obsoletes=0 kubeadm-1.17.4-0 kubelet-1.17.4-0 kubectl-1.17.4-0 -y

# 4、配置kubelet的cgroup
#编辑/etc/sysconfig/kubelet, 添加下面的配置
KUBELET_CGROUP_ARGS="--cgroup-driver=systemd"
KUBE_PROXY_MODE="ipvs"

# 5、设置kubelet开机自启
[root@master ~]# systemctl enable kubelet
```

2.6.11 准备集群镜像

```
# 在安装kubernetes集群之前, 必须要提前准备好集群需要的镜像, 所需镜像可以通过下面命令查看
[root@master ~]# kubeadm config images list

# 下载镜像
# 此镜像kubernetes的仓库中, 由于网络原因, 无法连接, 下面提供了一种替换方案
images=(
    kube-apiserver:v1.17.4
    kube-controller-manager:v1.17.4
    kube-scheduler:v1.17.4
    kube-proxy:v1.17.4
```



```

    pause:3.1
    etcd:3.4.3-0
    coredns:1.6.5
)

for imageName in ${images[@]};do
    docker pull registry.cn-hangzhou.aliyuncs.com/google_containers/$imageName
    docker tag registry.cn-hangzhou.aliyuncs.com/google_containers/$imageName
k8s.gcr.io/$imageName
    docker rmi registry.cn-hangzhou.aliyuncs.com/google_containers/$imageName
done

```

2.6.11 集群初始化

下面的操作只需要在master节点上执行即可

```

# 创建集群
[root@master ~]# kubeadm init \
    --apiserver-advertise-address=192.168.5.100 \
    --image-repository registry.aliyuncs.com/google_containers \
    --kubernetes-version=v1.17.4 \
    --service-cidr=10.96.0.0/12 \
    --pod-network-cidr=10.244.0.0/16

# 创建必要文件
[root@master ~]# mkdir -p $HOME/.kube
[root@master ~]# sudo cp -i /etc/kubernetes/admin.conf $HOME/.kube/config
[root@master ~]# sudo chown $(id -u):$(id -g) $HOME/.kube/config

```

下面的操作只需要在node节点上执行即可

```

kubeadm join 192.168.0.100:6443 --token awk15p.t6bamck54w69u4s8 \
    --discovery-token-ca-cert-hash
sha256:a94fa09562466d32d29523ab6cfff122186f1127599fa4dcd5fa0152694f17117

```

在master上查看节点信息

```

[root@master ~]# kubectl get nodes

```

NAME	STATUS	ROLES	AGE	VERSION
master	NotReady	master	6m	v1.17.4
node1	NotReady	<none>	22s	v1.17.4
node2	NotReady	<none>	19s	v1.17.4

2.6.13 安装网络插件，只在master节点操作即可

```

wget https://raw.githubusercontent.com/coreos/flannel/master/Documentation/kube-flannel.yml

```

由于外网不好访问，如果出现无法访问的情况，可以直接用下面的 记得文件名是kube-flannel.yml，位置：/root/kube-flannel.yml内容：

```

wget https://github.com/flannel-io/flannel/tree/master/Documentation/kube-flannel.yml

```

也可手动拉取指定版本

```
docker pull quay.io/coreos/flannel:v0.14.0      #拉取flannel网络，三台主机  
docker images      #查看仓库是否拉去下来
```

个人笔记

若是集群状态一直是 notready,用下面语句查看原因,

```
journalctl -f -u kubelet.service
```

若原因是: cni.go:237] Unable to update cni config: no networks found in /etc/cni/net.d

```
mkdir -p /etc/cni/net.d      #创建目录给flannel做配置文件
```

```
vim /etc/cni/net.d/10-flannel.conf      #编写配置文件
```

```
{  
  "name": "cbr0",  
  "cniVersion": "0.3.1",  
  "type": "flannel",  
  "delegated": {  
    "hairpinMode": true,  
    "isDefaultGateway": true  
  }  
}
```

2.6.14 使用kubeadm reset重置集群

```
#在master节点之外的节点进行操作  
kubeadm reset  
systemctl stop kubelet  
systemctl stop docker  
rm -rf /var/lib/cni/  
rm -rf /var/lib/kubelet/*  
rm -rf /etc/cni/  
ifconfig cni0 down  
ifconfig flannel.1 down  
ifconfig docker0 down  
ip link delete cni0  
ip link delete flannel.1  
##重启kubelet  
systemctl restart kubelet  
##重启docker  
systemctl restart docker
```

2.6.15 重启kubelet和docker

```
# 重启kubelet  
systemctl restart kubelet  
# 重启docker  
systemctl restart docker
```

使用配置文件启动flannel

```
kubect1 apply -f kube-flannel.yml
```

等待它安装完毕 发现已经是 集群的状态已经是Ready

```
[root@kubernetes-master ~]# kubectl get nodes
```

NAME	STATUS	ROLES	AGE	VERSION
kubernetes-master	Ready	master	12m	v1.19.1
kubernetes-node1	Ready	<none>	8m24s	v1.19.1
kubernetes-node2	Ready	<none>	8m17s	v1.19.1

2.6.16 kubeadm中的命令

```
# 生成 新的token
[root@master ~]# kubeadm token create --print-join-command
```

2.7 集群测试

2.7.1 创建一个nginx服务

```
kubectl create deployment nginx --image=nginx:1.14-alpine
```

2.7.2 暴露端口

```
kubectl expose deploy nginx --port=80 --target-port=80 --type=NodePort
```

2.7.3 查看服务

```
kubectl get pod,svc
```

2.7.4 查看pod

```
[root@kubernetes-master ~]# kubectl create deployment nginx --image=nginx:1.14-alpine
deployment.apps/nginx created
[root@kubernetes-master ~]# kubectl expose deploy nginx --port=80 --target-port=80 --type=NodePort
service/nginx exposed
[root@kubernetes-master ~]# kubectl get svc
```

NAME	TYPE	CLUSTER-IP	EXTERNAL-IP	PORT(S)	AGE
kubernetes	ClusterIP	10.96.0.1	<none>	443/TCP	17m
nginx	NodePort	10.103.93.152	<none>	80:31191/TCP	10s

```
[root@kubernetes-master ~]# ^C
[root@kubernetes-master ~]# kubectl get pod
```

NAME	READY	STATUS	RESTARTS	AGE
nginx-65c4bffc6b-cpk59	1/1	Running	0	74s

浏览器测试结果：

Welcome to nginx!

If you see this page, the nginx web server is successfully installed and working. Further configuration is required.

For online documentation and support please refer to nginx.org.
Commercial support is available at nginx.com.

Thank you for using nginx.

Welcome to nginx!

If you see this page, the nginx web server is successfully installed and working. Further configuration is required.

For online documentation and support please refer to nginx.org.
Commercial support is available at nginx.com.

Thank you for using nginx.

Welcome to nginx!

If you see this page, the nginx web server is successfully installed and working. Further configuration is required.

For online documentation and support please refer to nginx.org.
Commercial support is available at nginx.com.

Thank you for using nginx.

3. 资源管理

3.1 资源管理介绍

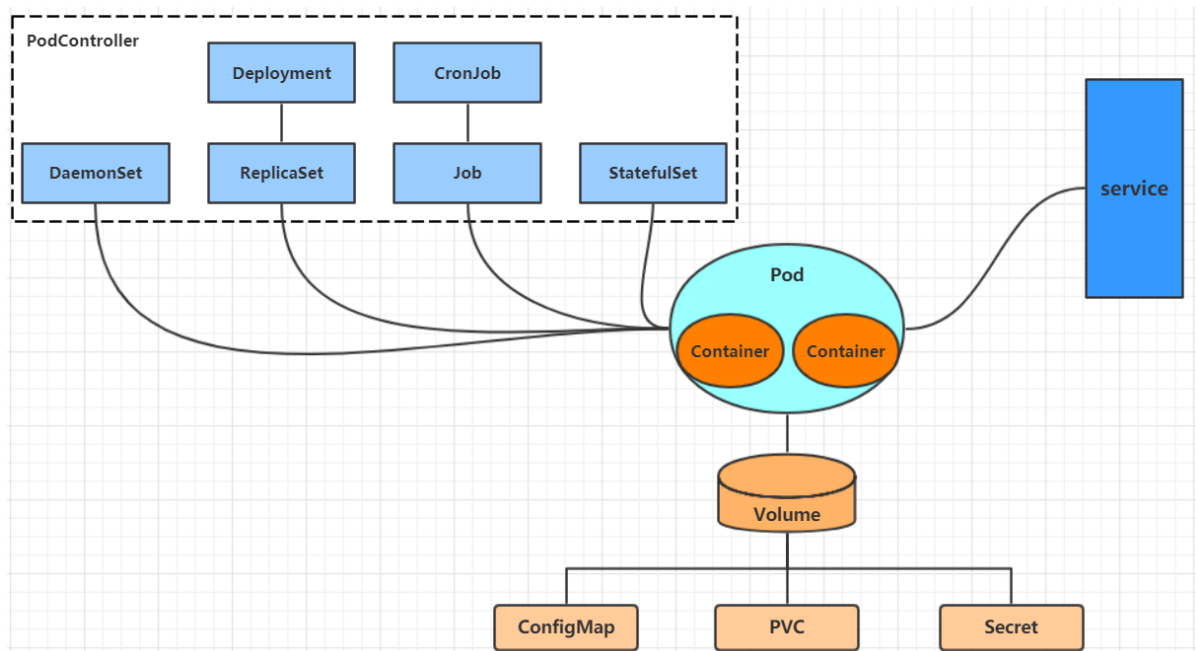
在kubernetes中，所有的内容都抽象为资源，用户需要通过操作资源来管理kubernetes。

kubernetes的本质就是一个集群系统，用户可以在集群中部署各种服务，所谓的部署服务，其实就是在kubernetes集群中运行一个个的容器，并将指定的程序跑在容器中。

kubernetes的最小管理单元是pod而不是容器，所以只能将容器放在 Pod 中，而kubernetes一般也不会直接管理Pod，而是通过 Pod控制器 来管理Pod的。

Pod可以提供服务之后，就要考虑如何访问Pod中服务，kubernetes提供了 Service 资源实现这个功能。

当然，如果Pod中程序的数据需要持久化，kubernetes还提供了各种 存储 系统。



学习kubernetes的核心，就是学习如何对集群上的 Pod、Pod控制器、Service、存储 等各种资源进行操作

3.2 YAML语言介绍

YAML是一个类似 XML、JSON 的标记性语言。它强调以**数据**为中心，并不是以标识语言为重点。因而YAML本身的定义比较简单，号称"一种人性化的数据格式语言"。

```
<jax>
  <age>15</age>
  <address>Beijing</address>
</jax>
```

```
jax:
  age: 15
  address: Beijing
```

YAML的语法比较简单，主要有下面几个：

- 大小写敏感
- 使用缩进表示层级关系
- 缩进不允许使用tab，只允许空格(低版本限制)
- 缩进的空格数不重要，只要相同层级的元素左对齐即可
- '#'表示注释

YAML支持以下几种数据类型：

- 纯量：单个的、不可再分的值
- 对象：键值对的集合，又称为映射 (mapping) / 哈希 (hash) / 字典 (dictionary)
- 数组：一组按次序排列的值，又称为序列 (sequence) / 列表 (list)

```
# 纯量，就是指的一个简单的值，字符串、布尔值、整数、浮点数、Null、时间、日期
# 1 布尔类型
c1: true (或者True)
```

```
# 2 整型
c2: 234
# 3 浮点型
c3: 3.14
# 4 null类型
c4: ~ # 使用~表示null
# 5 日期类型
c5: 2018-02-17 # 日期必须使用ISO 8601格式，即yyyy-MM-dd
# 6 时间类型
c6: 2018-02-17T15:02:31+08:00 # 时间使用ISO 8601格式，时间和日期之间使用T连接，最后使用+代表时区
# 7 字符串类型
c7: jax # 简单写法，直接写值，如果字符串中间有特殊字符，必须使用双引号或者单引号包裹
c8: line1
    line2 # 字符串过多的情况可以拆成多行，每一行会被转化成一个空格
```

```
# 对象
# 形式一(推荐):
jax:
  age: 15
  address: Beijing
# 形式二(了解):
jax: {age: 15,address: Beijing}
```

```
# 数组
# 形式一(推荐):
address:
  - 顺义
  - 昌平
# 形式二(了解):
address: [顺义,昌平]
```

小提示:

- 1 书写yaml切记：后面要加一个空格
- 2 如果需要将多段yaml配置放在一个文件中，中间要使用---分隔
- 3 下面是一个yaml转json的网站，可以通过它验证yaml是否书写正确

<https://www.json2yaml.com/convert-yaml-to-json>

3.3 资源管理方式

- 命令式对象管理：直接使用命令去操作kubernetes资源

```
kubectl run nginx-pod --image=nginx:1.17.1 --port=80
```

- 命令式对象配置：通过命令配置和配置文件去操作kubernetes资源

```
kubectl create/patch -f nginx-pod.yaml
```

- 声明式对象配置：通过apply命令和配置文件去操作kubernetes资源，创建或者更新资源

```
kubectl apply -f nginx-pod.yaml
```

类型	操作对象	适用环境	优点	缺点
命令式对象管理	对象	测试	简单	只能操作活动对象，无法审计、跟踪
命令式对象配置	文件	开发	可以审计、跟踪	项目大时，配置文件多，操作麻烦
声明式对象配置	目录	开发	支持目录操作	意外情况下难以调试

3.3.1 命令式对象管理

kubectl命令

kubectl是kubernetes集群的命令行工具，通过它能够对集群本身进行管理，并能够在集群上进行容器化应用的安装部署。kubectl命令的语法如下：

```
kubectl [command] [type] [name] [flags]
```

command：指定要对资源执行的操作，例如create、get、delete

type：指定资源类型，比如deployment、pod、service

name：指定资源的名称，名称大小写敏感

flags：指定额外的可选参数

```
# 查看所有pod
kubectl get pod

# 查看某个pod
kubectl get pod pod_name

# 查看某个pod,更宽查看
kubectl get pod pod_name -o wide

# 查看某个pod,以yaml格式展示结果
kubectl get pod pod_name -o yaml

# 查看某个pod,以json格式展示结果
kubectl get pod pod_name -o json

# 查看某个pod的信息
kubectl describe pod pod_name
```

```
[root@master ~]# kubectl get pod nginx-6867cdf567-jng5c -o wide
NAME                                READY    STATUS    RESTARTS   AGE    IP              NODE    NOMINATED NODE    READINESS GATES
nginx-6867cdf567-jng5c              1/1     Running   0           4d16h   10.244.1.2      node2    <none>             <none>
```

```
[root@master ~]# kubectl get pod nginx-6867cdf567-jng5c -o yaml
apiVersion: v1
kind: Pod
metadata:
  creationTimestamp: "2022-04-02T09:58:09Z"
  generateName: nginx-6867cdf567-
  labels:
    app: nginx
    pod-template-hash: 6867cdf567
  name: nginx-6867cdf567-jng5c
  namespace: default
  ownerReferences:
    - apiVersion: apps/v1
      blockOwnerDeletion: true
      controller: true
      kind: ReplicaSet
      name: nginx-6867cdf567
      uid: 6ffbc927-9745-415a-97b7-ee8cd0b03744
  resourceVersion: "6407"
  selfLink: /api/v1/namespaces/default/pods/nginx-6867cdf567-jng5c
  uid: a27ad776-3fd1-47f9-abbb-d3b8a2500d7f
spec:
  containers:
    - image: nginx:1.14-alpine
      imagePullPolicy: IfNotPresent
      name: nginx
      resources: {}
      terminationMessagePath: /dev/termination-log
      terminationMessagePolicy: File
      volumeMounts:
        - mountPath: /var/run/secrets/kubernetes.io/serviceaccount
          name: default-token-nm896
          readOnly: true
  dnsPolicy: ClusterFirst
  enableServiceLinks: true
  nodeName: node2
  priority: 0
  restartPolicy: Always
  schedulerName: default-scheduler
  securityContext: {}
  serviceAccount: default
```



```
nginx-6867cdf567-jng5c 1/1 Running 0 nginx-10.2.1.12 node2
[root@master ~]# kubectl get pod nginx-6867cdf567-jng5c -o json
{
  "apiVersion": "v1",
  "kind": "Pod",
  "metadata": {
    "creationTimestamp": "2022-04-02T09:58:09Z",
    "generateName": "nginx-6867cdf567-",
    "labels": {
      "app": "nginx",
      "pod-template-hash": "6867cdf567"
    },
    "name": "nginx-6867cdf567-jng5c",
    "namespace": "default",
    "ownerReferences": [
      {
        "apiVersion": "apps/v1",
        "blockOwnerDeletion": true,
        "controller": true,
        "kind": "ReplicaSet",
        "name": "nginx-6867cdf567",
        "uid": "6ffbc927-9745-415a-97b7-ee8cd0b03744"
      }
    ],
    "resourceVersion": "6407",
    "selfLink": "/api/v1/namespaces/default/pods/nginx-6867cdf567-jng5c",
    "uid": "a27ad776-3fd1-47f9-abbb-d3b8a2500d7f"
  },
  "spec": {
    "containers": [
      {
        "image": "nginx:1.14-alpine",
        "imagePullPolicy": "IfNotPresent",
        "name": "nginx",
        "resources": {},
        "terminationMessagePath": "/dev/termination-log",
        "terminationMessagePolicy": "File",
        "volumeMounts": [
          {

```

```
[root@master ~]# kubectl describe pod nginx-6867cdf567-jng5c
Name:          nginx-6867cdf567-jng5c
Namespace:     default
Priority:       0
Node:          node2/192.168.5.102
Start Time:    Sat, 02 Apr 2022 17:58:09 +0800
Labels:        app=nginx
               pod-template-hash=6867cdf567
Annotations:   <none>
Status:        Running
IP:            10.244.1.2
IPs:
  IP:          10.244.1.2
Controlled By: ReplicaSet/nginx-6867cdf567
Containers:
  nginx:
    Container ID:  docker://56c0bfe388888a2fb850d6eaf9afefba7706f4ea6f7c7f4a0ad5121d1a081cc8
    Image:         nginx:1.14-alpine
    Image ID:      docker-pullable://nginx@sha256:485b610fefec7ff6c463ced9623314a04ed67e3945b9c08d7e53a47f6d1
    Port:          <none>
    Host Port:     <none>
    State:         Running
      Started:     Sat, 02 Apr 2022 17:58:39 +0800
      Ready:       True
      Restart Count: 0
    Environment:   <none>
    Mounts:
      /var/run/secrets/kubernetes.io/serviceaccount from default-token-nm896 (ro)
Conditions:
  Type          Status
  Initialized    True
  Ready         True
  ContainersReady True
  PodScheduled   True
Volumes:
  default-token-nm896:
    Type:          Secret (a volume populated by a Secret)
    SecretName:     default-token-nm896
    Optional:       false
QoS Class:        BestEffort
Node-Selectors:   <none>
Tolerations:      node.kubernetes.io/not-ready:NoExecute for 300s
                  node.kubernetes.io/unreachable:NoExecute for 300s
Events:
  Type     Reason      Age   From          Message
  ----     -
  Normal   Scheduled   4d17h default-scheduler Successfully assigned default/nginx-6867cdf567-jng5c to node2
  Normal   Pulling     4d17h kubelet, node2 Pulling image "nginx:1.14-alpine"
  Normal   Pulled      4d17h kubelet, node2 Successfully pulled image "nginx:1.14-alpine"
```

资源类型

kubernetes中所有的内容都抽象为资源，可以通过下面的命令进行查看:

```
kubectl api-resources
```

经常使用的资源有下面这些:

资源分类	资源名称	缩写	资源作用
集群级别资源	nodes	no	集群组成部分
命名空间	namespaces	ns	隔离Pod
pod资源	pods	po	装载容器
pod资源控制器	replicationcontrollers	rc	控制pod资源
	replicasets	rs	控制pod资源
	deployments	deploy	控制pod资源
	daemonsets	ds	控制pod资源
	jobs		控制pod资源
	cronjobs	cj	控制pod资源

资源分类	资源名称	缩写	资源作用
	horizontalpodautoscalers	hpa	控制pod资源
	statefulsets	sts	控制pod资源
服务发现资源	services	svc	统一pod对外接口
	ingress	ing	统一pod对外接口
存储资源	volumeattachments		存储
	persistentvolumes	pv	存储
	persistentvolumeclaims	pvc	存储
配置资源	configmaps	cm	配置
	secrets		配置

操作

kubernetes允许对资源进行多种操作，可以通过--help查看详细的操作命令

```
kubect1 --help
```

经常使用的操作有下面这些：

命令分类	命令	翻译	命令作用
基本命令	create	创建	创建一个资源
	edit	编辑	编辑一个资源
	get	获取	获取一个资源
	patch	更新	更新一个资源
	delete	删除	删除一个资源
	explain	解释	展示资源文档
运行和调试	run	运行	在集群中运行一个指定的镜像
	expose	暴露	暴露资源为Service
	describe	描述	显示资源内部信息
	logs	日志输出容器在 pod 中的日志	输出容器在 pod 中的日志
	attach	缠绕进入运行中的容器	进入运行中的容器
	exec	执行容器中的一个命令	执行容器中的一个命令
	cp	复制	在Pod内外复制文件
	rollout	首次展示	管理资源的发布
	scale	规模	扩(缩)容Pod的数量

命令分类	命令	翻译	命令作用
	autoscale	自动调整	自动调整Pod的数量
高级命令	apply	rc	通过文件对资源进行配置
	label	标签	更新资源上的标签
其他命令	cluster-info	集群信息	显示集群信息
	version	版本	显示当前Server和Client的版本

下面以一个namespace / pod的创建和删除简单演示下命令的使用：

```
# 创建一个namespace
[root@master ~]# kubectl create namespace dev
namespace/dev created

# 获取namespace
[root@master ~]# kubectl get ns
NAME                STATUS    AGE
default             Active    21h
dev                 Active    21s
kube-node-lease     Active    21h
kube-public         Active    21h
kube-system         Active    21h

# 在此namespace下创建并运行一个nginx的Pod
[root@master ~]# kubectl run pod --image=nginx:latest -n dev
kubectl run --generator=deployment/apps.v1 is DEPRECATED and will be removed in a
future version. Use kubectl run --generator=run-pod/v1 or kubectl create instead.
deployment.apps/pod created

# 查看新创建的pod
[root@master ~]# kubectl get pod -n dev
NAME    READY   STATUS    RESTARTS   AGE
pod     1/1     Running   0           21s

# 删除指定的pod
[root@master ~]# kubectl delete pod pod-864f9875b9-pcw7x
pod "pod" deleted

# 删除指定的namespace
[root@master ~]# kubectl delete ns dev
namespace "dev" deleted
```

3.3.2 命令式对象配置

命令式对象配置就是使用命令配合配置文件一起来操作kubernetes资源。

1) 创建一个nginxpod.yaml，内容如下：

```
apiVersion: v1
kind: Namespace
metadata:
```

```
name: dev

---

apiVersion: v1
kind: Pod
metadata:
  name: nginxpod
  namespace: dev
spec:
  containers:
  - name: nginx-containers
    image: nginx:latest
```

2) 执行create命令，创建资源：

```
[root@master ~]# kubectl create -f nginxpod.yaml
namespace/dev created
pod/nginxpod created
```

此时发现创建了两个资源对象，分别是namespace和pod

3) 执行get命令，查看资源：

```
[root@master ~]# kubectl get -f nginxpod.yaml
```

NAME	STATUS	AGE
namespace/dev	Active	18s

NAME	READY	STATUS	RESTARTS	AGE
pod/nginxpod	1/1	Running	0	17s

这样就显示了两个资源对象的信息

4) 执行delete命令，删除资源：

```
[root@master ~]# kubectl delete -f nginxpod.yaml
namespace "dev" deleted
pod "nginxpod" deleted
```

此时发现两个资源对象被删除了

总结：

命令式对象配置的方式操作资源，可以简单的认为：命令 + yaml配置文件（里面是命令需要的各种参数）

3.3.3 声明式对象配置

声明式对象配置跟命令式对象配置很相似，但是它只有一个命令apply。

```
# 首先执行一次kubectl apply -f yaml文件，发现创建了资源
[root@master ~]# kubectl apply -f nginxpod.yaml
namespace/dev created
pod/nginxpod created

# 再次执行一次kubectl apply -f yaml文件，发现说资源没有变动
[root@master ~]# kubectl apply -f nginxpod.yaml
namespace/dev unchanged
pod/nginxpod unchanged
```

总结：

其实声明式对象配置就是使用apply描述一个资源最终的状态（在yaml中定义状态）

使用apply操作资源：

如果资源不存在，就创建，相当于 kubectl create

如果资源已存在，就更新，相当于 kubectl patch

扩展：kubectl可以在node节点上运行吗？

kubectl的运行是需要进行配置的，它的配置文件是\$HOME/.kube，如果想要在node节点运行此命令，需要将master上的.kube文件复制到node节点上，即在master节点上执行下面操作：

```
scp -r HOME/.kube node1: HOME/
```

使用推荐：三种方式应该怎么用？

创建/更新资源 使用声明式对象配置 kubectl apply -f XXX.yaml

删除资源 使用命令式对象配置 kubectl delete -f XXX.yaml

查询资源 使用命令式对象管理 kubectl get(describe) 资源名称

4. 实战入门

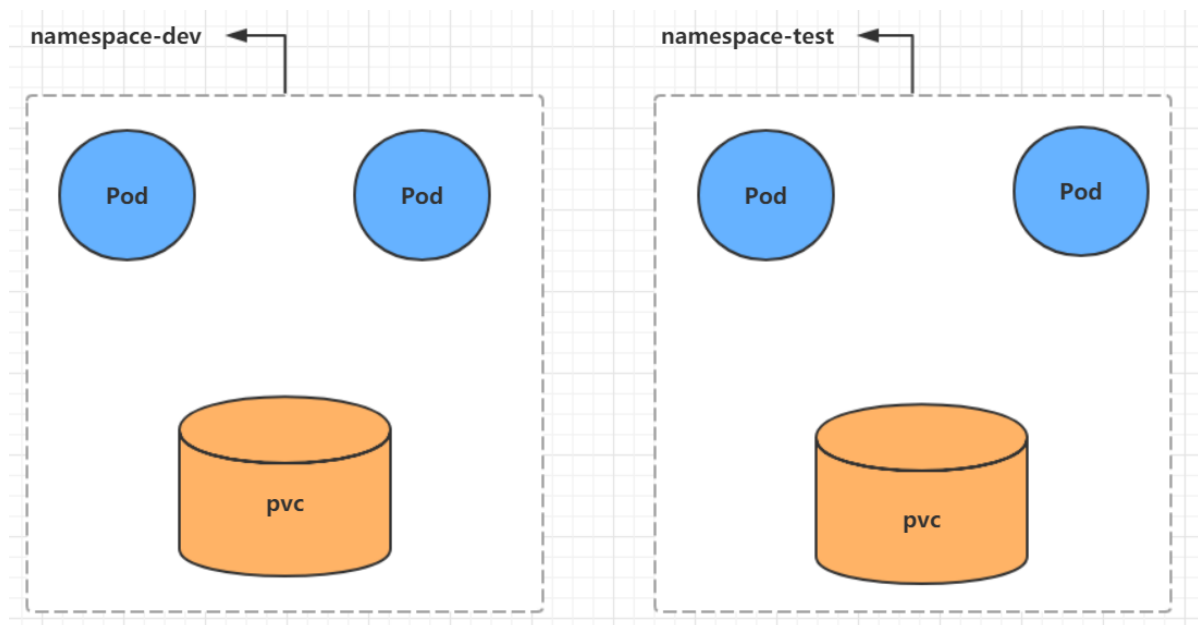
本章节将介绍如何在kubernetes集群中部署一个nginx服务，并且能够对其进行访问。

4.1 Namespace

Namespace是kubernetes系统中的一种非常重要资源，它的主要作用是用来实现**多套环境的资源隔离**或者**多租户的资源隔离**。

默认情况下，kubernetes集群中的所有的Pod都是可以相互访问的。但是在实际中，可能不想让两个Pod之间进行互相的访问，那此时就可以将两个Pod划分到不同的namespace下。kubernetes通过将集群内部的资源分配到不同的Namespace中，可以形成逻辑上的“组”，以方便不同的组的资源进行隔离使用和管理。

可以通过kubernetes的授权机制，将不同的namespace交给不同租户进行管理，这样就实现了多租户的资源隔离。此时还能结合kubernetes的资源配额机制，限定不同租户能占用的资源，例如CPU使用量、内存使用量等等，来实现租户可用资源的管理。



kubernetes在集群启动之后，会默认创建几个namespace

```
[root@master ~]# kubectl get namespace
```

NAME	STATUS	AGE	
default	Active	45h	# 所有未指定Namespace的对象都会被分配在default命名空间
kube-node-lease	Active	45h	# 集群节点之间的心跳维护，v1.13开始引入
kube-public	Active	45h	# 此命名空间下的资源可以被所有人访问（包括未认证用户）
kube-system	Active	45h	# 所有由Kubernetes系统创建的资源都处于这个命名空间

```
[root@master ~]# kubectl get pods -n kube-system
```

NAME	READY	STATUS	RESTARTS	AGE
coredns-9d85f5447-ndj5n	0/1	Running	0	4d21h
coredns-9d85f5447-r4kpx	0/1	Running	0	4d21h
etcd-master	1/1	Running	0	4d21h
kube-apiserver-master	1/1	Running	0	4d21h
kube-controller-manager-master	1/1	Running	0	4d21h
kube-flannel-ds-2l7nm	1/1	Running	0	4d20h
kube-flannel-ds-c9mb1	1/1	Running	0	4d20h
kube-flannel-ds-rjd5w	1/1	Running	0	4d20h
kube-proxy-2zvzv	1/1	Running	0	4d21h
kube-proxy-95h26	1/1	Running	0	4d21h
kube-proxy-h27h7	1/1	Running	0	4d21h
kube-scheduler-master	1/1	Running	0	4d21h

下面来看namespace资源的具体操作：

4.1.1 查看

```
# 1 查看所有的ns 命令: kubectl get ns
```

```
[root@master ~]# kubectl get ns
```

NAME	STATUS	AGE
default	Active	45h
kube-node-lease	Active	45h
kube-public	Active	45h
kube-system	Active	45h

```
# 2 查看指定的ns 命令: kubectl get ns ns名称
[root@master ~]# kubectl get ns default
NAME          STATUS    AGE
default       Active    45h

# 3 指定输出格式 命令: kubectl get ns ns名称 -o 格式参数
# kubernetes支持的格式有很多, 比较常见的是wide、json、yaml
[root@master ~]# kubectl get ns default -o yaml
apiVersion: v1
kind: Namespace
metadata:
  creationTimestamp: "2021-05-08T04:44:16Z"
  name: default
  resourceVersion: "151"
  selfLink: /api/v1/namespaces/default
  uid: 7405f73a-e486-43d4-9db6-145f1409f090
spec:
  finalizers:
  - kubernetes
status:
  phase: Active

# 4 查看ns详情 命令: kubectl describe ns ns名称
[root@master ~]# kubectl describe ns default
Name:          default
Labels:         <none>
Annotations:    <none>
Status:        Active # Active 命名空间正在使用中 Terminating 正在删除命名空间

# ResourceQuota 针对namespace做的资源限制
# LimitRange针对namespace中的每个组件做的资源限制
No resource quota.
No LimitRange resource.
```

4.1.2 创建

```
# 创建namespace
[root@master ~]# kubectl create ns dev
namespace/dev created
```

4.1.3 删除

```
# 删除namespace
[root@master ~]# kubectl delete ns dev
namespace "dev" deleted
```

4.1.4 配置方式

首先准备一个yaml文件: ns-dev.yaml

```
apiVersion: v1
kind: Namespace
metadata:
  name: dev
```


然后就可以执行对应的创建和删除命令了：

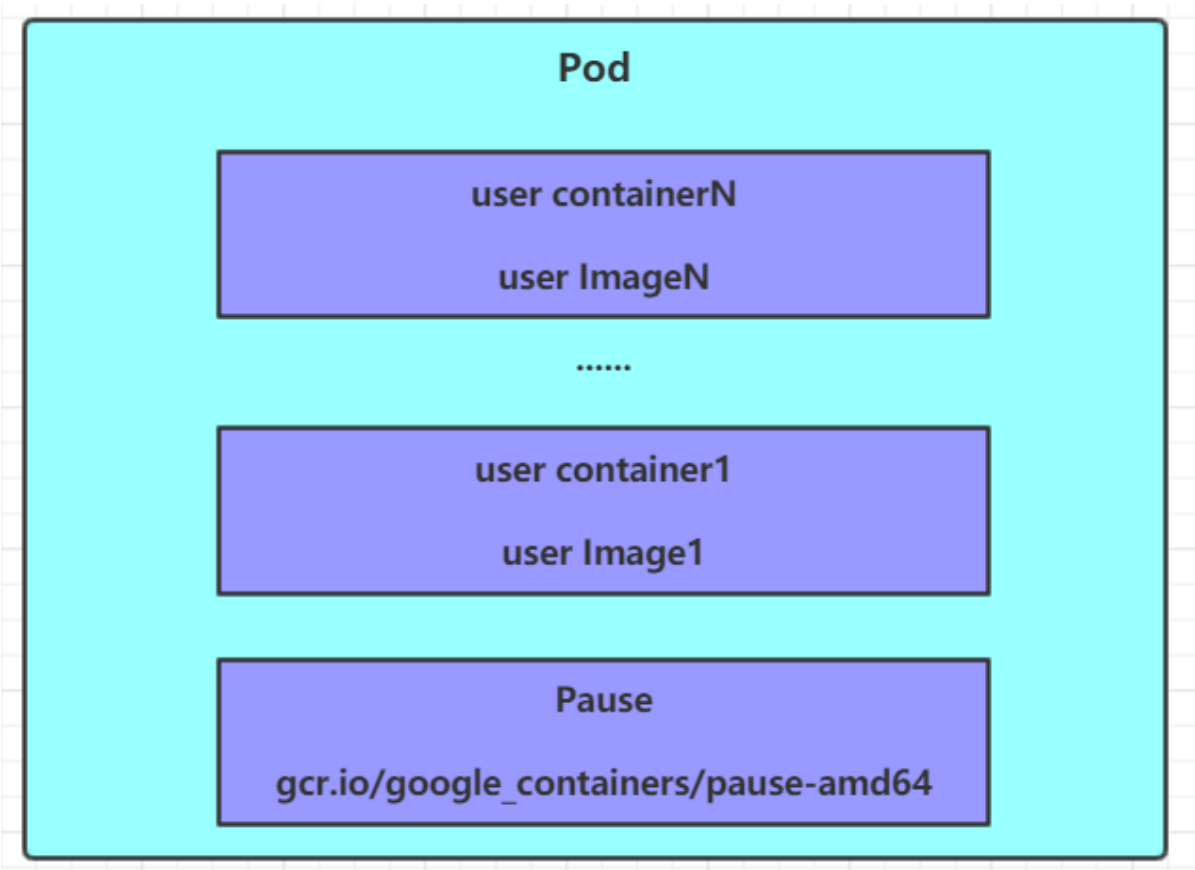
创建：kubectl create -f ns-dev.yaml

删除：kubectl delete -f ns-dev.yaml

4.2 Pod

Pod是kubernetes集群进行管理的最小单元，程序要运行必须部署在容器中，而容器必须存在于Pod中。

Pod可以认为是容器的封装，一个Pod中可以存在一个或者多个容器。



kubernetes在集群启动之后，集群中的各个组件也都是以Pod方式运行的。可以通过下面命令查看：

```
[root@master ~]# kubectl get pod -n kube-system
```

NAMESPACE	NAME	READY	STATUS	RESTARTS	AGE
kube-system	coredns-6955765f44-68g6v	1/1	Running	0	2d1h
kube-system	coredns-6955765f44-cs5r8	1/1	Running	0	2d1h
kube-system	etcd-master	1/1	Running	0	2d1h
kube-system	kube-apiserver-master	1/1	Running	0	2d1h
kube-system	kube-controller-manager-master	1/1	Running	0	2d1h
kube-system	kube-flannel-ds-amd64-47r25	1/1	Running	0	2d1h
kube-system	kube-flannel-ds-amd64-1s5lh	1/1	Running	0	2d1h
kube-system	kube-proxy-685tk	1/1	Running	0	2d1h
kube-system	kube-proxy-87spt	1/1	Running	0	2d1h
kube-system	kube-scheduler-master	1/1	Running	0	2d1h

4.2.1 创建并运行

kubernetes没有提供单独运行Pod的命令，都是通过Pod控制器来实现的

```
# 命令格式: kubectl run (pod控制器名称) [参数]
# --image 指定Pod的镜像
# --port 指定端口
# --namespace 指定namespace
[root@master ~]# kubectl run nginx --image=nginx:latest --port=80 --namespace dev
deployment.apps/nginx created
```

4.2.2 查看pod信息

```
# 查看Pod基本信息
[root@master ~]# kubectl get pods -n dev
NAME      READY   STATUS    RESTARTS   AGE
nginx     1/1     Running   0           43s

# 查看Pod的详细信息
[root@master ~]# kubectl describe pod nginx -n dev
Name:      nginx
Namespace: dev
Priority:   0
Node:      node1/192.168.5.4
Start Time: Wed, 08 May 2021 09:29:24 +0800
Labels:    pod-template-hash=5ff7956ff6
           run=nginx
Annotations: <none>
Status:     Running
IP:         10.244.1.23
IPs:
  IP:       10.244.1.23
Controlled By: ReplicaSet/nginx
Containers:
  nginx:
    Container ID:
      docker://4c62b8c0648d2512380f4ffa5da2c99d16e05634979973449c98e9b829f6253c
    Image:      nginx:latest
    Image ID:   docker-
pullable://nginx@sha256:485b610fefec7ff6c463ced9623314a04ed67e3945b9c08d7e53a47f6d108dc7
    Port:      80/TCP
    Host Port: 0/TCP
    State:     Running
      Started: Wed, 08 May 2021 09:30:01 +0800
    Ready:     True
    Restart Count: 0
    Environment: <none>
    Mounts:
      /var/run/secrets/kubernetes.io/serviceaccount from default-token-hwvww (ro)
Conditions:
  Type           Status
  Initialized    True
  Ready          True
  ContainersReady True
```

```

PodScheduled      True
Volumes:
  default-token-hwvww:
    Type:          Secret (a volume populated by a Secret)
    SecretName:     default-token-hwvww
    Optional:      false
QoS Class:        BestEffort
Node-Selectors:    <none>
Tolerations:       node.kubernetes.io/not-ready:NoExecute for 300s
                   node.kubernetes.io/unreachable:NoExecute for 300s

Events:
  Type    Reason      Age    From          Message
  ----    -
  Normal  Scheduled   <unknown> default-scheduler Successfully assigned
dev/nginx-5ff7956ff6-fg2db to node1
  Normal  Pulling     4m11s  kubelet, node1 Pulling image "nginx:latest"
  Normal  Pulled      3m36s  kubelet, node1 Successfully pulled image
"nginx:latest"
  Normal  Created     3m36s  kubelet, node1 Created container nginx
  Normal  Started     3m36s  kubelet, node1 Started container nginx

```

4.2.3 访问Pod

```

# 获取podIP
[root@master ~]# kubectl get pods -n dev -o wide
NAME      READY   STATUS    RESTARTS   AGE    IP             NODE   ...
nginx     1/1     Running   0           190s   10.244.1.23    node1   ...

#访问POD
[root@master ~]# curl http://10.244.1.23:80
<!DOCTYPE html>
<html>
<head>
  <title>welcome to nginx!</title>
</head>
<body>
  <p><em>Thank you for using nginx.</em></p>
</body>
</html>

```

4.2.4 删除指定Pod

```

# 删除指定Pod
[root@master ~]# kubectl delete pod nginx -n dev
pod "nginx" deleted

# 此时，显示删除Pod成功，但是再查询，发现又新产生了一个
[root@master ~]# kubectl get pods -n dev
NAME      READY   STATUS    RESTARTS   AGE
nginx     1/1     Running   0           21s

# 这是因为当前Pod是由Pod控制器创建的，控制器会监控Pod状况，一旦发现Pod死亡，会立即重建
# 此时要想删除Pod，必须删除Pod控制器

# 先来查询一下当前namespace下的Pod控制器

```

```
[root@master ~]# kubectl get deploy -n dev
NAME      READY   UP-TO-DATE   AVAILABLE   AGE
nginx     1/1     1             1           9m7s

# 接下来，删除此PodPod控制器
[root@master ~]# kubectl delete deploy nginx -n dev
deployment.apps "nginx" deleted

# 稍等片刻，再查询Pod，发现Pod被删除了
[root@master ~]# kubectl get pods -n dev
No resources found in dev namespace.
```

4.2.5 配置操作

创建一个pod-nginx.yaml，内容如下：

```
apiVersion: v1
kind: Pod
metadata:
  name: nginx
  namespace: dev
spec:
  containers:
  - image: nginx:latest
    name: pod
    ports:
    - name: nginx-port
      containerPort: 80
      protocol: TCP
```

然后就可以执行对应的创建和删除命令了：

创建：kubectl create -f pod-nginx.yaml

删除：kubectl delete -f pod-nginx.yaml

4.3 Label

Label是kubernetes系统中的一个重要概念。它的作用就是在资源上添加标识，用来对它们进行区分和选择。

Label的特点：

- 一个Label会以key/value键值对的形式附加到各种对象上，如Node、Pod、Service等等
- 一个资源对象可以定义任意数量的Label，同一个Label也可以被添加到任意数量的资源对象上去
- Label通常在资源对象定义时确定，当然也可以在对象创建后动态添加或者删除

可以通过Label实现资源的多维度分组，以便灵活、方便地进行资源分配、调度、配置、部署等管理工作。

一些常用的Label 示例如下：

- 版本标签: "version":"release", "version":"stable".....
- 环境标签: "environment":"dev", "environment":"test", "environment":"pro"
- 架构标签: "tier":"frontend", "tier":"backend"

标签定义完毕之后，还要考虑到标签的选择，这就要使用到Label Selector，即：

Label用于给某个资源对象定义标识

Label Selector用于查询和筛选拥有某些标签的资源对象

当前有两种Label Selector：

- 基于等式的Label Selector
 - name = slave: 选择所有包含Label中key="name"且value="slave"的对象
 - env != production: 选择所有包括Label中的key="env"且value不等于"production"的对象
- 基于集合的Label Selector
 - name in (master, slave): 选择所有包含Label中的key="name"且value="master"或"slave"的对象
 - name not in (frontend): 选择所有包含Label中的key="name"且value不等于"frontend"的对象

标签的选择条件可以使用多个，此时将多个Label Selector进行组合，使用逗号","进行分隔即可。例如：

name=slave, env!=production

name not in (frontend), env!=production

4.3.1 命令方式

```
# 为pod资源打标签
[root@master ~]# kubectl label pod nginx-pod version=1.0 -n dev
pod/nginx-pod labeled

# 为pod资源更新标签
[root@master ~]# kubectl label pod nginx-pod version=2.0 -n dev --overwrite
pod/nginx-pod labeled

# 查看标签
[root@master ~]# kubectl get pod nginx-pod -n dev --show-labels
NAME          READY   STATUS    RESTARTS   AGE   LABELS
nginx-pod     1/1     Running   0           10m   version=2.0

# 筛选标签
[root@master ~]# kubectl get pod -n dev -l version=2.0 --show-labels
NAME          READY   STATUS    RESTARTS   AGE   LABELS
nginx-pod     1/1     Running   0           17m   version=2.0
[root@master ~]# kubectl get pod -n dev -l version!=2.0 --show-labels
No resources found in dev namespace.

#删除标签
[root@master ~]# kubectl label pod nginx-pod version- -n dev
pod/nginx-pod labeled
```

4.3.2 配置方式

```
apiVersion: v1
kind: Pod
metadata:
  name: nginx
  namespace: dev
  labels:
```

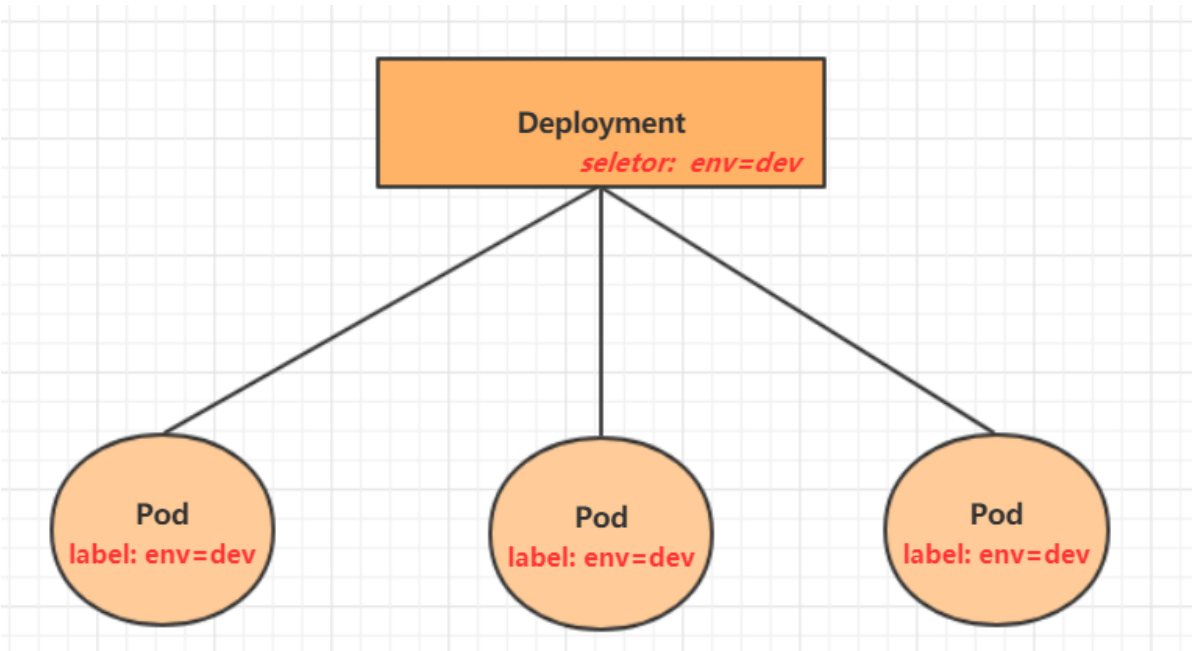
```
version: "3.0"
env: "test"
spec:
  containers:
  - image: nginx:latest
    name: pod
    ports:
    - name: nginx-port
      containerPort: 80
      protocol: TCP
```

然后就可以执行对应的更新命令了：`kubectl apply -f pod-nginx.yaml`

4.4 Deployment

在kubernetes中，Pod是最小的控制单元，但是kubernetes很少直接控制Pod，一般都是通过Pod控制器来完成的。Pod控制器用于pod的管理，确保pod资源符合预期的状态，当pod的资源出现故障时，会尝试进行重启或重建pod。

在kubernetes中Pod控制器的种类有很多，本章节只介绍一种：Deployment。



4.4.1 命令操作

```
# 命令格式: kubectl create deployment 名称 [参数]
# --image 指定pod的镜像
# --port 指定端口
# --replicas 指定创建pod数量
# --namespace 指定namespace
[root@master ~]#旧版本的k8s kubectl run nginx --image=nginx:latest --port=80 --replicas=3 -n dev
deployment.apps/nginx created
#新版的k8s, 自己新建的命名空间, kubectl run 不会创建deployment, 需要用下面的命令执行
[root@master ~] kubectl create deployment nginx --image=nginx:latest --port=80 --replicas=3 -n dev

# 查看创建的Pod
[root@master ~]# kubectl get pods -n dev
```

NAME	READY	STATUS	RESTARTS	AGE
------	-------	--------	----------	-----

```
nginx-5ff7956ff6-6k8cb 1/1 Running 0 19s
nginx-5ff7956ff6-jxfjt 1/1 Running 0 19s
nginx-5ff7956ff6-v6jqw 1/1 Running 0 19s
```

查看deployment的信息

```
[root@master ~]# kubectl get deploy -n dev
```

NAME	READY	UP-TO-DATE	AVAILABLE	AGE
nginx	3/3	3	3	2m42s

UP-TO-DATE: 成功升级的副本数量

AVAILABLE: 可用副本的数量

```
[root@master ~]# kubectl get deploy -n dev -o wide
```

NAME	READY	UP-TO-DATE	AVAILABLE	AGE	CONTAINERS	IMAGES
nginx	3/3	3	3	2m51s	nginx	nginx:latest

run=nginx

查看deployment的详细信息

```
[root@master ~]# kubectl describe deploy nginx -n dev
```

```
Name: nginx
Namespace: dev
CreationTimestamp: wed, 08 May 2021 11:14:14 +0800
Labels: run=nginx
Annotations: deployment.kubernetes.io/revision: 1
Selector: run=nginx
Replicas: 3 desired | 3 updated | 3 total | 3 available | 0
unavailable
StrategyType: RollingUpdate
MinReadySeconds: 0
RollingUpdateStrategy: 25% max unavailable, 25% max surge
Pod Template:
  Labels: run=nginx
  Containers:
    nginx:
      Image: nginx:latest
      Port: 80/TCP
      Host Port: 0/TCP
      Environment: <none>
      Mounts: <none>
      Volumes: <none>
```

Conditions:

Type	Status	Reason
Available	True	MinimumReplicasAvailable
Progressing	True	NewReplicaSetAvailable

OldReplicaSets: <none>

NewReplicaSet: nginx-5ff7956ff6 (3/3 replicas created)

Events:

Type	Reason	Age	From	Message
Normal	ScalingReplicaSet	5m43s	deployment-controller	Scaled up replicaset nginx-5ff7956ff6 to 3

删除

```
[root@master ~]# kubectl delete deploy nginx -n dev
```

deployment.apps "nginx" deleted

4.4.2 配置操作

创建一个deploy-nginx.yaml, 内容如下:

```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: nginx
  namespace: dev
spec:
  replicas: 3
  selector:
    matchLabels:
      run: nginx
  template:
    metadata:
      labels:
        run: nginx
    spec:
      containers:
      - image: nginx:latest
        name: nginx
        ports:
        - containerPort: 80
          protocol: TCP
```

yaml介绍

```
apiVersion: apps/v1
kind: Deployment #创建是 deployment
metadata:
  name: nginx
  namespace: dev
spec:
  replicas: 3
  selector: #创建选择器, 和label标签为 run: nginx的pod做关联
    matchLabels:
      run: nginx
  template: #创建pod模板
    metadata: #pod的元数据
      labels: #pod的标签
        run: nginx
    spec: #描述
      containers:
      - image: nginx:latest
        name: nginx
        ports:
        - containerPort: 80
          protocol: TCP
```


然后就可以执行对应的创建和删除命令了：

创建：kubectl create -f deploy-nginx.yaml

删除：kubectl delete -f deploy-nginx.yaml

4.4.3 集群内部访问

```
[root@master ~]# kubectl create -f deploy-nginx.yaml
deployment.apps/nginx created
[root@master ~]# kubectl get deployment,pod -n dev
```

NAME	READY	UP-TO-DATE	AVAILABLE	AGE
deployment.apps/nginx	3/3	3	3	20s


```
[root@master ~]# kubectl get pod -n dev -o wide
```

NAME	READY	STATUS	RESTARTS	AGE	IP	NODE
pod/nginx-dd6b5d745-5jzdv	1/1	Running	0	20s		
pod/nginx-dd6b5d745-h2nb6	1/1	Running	0	20s		
pod/nginx-dd6b5d745-krvgp	1/1	Running	0	20s		


```
[root@master ~]# kubectl get pod -n dev -o wide
```

NAME	READY	STATUS	RESTARTS	AGE	IP	NODE
nginx-dd6b5d745-5jzdv	1/1	Running	0	38s	10.244.2.6	node1
nginx-dd6b5d745-h2nb6	1/1	Running	0	38s	10.244.2.5	node1
nginx-dd6b5d745-krvgp	1/1	Running	0	38s	10.244.1.4	node2


```
[root@master ~]# curl 10.244.2.6:80
```

```
<!DOCTYPE html>
<html>
<head>
<title>welcome to nginx!</title>
<style>
html { color-scheme: light dark; }
body { width: 35em; margin: 0 auto;
font-family: Tahoma, Verdana, Arial, sans-serif; }
</style>
</head>
<body>
<h1>welcome to nginx!</h1>
<p>If you see this page, the nginx web server is successfully installed and
working. Further configuration is required.</p>

<p>For online documentation and support please refer to
<a href="http://nginx.org/">nginx.org</a>.<br/>
Commercial support is available at
<a href="http://nginx.com/">nginx.com</a>.</p>

<p><em>Thank you for using nginx.</em></p>
</body>
</html>
[root@master ~]#
```

4.5 Service(集群外部访问)

通过上节课的学习，已经能够利用Deployment来创建一组Pod来提供具有高可用性的服务。

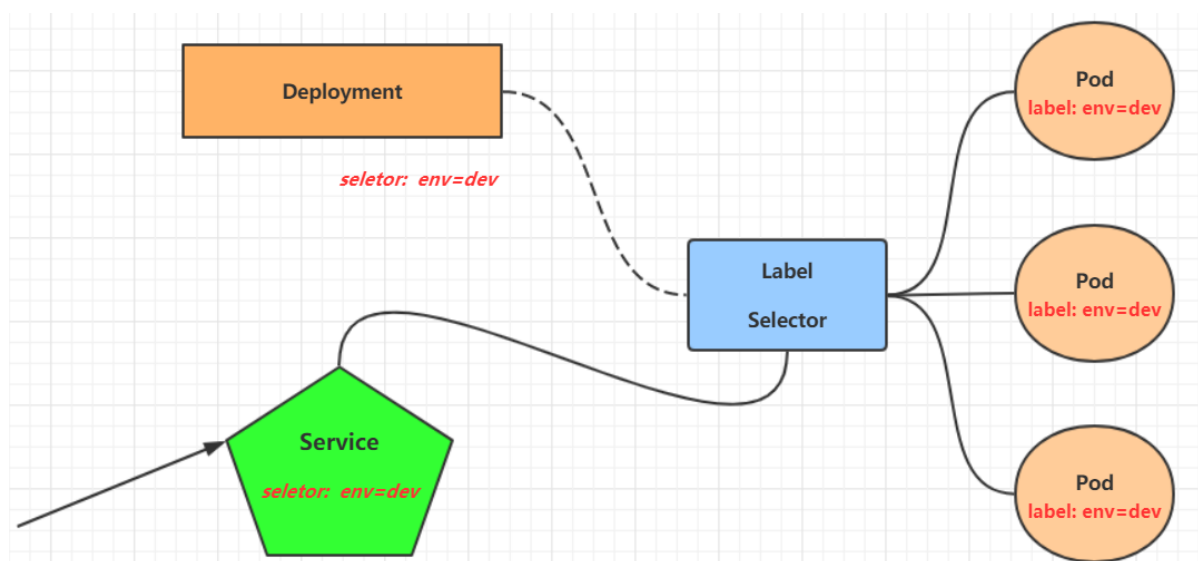
虽然每个Pod都会分配一个单独的Pod IP，然而却存在如下两问题：

- Pod IP 会随着Pod的重建产生变化
- Pod IP 仅仅是集群内可见的虚拟IP，外部无法访问

这样对于访问这个服务带来了难度。因此，kubernetes设计了Service来解决这个问题。

Service可以看作是一组同类Pod**对外的访问接口**。借助Service，应用可以方便地实现服务发现和负载均衡。

Service根据标签选择器selector，来找到对应的pod，service不删，ip地址不变，所以能够为外部提供一个稳定的ip地址，不会受到pod的创建，扩容，收缩等影响，导致访问不到集群内部的pod



4.5.1 创建集群内部可访问的Service

```
# 暴露Service，这里是借助 deployment，来找到对应的pod，实际中，可以直接通过 selector选择器关联pod
# 其中 --port是service端口 --target-port 是目标端口
[root@master ~]# kubectl expose deploy nginx --name=svc-nginx1 --type=ClusterIP --port=80 --target-port=80 -n dev
service/svc-nginx1 exposed

# 查看service
[root@master ~]# kubectl get svc svc-nginx1 -n dev -o wide
NAME          TYPE          CLUSTER-IP      EXTERNAL-IP      PORT(S)      AGE
SELECTOR
svc-nginx1    ClusterIP     10.109.179.231  <none>           80/TCP       3m51s
run=nginx

# 问题：svc访问很慢，大概一分做钟才通
# 使用命令，临时关闭Checksum 校验 ethtool -K flannel.1 tx-checksum-ip-generic off
# 原因是flannel虚拟网卡会开启了Checksum校验
# 这里产生了一个CLUSTER-IP，这就是service的IP，在Service的生命周期中，这个地址是不会变动的
# 可以通过这个IP访问当前service对应的POD
[root@master ~]# curl 10.109.179.231:80
```

```

<!DOCTYPE html>
<html>
<head>
<title>welcome to nginx!</title>
</head>
<body>
<h1>welcome to nginx!</h1>
.....
</body>
</html>

```

永久关闭flannel虚拟网卡Checksum校验

```

#使用以下代码创建服务
cat >/etc/systemd/system/k8s-flannel-tx-checksum-off.service <<EOF
[Unit]
Description=Turn off checksum offload on flannel.1
After=sys-devices-virtual-net-flannel.1.device

[Install]
WantedBy=sys-devices-virtual-net-flannel.1.device

[Service]
Type=oneshot
ExecStart=/sbin/ethtool -K flannel.1 tx-checksum-ip-generic off
EOF

#开机自启动，并启动服务
systemctl enable k8s-flannel-tx-checksum-off
systemctl start k8s-flannel-tx-checksum-off

```

4.5.2 创建集群外部也可访问的Service

```

# 上面创建的Service的type类型为ClusterIP，这个ip地址只用集群内部可访问
# 如果需要创建外部也可以访问的Service，需要修改type为NodePort
[root@master ~]# kubectl expose deploy nginx --name=svc-nginx2 --type=NodePort --
port=80 --target-port=80 -n dev
service/svc-nginx2 exposed

# 此时查看，会发现出现了NodePort类型的Service，而且有一对Port（80:31662/TCP）
[root@master ~]# kubectl get svc svc-nginx2 -n dev -o wide

```

NAME	TYPE	CLUSTER-IP	EXTERNAL-IP	PORT(S)	AGE
svc-nginx2	NodePort	10.100.94.0	<none>	80:31662/TCP	9s

```

run=nginx

# 接下来就可以通过集群外的主机访问 节点IP:31928访问服务了
# 例如在的电脑主机上通过浏览器访问下面的地址
http://192.168.5.100:31662/

```

4.5.3 删除Service

```
[root@master ~]# kubectl delete svc svc-nginx-1 -n dev
service "svc-nginx-1" deleted
```

4.5.4 配置方式

创建一个svc-nginx.yaml，内容如下：

```
apiVersion: v1
kind: Service
metadata:
  name: svc-nginx
  namespace: dev
spec:
  clusterIP: 10.109.179.231 #固定svc的内网ip
  ports:
  - port: 80
    protocol: TCP
    targetPort: 80
  selector:
    run: nginx
  type: ClusterIP
```

然后就可以执行对应的创建和删除命令了：

创建：kubectl create -f svc-nginx.yaml

删除：kubectl delete -f svc-nginx.yaml

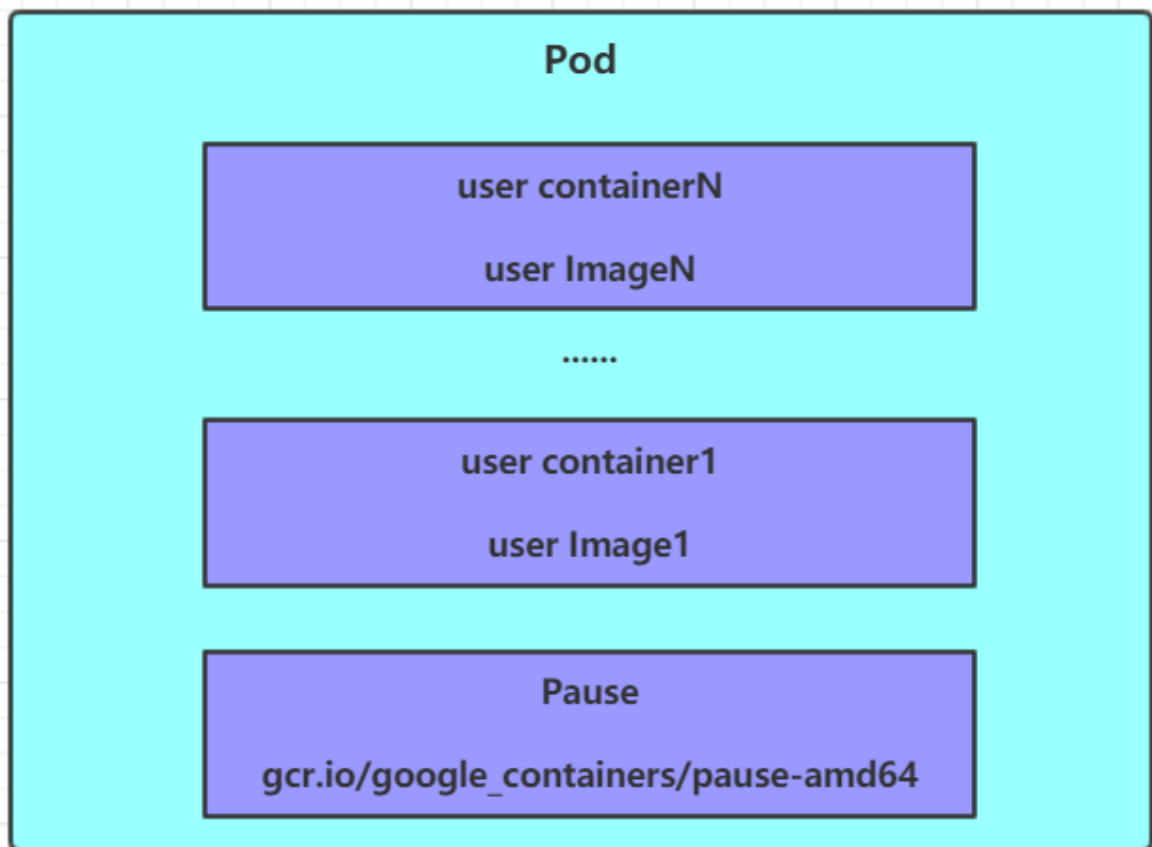
小结

至此，已经掌握了Namespace、Pod、Deployment、Service资源的基本操作，有了这些操作，就可以在kubernetes集群中实现一个服务的简单部署和访问了，但是如果想要更好的使用kubernetes，就需要深入学习这几种资源的细节和原理。

5. Pod详解

5.1 Pod介绍

5.1.1 Pod结构



每个Pod中都可以包含一个或者多个容器，这些容器可以分为两类：

- 用户程序所在的容器，数量可多可少
- Pause容器，这是每个Pod都会有一个**根容器**，它的作用有两个：
 - 可以以它为依据，评估整个Pod的健康状态
 - 可以在根容器上设置Ip地址，其它容器都此Ip（Pod IP），以实现Pod内部的网路通信

这里是Pod内部的通讯，Pod的之间的通讯采用虚拟二层网络技术来实现，我们当前环境用的是Flannel

5.1.2 Pod定义

下面是Pod的资源清单：

```

apiVersion: v1          #必选，版本号，例如v1
kind: Pod               #必选，资源类型，例如 Pod
metadata:               #必选，元数据
  name: string          #必选，Pod名称
  namespace: string     #Pod所属的命名空间,默认为"default"
  labels:               #自定义标签列表
    - name: string
spec: #必选，Pod中容器的详细定义
  containers: #必选，Pod中容器列表
    - name: string      #必选，容器名称
      image: string     #必选，容器的镜像名称
      imagePullPolicy: [ Always|Never|IfNotPresent ] #获取镜像的策略
      command: [string] #容器的启动命令列表，如不指定，使用打包时使用的启动命令
      args: [string]    #容器的启动命令参数列表
      workingDir: string #容器的工作目录
      volumeMounts:     #挂载到容器内部的存储卷配置

```

```

- name: string          #引用pod定义的共享存储卷的名称，需用volumes[]部分定义的卷名
  mountPath: string    #存储卷在容器内mount的绝对路径，应少于512字符
  readOnly: boolean    #是否为只读模式
ports: #需要暴露的端口号列表
- name: string          #端口的名称
  containerPort: int    #容器需要监听的端口号
  hostPort: int         #容器所在主机需要监听的端口号，默认与Container相同
  protocol: string      #端口协议，支持TCP和UDP，默认TCP
env: #容器运行前需设置的环境变量列表
- name: string          #环境变量名称
  value: string         #环境变量的值
resources: #资源限制和请求的设置
  limits: #资源限制的设置
    cpu: string         #Cpu的限制，单位为core数，将用于docker run --cpu-shares参数
    memory: string      #内存限制，单位可以为Mib/Gib，将用于docker run --memory参数
  requests: #资源请求的设置
    cpu: string         #Cpu请求，容器启动的初始可用数量
    memory: string      #内存请求，容器启动的初始可用数量
lifecycle: #生命周期钩子
  postStart: #容器启动后立即执行此钩子，如果执行失败，会根据重启策略进行重启
  preStop: #容器终止前执行此钩子，无论结果如何，容器都会终止
livenessProbe: #对Pod内各容器健康检查的设置，当探测无响应几次后将自动重启该容器
  exec: #对Pod容器内检查方式设置为exec方式
    command: [string]  #exec方式需要制定的命令或脚本
  httpGet: #对Pod内个容器健康检查方法设置为HttpGet，需要制定Path、port
    path: string
    port: number
    host: string
    scheme: string
    httpHeaders:
      - name: string
        value: string
  tcpSocket: #对Pod内个容器健康检查方式设置为tcpSocket方式
    port: number
    initialDelaySeconds: 0 #容器启动完成后首次探测的时间，单位为秒
    timeoutSeconds: 0      #对容器健康检查探测等待响应的超时时间，单位秒，默认1秒
    periodSeconds: 0       #对容器监控检查的定期探测时间设置，单位秒，默认10秒一次

    successThreshold: 0
    failureThreshold: 0
    securityContext:
      privileged: false
restartPolicy: [Always | Never | OnFailure] #Pod的重启策略
nodeName: <string> #设置NodeName表示将该Pod调度到指定名称的node节点上
nodeSelector: object #设置NodeSelector表示将该Pod调度到包含这个label的node上
imagePullSecrets: #Pull镜像时使用的secret名称，以key: secretkey格式指定
- name: string
hostNetwork: false #是否使用主机网络模式，默认为false，如果设置为true，表示使用宿主机网络
volumes: #在该pod上定义共享存储卷列表
- name: string #共享存储卷名称（volumes类型有很多种）
  emptyDir: {} #类型为emptyDir的存储卷，与Pod同生命周期的一个临时目录。为空值
  hostPath: string #类型为hostPath的存储卷，表示挂载Pod所在宿主机的目录
    path: string #Pod所在宿主机的目录，将被用于同期中mount的目录
  secret: #类型为secret的存储卷，挂载集群与定义的secret对象到容器内部
    secretName: string

```

```

    items:
      - key: string
        path: string
  configMap:      #类型为configMap的存储卷，挂载预定义的configMap对象到容器内部
    name: string
    items:
      - key: string
        path: string

```

#小提示:

在这里，可通过一个命令来查看每种资源的可配置项

`kubectl explain 资源类型` 查看某种资源可以配置的一级属性

`kubectl explain 资源类型.属性` 查看属性的子属性

[root@k8s-master01 ~]# `kubectl explain pod`

```

KIND:      Pod
VERSION:   v1
FIELDS:
  apiVersion    <string>
  kind          <string>
  metadata      <Object>
  spec          <Object>
  status        <Object>

```

[root@k8s-master01 ~]# `kubectl explain pod.metadata`

```

KIND:      Pod
VERSION:   v1
RESOURCE:  metadata <Object>
FIELDS:
  annotations    <map[string]string>
  clusterName    <string>
  creationTimestamp    <string>
  deletionGracePeriodSeconds    <integer>
  deletionTimestamp    <string>
  finalizers      <[]string>
  generateName     <string>
  generation       <integer>
  labels           <map[string]string>
  managedFields    <[]Object>
  name             <string>
  namespace        <string>
  ownerReferences  <[]Object>
  resourceVersion  <string>
  selfLink         <string>
  uid              <string>

```

在kubernetes中基本所有资源的一级属性都是一样的，主要包含5部分：

- `apiVersion` 版本，由kubernetes内部定义，版本号必须可以用 `kubectl api-versions` 查询到
- `kind` 类型，由kubernetes内部定义，版本号必须可以用 `kubectl api-resources` 查询到
- `metadata` 元数据，主要是资源标识和说明，常用的有`name`、`namespace`、`labels`等
- `spec` 描述，这是配置中最重要的一部分，里面是对各种资源配置的详细描述
- `status` 状态信息，里面的内容不需要定义，由kubernetes自动生成

在上面的属性中，spec是接下来研究的重点，继续看下它的常见子属性：

- containers <[]Object> 容器列表，用于定义容器的详细信息
- nodeName 根据nodeName的值将pod调度到指定的Node节点上
- nodeSelector <map[]> 根据NodeSelector中定义的信息选择将该Pod调度到包含这些label的Node上
- hostNetwork 是否使用主机网络模式，默认为false，如果设置为true，表示使用宿主机网络(192.168.5.100)
- volumes <[]Object> 存储卷，用于定义Pod上面挂在的存储信息
- restartPolicy 重启策略，表示Pod在遇到故障的时候的处理策略

5.2 Pod配置

本小节主要来研究 pod.spec.containers 属性，这也是pod配置中最为关键的一项配置。

```
[root@k8s-master01 ~]# kubectl explain pod.spec.containers
KIND:      Pod
VERSION:   v1
RESOURCE: containers <[]Object>    # 数组，代表可以有多个容器
FIELDS:
  name <string>          # 容器名称
  image <string>          # 容器需要的镜像地址
  imagePullPolicy <string> # 镜像拉取策略
  command <[]string>     # 容器的启动命令列表，如不指定，使用打包时使用的启动命令
  args <[]string>        # 容器的启动命令需要的参数列表
  env <[]Object>         # 容器环境变量的配置
  ports <[]Object>       # 容器需要暴露的端口号列表
  resources <Object>     # 资源限制和资源请求的设置
```

5.2.1 基本配置

创建pod-base.yaml文件，内容如下：

```
apiVersion: v1
kind: Pod
metadata:
  name: pod-base
  namespace: dev
  labels:
    user: jiaxiong
spec:
  containers:
  - name: nginx
    image: nginx:1.17.1
  - name: busybox
    image: busybox:1.30
```



```
[root@k8s-master01 pod-k8s]# kubectl get pods -n dev
```

NAME	READY	STATUS	RESTARTS	AGE
pod-base	1/2	CrashLoopBackOff	1	39s

上面定义了一个比较简单Pod的配置，里面有两个容器：

- nginx：用1.17.1版本的nginx镜像创建，（nginx是一个轻量级web容器）
- busybox：用1.30版本的busybox镜像创建，（busybox是一个小巧的linux命令集合）

创建Pod

```
[root@k8s-master01 pod]# kubectl apply -f pod-base.yaml
pod/pod-base created
```

查看Pod状况

READY 1/2 ：表示当前Pod中有2个容器，其中1个准备就绪，1个未就绪

RESTARTS ：重启次数，因为有1个容器故障了，Pod一直在重启试图恢复它

```
[root@k8s-master01 pod]# kubectl get pod -n dev
```

NAME	READY	STATUS	RESTARTS	AGE
pod-base	1/2	Running	4	95s

可以通过describe查看内部的详情

此时已经运行起来了一个基本的Pod，虽然它暂时有问题

```
[root@k8s-master01 pod]# kubectl describe pod pod-base -n dev
```

5.2.2 镜像拉取

创建pod-imagepullpolicy.yaml文件，内容如下：

```
apiVersion: v1
kind: Pod
metadata:
  name: pod-imagepullpolicy
  namespace: dev
spec:
  containers:
  - name: nginx
    image: nginx:1.17.1
    imagePullPolicy: Never # 用于设置镜像拉取策略
  - name: busybox
    image: busybox:1.30
```

```
[root@k8s-master01 pod-k8s]# kubectl get pod -n dev
```

NAME	READY	STATUS	RESTARTS	AGE
pod-base	1/2	CrashLoopBackOff	6	8m50s
pod-imagepullpolicy	0/2	ContainerCreating	0	11s

imagePullPolicy，用于设置镜像拉取策略，kubernetes支持配置三种拉取策略：

- Always：总是从远程仓库拉取镜像（一直远程下载）
- IfNotPresent：本地有则使用本地镜像，本地没有则从远程仓库拉取镜像（本地有就本地 本地没远程下载）
- Never：只使用本地镜像，从不去远程仓库拉取，本地没有就报错（一直使用本地）

默认值说明:

如果镜像tag为具体版本号, 默认策略是: IfNotPresent

如果镜像tag为: latest (最终版本), 默认策略是always

```
# 创建Pod
[root@k8s-master01 pod]# kubectl create -f pod-imagepullpolicy.yaml
pod/pod-imagepullpolicy created

# 查看Pod详情
# 此时明显可以看到nginx镜像有一步Pulling image "nginx:1.17.1"的过程
[root@k8s-master01 pod]# kubectl describe pod pod-imagepullpolicy -n dev
.....
Events:
  Type           Reason            Age           From                    Message
  ----           -
  Normal        Scheduled         <unknown>     default-scheduler       Successfully assigned
dev/pod-imagePullPolicy to node1
  Normal        Pulling           32s           kubelet, node1          Pulling image
"nginx:1.17.1"
  Normal        Pulled            26s           kubelet, node1          Successfully pulled
image "nginx:1.17.1"
  Normal        Created           26s           kubelet, node1          Created container
nginx
  Normal        Started           25s           kubelet, node1          Started container
nginx
  Normal        Pulled            7s (x3 over 25s) kubelet, node1          Container image
"busybox:1.30" already present on machine
  Normal        Created           7s (x3 over 25s) kubelet, node1          Created container
busybox
  Normal        Started           7s (x3 over 25s) kubelet, node1          Started container
busybox
```

5.2.3 启动命令

在前面的案例中, 一直有一个问题没有解决, 就是的busybox容器一直没有成功运行, 那么到底是什么原因导致这个容器的故障呢?

原来busybox并不是一个程序, 而是类似于一个工具类的集合, kubernetes集群启动管理后, 它会自动关闭。解决方法就是让其一直在运行, 这就用到了command配置。

创建pod-command.yaml文件, 内容如下:

```

apiVersion: v1
kind: Pod
metadata:
  name: pod-command
  namespace: dev
spec:
  containers:
  - name: nginx
    image: nginx:1.17.1
  - name: busybox
    image: busybox:1.30
    command: ["/bin/sh", "-c", "touch /tmp/hello.txt; while true; do /bin/echo $(date +%T) >> /tmp/hello.txt; sleep 3; done;"]

```

```

[root@k8s-master01 pod-k8s]# kubectl get pod -n dev

```

NAME	READY	STATUS	RESTARTS	AGE
pod-base	1/2	CrashLoopBackOff	7	15m
pod-command	2/2	Running	0	36s
pod-imagepullpolicy	0/2	CrashLoopBackOff	6	7m6s

command, 用于在pod中的容器初始化完毕之后运行一个命令。

稍微解释下上面命令的意思：

"/bin/sh", "-c", 使用sh执行命令

touch /tmp/hello.txt; 创建一个/tmp/hello.txt 文件

while true; do /bin/echo \$(date +%T) >> /tmp/hello.txt; sleep 3; done; 每隔3秒向文件中写入当前时间

创建Pod

```

[root@k8s-master01 pod]# kubectl create -f pod-command.yaml
pod/pod-command created

```

查看Pod状态

此时发现两个pod都正常运行了

```

[root@k8s-master01 pod]# kubectl get pods pod-command -n dev

```

NAME	READY	STATUS	RESTARTS	AGE
pod-command	2/2	Runing	0	2s

进入pod中的busybox容器, 查看文件内容

补充一个命令: kubectl exec pod名称 -n 命名空间 -it -c 容器名称 /bin/sh 在容器内部执行命令

使用这个命令就可以进入某个容器的内部, 然后进行相关操作了

比如, 可以查看txt文件的内容

```

[root@master ~]# kubectl exec pod-base -n dev -it -c busybox /bin/sh

```

```

/ # ls

```

```

bin dev etc home proc root sys tmp usr var

```

```

/ # cd tmp

```

```

/tmp # ls

```

```

hello.txt

```

```

/tmp # tail -f hello.txt

```

```

03:03:33

```

03:03:36
03:03:39
03:03:42
03:03:45

特别说明:

通过上面发现command已经可以完成启动命令和传递参数的功能,为什么这里还要提供一个args选项,用于传递参数呢?这其实跟docker有点关系,kubernetes中的command、args两项其实是实现覆盖Dockerfile中ENTRYPOINT的功能。

- 1 如果command和args均没有写,那么用Dockerfile的配置。
- 2 如果command写了,但args没有写,那么Dockerfile默认的配置会被忽略,执行输入的command
- 3 如果command没写,但args写了,那么Dockerfile中配置的ENTRYPOINT的命令会被执行,使用当前args的参数
- 4 如果command和args都写了,那么Dockerfile的配置被忽略,执行command并追加args参数

5.2.4 环境变量

创建pod-env.yaml文件,内容如下:

```
apiVersion: v1
kind: Pod
metadata:
  name: pod-env
  namespace: dev
spec:
  containers:
  - name: busybox
    image: busybox:1.30
    command: ["/bin/sh", "-c", "while true;do /bin/echo $(date +%T);sleep 60;done;"]
    env: # 设置环境变量列表
    - name: "username"
      value: "admin"
    - name: "password"
      value: "123456"
```

env, 环境变量, 用于在pod中的容器设置环境变量。

```
# 创建Pod
[root@k8s-master01 ~]# kubectl create -f pod-env.yaml
pod/pod-env created

# 进入容器, 输出环境变量
[root@k8s-master01 ~]# kubectl exec pod-env -n dev -c busybox -it /bin/sh
/ # echo $username
admin
/ # echo $password
123456
```

这种方式不是很推荐, 推荐将这些配置单独存储在配置文件中, 这种方式将在后面介绍。

5.2.5 端口设置

本小节来介绍容器的端口设置，也就是containers的ports选项。

首先看下ports支持的子选项：

```
[root@k8s-master01 ~]# kubectl explain pod.spec.containers.ports
KIND:      Pod
VERSION:   v1
RESOURCE:  ports <[]Object>
FIELDS:
  name                <string> # 端口名称，如果指定，必须保证name在pod中是唯一的
  containerPort<integer> # 容器要监听的端口(0<x<65536)
  hostPort            <integer> # 容器要在主机上公开的端口，如果设置，主机上只能运行容器的一个副本
                        (一般省略)，
                        #多个pod对同一端口冲突
  hostIP              <string> # 要将外部端口绑定到的主机IP(一般省略)，同上
  protocol            <string> # 端口协议。必须是UDP、TCP或SCTP。默认为“TCP”。
```

接下来，编写一个测试案例，创建pod-ports.yaml

```
apiVersion: v1
kind: Pod
metadata:
  name: pod-ports
  namespace: dev
spec:
  containers:
  - name: nginx
    image: nginx:1.17.1
    ports: # 设置容器暴露的端口列表
    - name: nginx-port
      containerPort: 80
      protocol: TCP
```

```
# 创建Pod
[root@k8s-master01 ~]# kubectl create -f pod-ports.yaml
pod/pod-ports created

# 查看pod
# 在下面可以明显看到配置信息
[root@k8s-master01 ~]# kubectl get pod pod-ports -n dev -o yaml
.....
spec:
  containers:
  - image: nginx:1.17.1
    imagePullPolicy: IfNotPresent
    name: nginx
    ports:
    - containerPort: 80
      name: nginx-port
      protocol: TCP
.....
```

访问容器中的程序需要使用的是 `Podip:containerPort`

5.2.6 资源配额

容器中的程序要运行，肯定是要占用一定资源的，比如cpu和内存等，如果不对某个容器的资源做限制，那么它就可能吃掉大量资源，导致其它容器无法运行。针对这种情况，kubernetes提供了对内存和cpu的资源进行配额的机制，这种机制主要通过resources选项实现，他有两个子选项：

- limits：用于限制运行时容器的最大占用资源，当容器占用资源超过limits时会被终止，并进行重启
- requests：用于设置容器需要的最小资源，如果环境资源不够，容器将无法启动

可以通过上面两个选项设置资源的上下限。

接下来，编写一个测试案例，创建pod-resources.yaml

```
apiVersion: v1
kind: Pod
metadata:
  name: pod-resources
  namespace: dev
spec:
  containers:
  - name: nginx
    image: nginx:1.17.1
    resources: # 资源配额
      limits: # 限制资源（上限）
        cpu: "2" # CPU限制，单位是core数
        memory: "10Gi" # 内存限制
      requests: # 请求资源（下限）
        cpu: "1" # CPU限制，单位是core数
        memory: "10Mi" # 内存限制
```

在这对cpu和memory的单位做一个说明：

- cpu: core数，可以为整数或小数
- memory: 内存大小，可以使用Gi、Mi、G、M等形式

```
# 运行Pod
[root@k8s-master01 ~]# kubectl create -f pod-resources.yaml
pod/pod-resources created

# 查看发现pod运行正常
[root@k8s-master01 ~]# kubectl get pod pod-resources -n dev
NAME             READY   STATUS    RESTARTS   AGE
pod-resources    1/1     Running   0           39s

# 接下来，停止Pod
[root@k8s-master01 ~]# kubectl delete -f pod-resources.yaml
pod "pod-resources" deleted

# 编辑pod，修改resources.requests.memory的值为40Gi，不要设置太小了，比如我电脑32g内存，分给了虚拟机20Gi，试了半天10Gi，一直能跑
# 把limits 也改高点，不低于40Gi
[root@k8s-master01 ~]# vim pod-resources.yaml
```

```
# 再次启动pod
[root@k8s-master01 ~]# kubectl create -f pod-resources.yaml
pod/pod-resources created

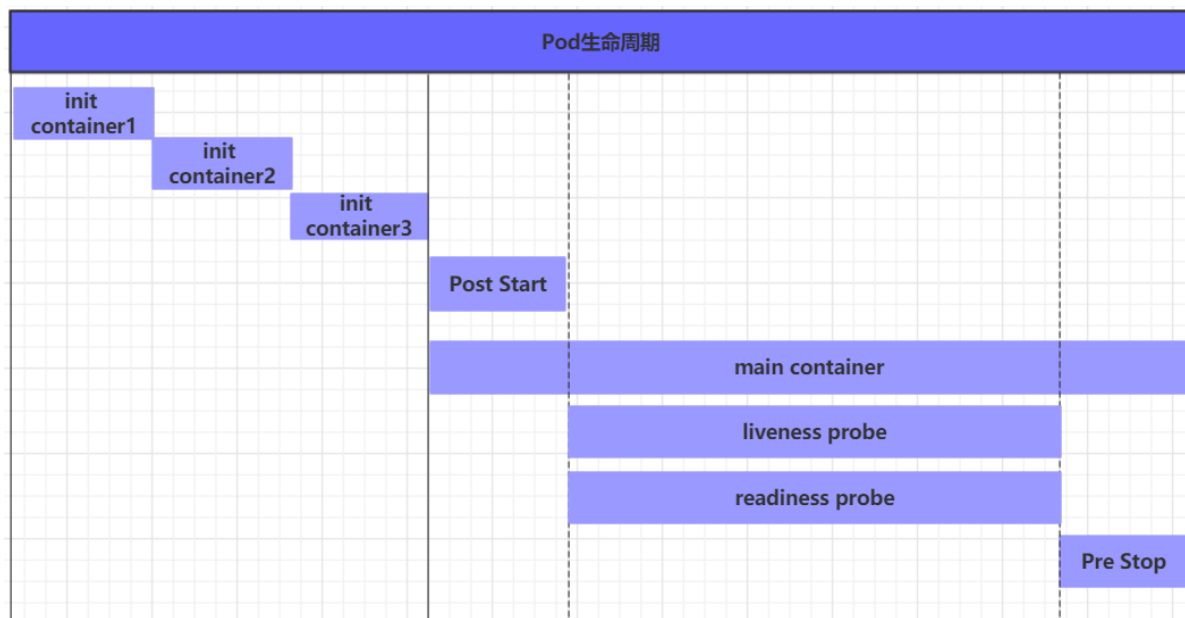
# 查看Pod状态，发现Pod启动失败
[root@k8s-master01 ~]# kubectl get pod pod-resources -n dev -o wide
NAME                READY   STATUS    RESTARTS   AGE
pod-resources        0/1     Pending   0           20s

# 查看pod详情会发现，如下提示
[root@k8s-master01 ~]# kubectl describe pod pod-resources -n dev
.....
Warning FailedScheduling 35s default-scheduler 0/3 nodes are available: 1
node(s) had taint {node-role.kubernetes.io/master: }, that the pod didn't
tolerate, 2 Insufficient memory.(内存不足)
```

5.3 Pod生命周期

我们一般将pod对象从创建至终的这段时间范围称为pod的生命周期，它主要包含下面的过程：

- pod创建过程
- 运行初始化容器（init container）过程
- 运行主容器（main container）
 - 容器启动后钩子（post start）、容器终止前钩子（pre stop）
 - 容器的存活性探测（liveness probe）、就绪性探测（readiness probe）
- pod终止过程
- 下方 init container1 - init container3 属于 初始化容器 的运行过程，Post Start - Pre Stop属于 main容器 的运行过程



在整个生命周期中，Pod会出现5种**状态（相位）**，分别如下：

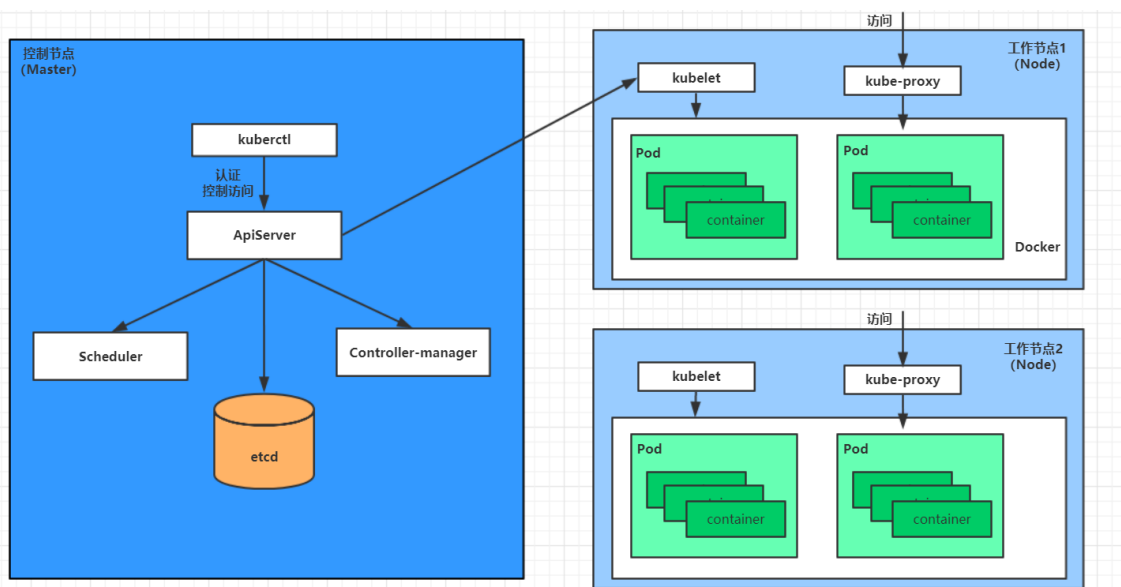
- 挂起（Pending）：apiserver已经创建了pod资源对象，但它尚未被调度完成或者仍处于下载镜像的过程中
- 运行中（Running）：pod已经被调度至某节点，并且所有容器都被kubelet创建完成
- 成功（Succeeded）：pod中的所有容器都已经成功终止并且不会被重启

- 失败 (Failed)：所有容器都已经终止，但至少有一个容器终止失败，即容器返回了非0值的退出状态
- 未知 (Unknown)：apiserver无法正常获取到pod对象的状态信息，通常由网络通信失败所导致

5.3.1 创建和终止

pod的创建过程

1. 用户通过kubectl或其他api客户端提交需要创建的pod信息给apiServer
2. apiServer开始生成pod对象的信息，并将信息存入etcd，然后返回确认信息至客户端
3. apiServer开始反映etcd中的pod对象的变化，其它组件使用watch机制来跟踪检查apiServer上的变动
4. scheduler发现有新的pod对象要创建，开始为Pod分配主机并将结果信息更新至apiServer
5. node节点上的kubelet发现有pod调度过来，尝试调用docker启动容器，并将结果回送至apiServer
6. apiServer将接收到的pod状态信息存入etcd中



pod的终止过程

1. 用户向apiServer发送删除pod对象的命令
2. apiServer中的pod对象信息会随着时间的推移而更新，在宽限期内（默认30s），pod被视为dead
3. 将pod标记为terminating状态
4. kubelet在监控到pod对象转为terminating状态的同时启动pod关闭过程
5. 端点控制器监控到pod对象的关闭行为时将其从所有匹配到此端点的service资源的端点列表中移除
6. 如果当前pod对象定义了preStop钩子处理器，则在其标记为terminating后即会以同步的方式启动执行
7. pod对象中的容器进程收到停止信号
8. 宽限期结束后，若pod中还存在仍在运行的进程，那么pod对象会收到立即终止的信号
9. kubelet请求apiServer将此pod资源的宽限期设置为0从而完成删除操作，此时pod对于用户已不可见

5.3.2 初始化容器

初始化容器是在pod的主容器启动之前要运行的容器，主要是做一些主容器的前置工作，它具有两大特征：

1. 初始化容器必须运行完成直至结束，若某初始化容器运行失败，那么kubernetes需要重启它直到成功完成
2. 初始化容器必须按照定义的顺序执行，当且仅当前一个成功之后，后面的一个才能运行

初始化容器有很多的应用场景，下面列出的是最常见的几个：

- 提供主容器镜像中不具备的工具程序或自定义代码
- 初始化容器要先于应用容器串行启动并运行完成，因此可用于延后应用容器的启动直至其依赖的条件得到满足

接下来做一个案例，模拟下面这个需求：

假设要以主容器来运行nginx，但是要求在运行nginx之前先要能够连接上mysql和redis所在服务器

为了简化测试，事先规定好mysql (192.168.90.14) 和redis (192.168.90.15) 服务器的地址

创建pod-initcontainer.yaml，内容如下：

```
apiVersion: v1
kind: Pod
metadata:
  name: pod-initcontainer
  namespace: dev
spec:
  containers:
    - name: main-container
      image: nginx:1.17.1
      ports:
        - name: nginx-port
          containerPort: 80
  initContainers:
    - name: test-mysql
      image: busybox:1.30
      command: ['sh', '-c', 'until ping 192.168.5.14 -c 1 ; do echo waiting for mysql...; sleep 2; done;']
    - name: test-redis
      image: busybox:1.30
      command: ['sh', '-c', 'until ping 192.168.5.15 -c 1 ; do echo waiting for reids...; sleep 2; done;']
```

创建pod

```
[root@k8s-master01 ~]# kubectl create -f pod-initcontainer.yaml
pod/pod-initcontainer created
```

查看pod状态

发现pod卡在启动第一个初始化容器过程中，后面的容器不会运行

```
root@k8s-master01 ~]# kubectl describe pod pod-initcontainer -n dev
```

.....

Events:

Type	Reason	Age	From	Message
----	-----	----	----	-----

```

Normal   Scheduled   49s   default-scheduler   Successfully assigned dev/pod-
initcontainer to node1
Normal   Pulled      48s   kubelet, node1      Container image "busybox:1.30"
already present on machine
Normal   Created     48s   kubelet, node1      Created container test-mysql
Normal   Started     48s   kubelet, node1      Started container test-mysql

```

动态查看pod

```
[root@k8s-master01 ~]# kubectl get pods pod-initcontainer -n dev -w
```

```

NAME                                READY   STATUS             RESTARTS   AGE
pod-initcontainer                   0/1     Init:0/2           0          15s
pod-initcontainer                   0/1     Init:1/2           0          52s
pod-initcontainer                   0/1     Init:1/2           0          53s
pod-initcontainer                   0/1     PodInitializing    0          89s
pod-initcontainer                   1/1     Running             0          90s

```

接下来新开一个shell，为当前服务器新增两个ip，观察pod的变化

```

[root@k8s-master01 ~]# ifconfig ens33:1 192.168.5.14 netmask 255.255.255.0 up
[root@k8s-master01 ~]# ifconfig ens33:2 192.168.5.15 netmask 255.255.255.0 up

```

5.3.3 钩子函数

钩子函数能够感知自身生命周期中的事件，并在相应的时刻到来时运行用户指定的程序代码。

kubernetes在主容器的启动之后和停止之前提供了两个钩子函数：

- post start：容器创建之后执行，如果失败了会重启容器
- pre stop：容器终止之前执行，执行完成之后容器将成功终止，在其完成之前会阻塞删除容器的操作

钩子处理器支持使用下面三种方式定义动作：

- Exec命令：在容器内执行一次命令

```

.....
  lifecycle:
    postStart:
      exec:
        command:
          - cat
          - /tmp/healthy
.....

```

- TCP Socket：在当前容器尝试访问指定的socket

```

.....
  lifecycle:
    postStart:
      tcpSocket:
        port: 8080
.....

```

- HTTP Get：在当前容器中向某url发起http请求

```

.....
lifecycle:
  postStart:
    httpGet:
      path: / #URI地址
      port: 80 #端口号
      host: 192.168.5.3 #主机地址
      scheme: HTTP #支持的协议，http或者https
.....

```

接下来，以exec方式为例，演示下钩子函数的使用，创建pod-hook-exec.yaml文件，内容如下：

```

apiVersion: v1
kind: Pod
metadata:
  name: pod-hook-exec
  namespace: dev
spec:
  containers:
  - name: main-container
    image: nginx:1.17.1
    ports:
    - name: nginx-port
      containerPort: 80
    lifecycle:
      postStart:
        exec: # 在容器启动的时候执行一个命令，修改掉nginx的默认首页内容
          command: ["/bin/sh", "-c", "echo postStart... > /usr/share/nginx/html/index.html"]
      preStop:
        exec: # 在容器停止之前停止nginx服务
          command: ["/usr/sbin/nginx", "-s", "quit"]

```

```

# 创建pod
[root@k8s-master01 ~]# kubectl create -f pod-hook-exec.yaml
pod/pod-hook-exec created

# 查看pod
[root@k8s-master01 ~]# kubectl get pods pod-hook-exec -n dev -o wide
NAME                READY    STATUS    RESTARTS   AGE    IP             NODE
pod-hook-exec       1/1      Running   0           29s    10.244.2.48    node2

# 访问pod
[root@k8s-master01 ~]# curl 10.244.2.48
postStart...

```

5.3.4 容器探测

容器探测用于检测容器中的应用实例是否正常工作，是保障业务可用性的一种传统机制。如果经过探测，实例的状态不符合预期，那么kubernetes就会把该问题实例"摘除"，不承担业务流量。

kubernetes提供了两种探针来实现容器探测，分别是：

- liveness probes：存活性探针，用于检测应用实例当前是否处于正常运行状态，如果不是，k8s会重启容器

- readiness probes: 就绪性探针, 用于检测应用实例当前是否可以接收请求, 如果不能, k8s不会转发流量

livenessProbe 决定是否重启容器, readinessProbe 决定是否将请求转发给容器。

上面两种探针目前均支持三种探测方式:

- Exec命令: 在容器内执行一次命令, 如果命令执行的退出码为0, 则认为程序正常, 否则不正常

```
.....
  livenessProbe:
    exec:
      command:
        - cat
        - /tmp/healthy
.....
```

- TCP Socket: 将会尝试访问一个用户容器的端口, 如果能够建立这条连接, 则认为程序正常, 否则不正常

```
.....
  livenessProbe:
    tcpSocket:
      port: 8080
.....
```

- HTTP Get: 调用容器内Web应用的URL, 如果返回的状态码在200和399之间, 则认为程序正常, 否则不正常

```
.....
  livenessProbe:
    httpGet:
      path: / #URI地址
      port: 80 #端口号
      host: 127.0.0.1 #主机地址
      scheme: HTTP #支持的协议, http或者https
.....
```

下面以liveness probes为例, 做几个演示:

方式一: Exec

创建pod-liveness-exec.yaml

```
apiVersion: v1
kind: Pod
metadata:
  name: pod-liveness-exec
  namespace: dev
spec:
  containers:
    - name: nginx
      image: nginx:1.17.1
      ports:
        - name: nginx-port
```

```

    containerPort: 80
  livenessProbe:
    exec:
      command: ["/bin/cat", "/tmp/hello.txt"] # 执行一个查看文件的命令

```

创建pod, 观察效果

```

# 创建Pod
[root@k8s-master01 ~]# kubectl create -f pod-liveness-exec.yaml
pod/pod-liveness-exec created

# 查看Pod详情
[root@k8s-master01 ~]# kubectl describe pods pod-liveness-exec -n dev
.....
    Normal    Created      20s (x2 over 50s)    kubelet, node1      Created container
nginx
    Normal    Started      20s (x2 over 50s)    kubelet, node1      Started container
nginx
    Normal    Killing      20s                  kubelet, node1      Container nginx
failed liveness probe, will be restarted
    Warning   Unhealthy    0s (x5 over 40s)    kubelet, node1      Liveness probe
failed: cat: can't open '/tmp/hello11.txt': No such file or directory

# 观察上面的信息就会发现nginx容器启动之后就进行了健康检查
# 检查失败之后, 容器被kill掉, 然后尝试进行重启 (这是重启策略的作用, 后面讲解)
# 稍等一会之后, 再观察pod信息, 就可以看到RESTARTS不再是0, 而是一直增长
[root@k8s-master01 ~]# kubectl get pods pod-liveness-exec -n dev
NAME                READY   STATUS              RESTARTS   AGE
pod-liveness-exec   0/1     CrashLoopBackOff    2          3m19s

# 将命令改成输入目录: "/bin/ls", "/tmp/" 再试, 结果就正常了.....

```

方式二: TCPSocket

创建pod-liveness-tcpsocket.yaml

```

apiVersion: v1
kind: Pod
metadata:
  name: pod-liveness-tcpsocket
  namespace: dev
spec:
  containers:
  - name: nginx
    image: nginx:1.17.1
    ports:
    - name: nginx-port
      containerPort: 80
    livenessProbe:
      tcpSocket:
        port: 8080 # 尝试访问8080端口

```

创建pod, 观察效果

```
# 创建Pod
[root@k8s-master01 ~]# kubectl create -f pod-liveness-tcpsocket.yaml
pod/pod-liveness-tcpsocket created

# 查看Pod详情
[root@k8s-master01 ~]# kubectl describe pods pod-liveness-tcpsocket -n dev
.....
Normal    Scheduled    31s                                default-scheduler
Successfully assigned dev/pod-liveness-tcpsocket to node2
Normal    Pulled       <invalid>                          kubelet, node2    Container
image "nginx:1.17.1" already present on machine
Normal    Created      <invalid>                          kubelet, node2    Created
container nginx
Normal    Started      <invalid>                          kubelet, node2    Started
container nginx
Warning   Unhealthy    <invalid> (x2 over <invalid>)      kubelet, node2    Liveness
probe failed: dial tcp 10.244.2.44:8080: connect: connection refused

# 观察上面的信息,发现尝试访问8080端口,但是失败了
# 稍等一会之后,再观察pod信息,就可以看到RESTARTS不再是0,而是一直增长
[root@k8s-master01 ~]# kubectl get pods pod-liveness-tcpsocket -n dev
NAME                                READY    STATUS                RESTARTS   AGE
pod-liveness-tcpsocket              0/1     CrashLoopBackOff      2          3m19s

# 当然接下来,可以修改成一个可以访问的端口,比如80,再试,结果就正常了.....
```

方式三: HTTPGet

创建pod-liveness-httpget.yaml

```
apiVersion: v1
kind: Pod
metadata:
  name: pod-liveness-httpget
  namespace: dev
spec:
  containers:
  - name: nginx
    image: nginx:1.17.1
    ports:
    - name: nginx-port
      containerPort: 80
    livenessProbe:
      httpGet: # 其实就是访问http://127.0.0.1:80/hello
        scheme: HTTP #支持的协议, http或者https
        port: 80 #端口号
        path: /hello #URI地址
```

创建pod, 观察效果

```
# 创建Pod
[root@k8s-master01 ~]# kubectl create -f pod-liveness-httpget.yaml
pod/pod-liveness-httpget created

# 查看Pod详情
```

```
[root@k8s-master01 ~]# kubectl describe pod pod-liveness-httpget -n dev
.....
Normal    Pulled      6s (x3 over 64s)  kubelet, node1    Container image
"nginx:1.17.1" already present on machine
Normal    Created     6s (x3 over 64s)  kubelet, node1    Created container
nginx
Normal    Started     6s (x3 over 63s)  kubelet, node1    Started container
nginx
Warning   Unhealthy   6s (x6 over 56s)  kubelet, node1    Liveness probe failed:
HTTP probe failed with statuscode: 404
Normal    Killing     6s (x2 over 36s)  kubelet, node1    Container nginx failed
liveness probe, will be restarted

# 观察上面信息，尝试访问路径，但是未找到，出现404错误
# 稍等一会之后，再观察pod信息，就可以看到RESTARTS不再是0，而是一直增长
[root@k8s-master01 ~]# kubectl get pod pod-liveness-httpget -n dev
NAME                                READY   STATUS    RESTARTS   AGE
pod-liveness-httpget                1/1     Running   5           3m17s

# 当然接下来，可以修改成一个可以访问的路径path，比如/，再试，结果就正常了.....
```

至此，已经使用liveness Probe演示了三种探测方式，但是查看livenessProbe的子属性，会发现除了这三种方式，还有一些其他的配置，在这里一并解释下：

```
[root@k8s-master01 ~]# kubectl explain pod.spec.containers.livenessProbe
FIELDS:
  exec <Object>
  tcpSocket <Object>
  httpGet <Object>
  initialDelaySeconds <integer> # 容器启动后等待多少秒执行第一次探测
  timeoutSeconds <integer> # 探测超时时间。默认1秒，最小1秒
  periodSeconds <integer> # 执行探测的频率。默认是10秒，最小1秒
  failureThreshold <integer> # 连续探测失败多少次才被认定为失败。默认是3。最小值是1
  successThreshold <integer> # 连续探测成功多少次才被认定为成功。默认是1
```

下面稍微配置两个，演示下效果即可：

```
[root@k8s-master01 ~]# more pod-liveness-httpget.yaml
apiVersion: v1
kind: Pod
metadata:
  name: pod-liveness-httpget
  namespace: dev
spec:
  containers:
  - name: nginx
    image: nginx:1.17.1
    ports:
    - name: nginx-port
      containerPort: 80
    livenessProbe:
      httpGet:
        scheme: HTTP
        port: 80
```

```
path: /
initialDelaySeconds: 30 # 容器启动后30s开始探测
timeoutSeconds: 5 # 探测超时时间为5s
```

5.3.5 重启策略

在上一节中，一旦容器探测出现了问题，kubernetes就会对容器所在的Pod进行重启，其实这是由pod的重启策略决定的，pod的重启策略有3种，分别如下：

- Always：容器失效时，自动重启该容器，这也是默认值。
- OnFailure：容器终止运行且退出码不为0时重启
- Never：不论状态为何，都不重启该容器

重启策略适用于pod对象中的所有容器，首次需要重启的容器，将在其需要时立即进行重启，随后再次需要重启的操作将由kubelet延迟一段时间后进行，且反复的重启操作的延迟时长以此为10s、20s、40s、80s、160s和300s，300s是最大延迟时长。

创建pod-restartpolicy.yaml：

```
apiVersion: v1
kind: Pod
metadata:
  name: pod-restartpolicy
  namespace: dev
spec:
  containers:
  - name: nginx
    image: nginx:1.17.1
    ports:
    - name: nginx-port
      containerPort: 80
    livenessProbe:
      httpGet:
        scheme: HTTP
        port: 80
        path: /hello
    restartPolicy: Never # 设置重启策略为Never
```

运行Pod测试

```
# 创建Pod
[root@k8s-master01 ~]# kubectl create -f pod-restartpolicy.yaml
pod/pod-restartpolicy created

# 查看Pod详情，发现nginx容器失败
[root@k8s-master01 ~]# kubectl describe pods pod-restartpolicy -n dev
.....
Warning Unhealthy 15s (x3 over 35s) kubelet, node1 Liveness probe
failed: HTTP probe failed with statuscode: 404
Normal Killing 15s kubelet, node1 Container nginx
failed liveness probe

# 多等一会，再观察pod的重启次数，发现一直是0，并未重启
[root@k8s-master01 ~]# kubectl get pods pod-restartpolicy -n dev
```


NAME	READY	STATUS	RESTARTS	AGE
pod-restartpolicy	0/1	Running	0	5min42s

5.4 Pod调度

在默认情况下，一个Pod在哪个Node节点上运行，是由Scheduler组件采用相应的算法计算出来的，这个过程是不受人工控制的。但是在实际使用中，这并不满足的需求，因为很多情况下，我们想控制某些Pod到达某些节点上，那么应该怎么做呢？这就要求了解kubernetes对Pod的调度规则，kubernetes提供了四大类调度方式：

- 自动调度：运行在哪个节点上完全由Scheduler经过一系列的算法计算得出
- 定向调度：nodeName、NodeSelector
- 亲和性调度：NodeAffinity、PodAffinity、PodAntiAffinity（和pod关系差）
- 污点（容忍）调度：Taints、Toleration

5.4.1 定向调度

定向调度，指的是利用在pod上声明nodeName或者nodeSelector，以此将Pod调度到期望的node节点上。注意，这里的调度是强制的，这就意味着即使要调度的目标Node不存在，也会向上面进行调度，只不过pod运行失败而已。

nodeName（强制性，如果不存在node节点，则无法正常运行，直接跳过Scheduler调度）

nodeName用于强制约束将Pod调度到指定的Name的Node节点上。这种方式，其实是直接跳过Scheduler的调度逻辑，直接将Pod调度到指定名称的节点。

接下来，实验一下：创建一个pod-nodename.yaml文件

```
apiVersion: v1
kind: Pod
metadata:
  name: pod-nodename
  namespace: dev
spec:
  containers:
  - name: nginx
    image: nginx:1.17.1
  nodeName: node1 # 指定调度到node1节点上
```

#创建Pod

```
[root@k8s-master01 ~]# kubectl create -f pod-nodename.yaml
pod/pod-nodename created
```

#查看Pod调度到NODE属性，确实是调度到了node1节点上

```
[root@k8s-master01 ~]# kubectl get pods pod-nodename -n dev -o wide
```

NAME	READY	STATUS	RESTARTS	AGE	IP	NODE	
pod-nodename	1/1	Running	0	56s	10.244.1.87	node1	

接下来，删除pod，修改nodeName的值为node3（并没有node3节点）

```
[root@k8s-master01 ~]# kubectl delete -f pod-nodename.yaml
pod "pod-nodename" deleted
[root@k8s-master01 ~]# vim pod-nodename.yaml
[root@k8s-master01 ~]# kubectl create -f pod-nodename.yaml
```

```
pod/pod-nodename created
```

#再次查看，发现已经向Node3节点调度，但是由于不存在node3节点，所以pod无法正常运行

```
[root@k8s-master01 ~]# kubectl get pods pod-nodename -n dev -o wide
```

NAME	READY	STATUS	RESTARTS	AGE	IP	NODE	
pod-nodename	0/1	Pending	0	6s	<none>	node3	

NodeSelector (通过Scheduler的MatchNodeSelector进行label匹配, 强制匹配, 找不到就运行失败)

NodeSelector用于将pod调度到添加了指定标签的node节点上。它是通过kubernetes的label-selector机制实现的, 也就是说, 在pod创建之前, 会由scheduler使用MatchNodeSelector调度策略进行label匹配, 找出目标node, 然后将pod调度到目标节点, 该匹配规则是强约束。

接下来, 实验一下:

1 首先分别为node节点添加标签

```
[root@k8s-master01 ~]# kubectl label nodes node1 nodeenv=pro
```

```
node/node2 labeled
```

```
[root@k8s-master01 ~]# kubectl label nodes node2 nodeenv=test
```

```
node/node2 labeled
```

#查询 已有的node, 并显示其标签

```
[root@k8s-master01 ~]# kubectl get nodes --show-label=true
```

2 创建一个pod-nodeselector.yaml文件, 并使用它创建Pod

```
apiVersion: v1
kind: Pod
metadata:
  name: pod-nodeselector
  namespace: dev
spec:
  containers:
  - name: nginx
    image: nginx:1.17.1
  nodeSelector:
    nodeenv: pro # 指定调度到具有nodeenv=pro标签的节点上
```

#创建Pod

```
[root@k8s-master01 ~]# kubectl create -f pod-nodeselector.yaml
```

```
pod/pod-nodeselector created
```

#查看Pod调度到NODE属性, 确实是调度到了node1节点上

```
[root@k8s-master01 ~]# kubectl get pods pod-nodeselector -n dev -o wide
```

NAME	READY	STATUS	RESTARTS	AGE	IP	NODE	
pod-nodeselector	1/1	Running	0	47s	10.244.1.87	node1	

接下来, 删除pod, 修改nodeSelector的值为nodeenv: xxxx (不存在打有此标签的节点)

```
[root@k8s-master01 ~]# kubectl delete -f pod-nodeselector.yaml
```

```
pod "pod-nodeselector" deleted
[root@k8s-master01 ~]# vim pod-nodeselector.yaml
[root@k8s-master01 ~]# kubectl create -f pod-nodeselector.yaml
pod/pod-nodeselector created

#再次查看,发现pod无法正常运行,Node的值为none
[root@k8s-master01 ~]# kubectl get pods -n dev -o wide
NAME                READY   STATUS    RESTARTS   AGE   IP        NODE
pod-nodeselector    0/1     Pending   0           2m20s <none>   <none>

# 查看详情,发现node selector匹配失败的提示
[root@k8s-master01 ~]# kubectl describe pods pod-nodeselector -n dev
.....
Events:
  Type            Reason             Age           From                    Message
  ----            -
  Warning         FailedScheduling   <unknown>     default-scheduler      0/3 nodes are
available: 3 node(s) didn't match node selector.
```

5.4.2 亲和性调度

上一节,介绍了两种定向调度的方式,使用起来非常方便,但是也有一定的问题,那就是如果没有满足条件的Node,那么Pod将不会被运行,即使在集群中还有可用Node列表也不行,这就限制了它的使用场景。

基于上面的问题, kubernetes还提供了一种亲和性调度 (Affinity) 。它在NodeSelector的基础之上的进行了扩展,可以通过配置的形式,实现优先选择满足条件的Node进行调度,如果没有,也可以调度到不满足条件的节点上,使调度更加灵活。

Affinity主要分为三类:

- nodeAffinity(node亲和性): 以node为目标, 解决pod可以调度到哪些node的问题
- podAffinity(pod亲和性): 以pod为目标, 解决pod可以和哪些已存在的pod部署在同一个拓扑域中的问题
- podAntiAffinity(pod反亲和性): 以pod为目标, 解决pod不能和哪些已存在pod部署在同一个拓扑域中的问题

关于亲和性(反亲和性)使用场景的说明:

亲和性: 如果两个应用频繁交互, 那就有必要利用亲和性让两个应用的尽可能的靠近, 这样可以减少因网络通信而带来的性能损耗。

反亲和性: 当应用的采用多副本部署时, 有必要采用反亲和性让各个应用实例打散分布在各个node上, 这样可以提高服务的高可用性。

NodeAffinity

首先来看一下 NodeAffinity 的可配置项:

```
pod.spec.affinity.nodeAffinity
  requiredDuringSchedulingIgnoredDuringExecution Node节点必须满足指定的所有规则才可以, 相当于硬限制
  nodeSelectorTerms 节点选择列表
    matchFields 按节点字段列出的节点选择器要求列表
    matchExpressions 按节点标签列出的节点选择器要求列表(推荐)
      key 键
```

values 值
operator 关系符 支持Exists, DoesNotExist, In, NotIn, Gt, Lt
preferredDuringSchedulingIgnoredDuringExecution 优先调度到满足指定的规则的Node, 相当于软限制 (倾向)
preference 一个节点选择器项, 与相应的权重相关联
matchFields 按节点字段列出的节点选择器要求列表
matchExpressions 按节点标签列出的节点选择器要求列表(推荐)
key 键
values 值
operator 关系符 支持In, NotIn, Exists, DoesNotExist, Gt, Lt
weight 倾向权重, 在范围1-100。

关系符的使用说明:

- matchExpressions:
 - key: nodeenv # 匹配存在标签的key为nodeenv的节点
operator: Exists
 - key: nodeenv # 匹配标签的key为nodeenv, 且value是"xxx"或"yyy"的节点
operator: In
values: ["xxx", "yyy"]
 - key: nodeenv # 匹配标签的key为nodeenv, 且value大于"xxx"的节点
operator: Gt
values: "xxx"

接下来首先演示一下 `requiredDuringSchedulingIgnoredDuringExecution`,

创建pod-nodeaffinity-required.yaml

```
apiVersion: v1
kind: Pod
metadata:
  name: pod-nodeaffinity-required
  namespace: dev
spec:
  containers:
  - name: nginx
    image: nginx:1.17.1
  affinity: #亲和性设置
    nodeAffinity: #设置node亲和性
      requiredDuringSchedulingIgnoredDuringExecution: # 硬限制
        nodeSelectorTerms:
        - matchExpressions: # 匹配env的值在["xxx", "yyy"]中的标签
          - key: nodeenv
            operator: In
            values: ["xxx", "yyy"]
```

```
# 创建pod
[root@k8s-master01 ~]# kubectl create -f pod-nodeaffinity-required.yaml
pod/pod-nodeaffinity-required created

# 查看pod状态 (运行失败)
[root@k8s-master01 ~]# kubectl get pods pod-nodeaffinity-required -n dev -o wide
NAME                                READY   STATUS    RESTARTS   AGE   IP          NODE
.....
```

```

pod-nodeaffinity-required 0/1 Pending 0 16s <none> <none>
.....

# 查看Pod的详情
# 发现调度失败, 提示node选择失败
[root@k8s-master01 ~]# kubectl describe pod pod-nodeaffinity-required -n dev
.....
Warning FailedScheduling <unknown> default-scheduler 0/3 nodes are
available: 3 node(s) didn't match node selector.
Warning FailedScheduling <unknown> default-scheduler 0/3 nodes are
available: 3 node(s) didn't match node selector.

#接下来, 停止pod
[root@k8s-master01 ~]# kubectl delete -f pod-nodeaffinity-required.yaml
pod "pod-nodeaffinity-required" deleted

# 修改文件, 将values: ["xxx","yyy"]-----> ["pro","yyy"]
[root@k8s-master01 ~]# vim pod-nodeaffinity-required.yaml

# 再次启动
[root@k8s-master01 ~]# kubectl create -f pod-nodeaffinity-required.yaml
pod/pod-nodeaffinity-required created

# 此时查看, 发现调度成功, 已经将pod调度到了node1上
[root@k8s-master01 ~]# kubectl get pods pod-nodeaffinity-required -n dev -o wide
NAME                                READY STATUS RESTARTS AGE IP NODE
.....
pod-nodeaffinity-required 1/1 Running 0 11s 10.244.1.89
node1 .....

```

接下来再演示一下 `requiredDuringSchedulingIgnoredDuringExecution`,

创建pod-nodeaffinity-preferred.yaml

```

apiVersion: v1
kind: Pod
metadata:
  name: pod-nodeaffinity-preferred
  namespace: dev
spec:
  containers:
  - name: nginx
    image: nginx:1.17.1
  affinity: #亲和性设置
    nodeAffinity: #设置node亲和性
      preferredDuringSchedulingIgnoredDuringExecution: # 软限制
      - weight: 1
        preference:
          matchExpressions: # 匹配env的值在["xxx","yyy"]中的标签(当前环境没有)
          - key: nodeenv
            operator: In
            values: ["xxx","yyy"]

```

```
# 创建pod
[root@k8s-master01 ~]# kubectl create -f pod-nodeaffinity-preferred.yaml
pod/pod-nodeaffinity-preferred created

# 查看pod状态 （运行成功）
[root@k8s-master01 ~]# kubectl get pod pod-nodeaffinity-preferred -n dev
```

NAME	READY	STATUS	RESTARTS	AGE
pod-nodeaffinity-preferred	1/1	Running	0	40s

NodeAffinity规则设置的注意事项:

- 1 如果同时定义了nodeSelector和nodeAffinity，那么必须两个条件都得到满足，Pod才能运行在指定的Node上
- 2 如果nodeAffinity指定了多个nodeSelectorTerms，那么只需要其中一个能够匹配成功即可
- 3 如果一个nodeSelectorTerms中有多个matchExpressions，则一个节点必须满足所有的才能匹配成功
- 4 如果一个pod所在的Node在Pod运行期间其标签发生了改变，不再符合该Pod的节点亲和性需求，则系统将忽略此变化

PodAffinity

PodAffinity主要实现以运行的Pod为参照，实现让新创建的Pod跟参照pod在一个区域的功能。

首先来看一下 PodAffinity 的可配置项:

```
pod.spec.affinity.podAffinity
requiredDuringSchedulingIgnoredDuringExecution 硬限制
  namespaces      指定参照pod的namespace
  topologyKey      指定调度作用域
  labelSelector    标签选择器
    matchExpressions 按节点标签列出的节点选择器要求列表(推荐)
      key          键
      values       值
      operator      关系符 支持In, NotIn, Exists, DoesNotExist.
    matchLabels     指多个matchExpressions映射的内容
preferredDuringSchedulingIgnoredDuringExecution 软限制
  podAffinityTerm 选项
    namespaces
    topologyKey
    labelSelector
    matchExpressions
      key          键
      values       值
      operator
    matchLabels
  weight          倾向权重，在范围1-100
```

topologyKey用于指定调度时作用域,例如:

- 如果指定为kubernetes.io/hostname，那就是以Node节点为区分范围
- 如果指定为beta.kubernetes.io/os,则以Node节点的操作系统类型来区分

接下来，演示下 requiredDuringSchedulingIgnoredDuringExecution，

- 1) 首先创建一个参照Pod， pod-podaffinity-target.yaml:

```

apiVersion: v1
kind: Pod
metadata:
  name: pod-podaffinity-target
  namespace: dev
  labels:
    podenv: pro #设置标签
spec:
  containers:
  - name: nginx
    image: nginx:1.17.1
  nodeName: node1 # 将目标pod名确指定到node1上

```

```

# 启动目标pod
[root@k8s-master01 ~]# kubectl create -f pod-podaffinity-target.yaml
pod/pod-podaffinity-target created

# 查看pod状况
[root@k8s-master01 ~]# kubectl get pods pod-podaffinity-target -n dev

```

NAME	READY	STATUS	RESTARTS	AGE
pod-podaffinity-target	1/1	Running	0	4s

2) 创建pod-podaffinity-required.yaml, 内容如下:

```

apiVersion: v1
kind: Pod
metadata:
  name: pod-podaffinity-required
  namespace: dev
spec:
  containers:
  - name: nginx
    image: nginx:1.17.1
  affinity: #亲和性设置
    podAffinity: #设置pod亲和性
      requiredDuringSchedulingIgnoredDuringExecution: # 硬限制
      - labelSelector:
          matchExpressions: # 匹配env的值在["xxx","yyy"]中的标签
            - key: podenv
              operator: In
              values: ["xxx","yyy"]
          topologyKey: kubernetes.io/hostname

```

上面配置表达的意思是: 新Pod必须要与拥有标签nodeenv=xxx或者nodeenv=yyy的pod在同一Node上, 显然现在没有这样pod, 接下来, 运行测试一下。

```

# 启动pod
[root@k8s-master01 ~]# kubectl create -f pod-podaffinity-required.yaml
pod/pod-podaffinity-required created

# 查看pod状态, 发现未运行
[root@k8s-master01 ~]# kubectl get pods pod-podaffinity-required -n dev

```

NAME	READY	STATUS	RESTARTS	AGE
------	-------	--------	----------	-----

```

pod-podaffinity-required    0/1    Pending    0    9s

# 查看详细信息
[root@k8s-master01 ~]# kubectl describe pods pod-podaffinity-required -n dev
.....
Events:
  Type            Reason              Age             From              Message
  ----            -
  Warning         FailedScheduling    <unknown>       default-scheduler 0/3 nodes are
available: 2 node(s) didn't match pod affinity rules, 1 node(s) had taints that
the pod didn't tolerate.

# 接下来修改 values: ["xxx","yyy"]----->values:["pro","yyy"]
# 意思是: 新Pod必须要与拥有标签nodeenv=xxx或者nodeenv=yyy的pod在同一Node上
[root@k8s-master01 ~]# vim pod-podaffinity-required.yaml

# 然后重新创建pod, 查看效果
[root@k8s-master01 ~]# kubectl delete -f pod-podaffinity-required.yaml
pod "pod-podaffinity-required" deleted
[root@k8s-master01 ~]# kubectl create -f pod-podaffinity-required.yaml
pod/pod-podaffinity-required created

# 发现此时Pod运行正常
[root@k8s-master01 ~]# kubectl get pods pod-podaffinity-required -n dev
NAME                                READY   STATUS    RESTARTS   AGE   LABELS
pod-podaffinity-required            1/1     Running   0           6s    <none>

```

关于 PodAffinity 的 `preferredDuringSchedulingIgnoredDuringExecution`，这里不再演示。

PodAntiAffinity

PodAntiAffinity 主要实现以运行的 Pod 为参照，让新创建的 Pod 跟参照 pod 不在一个区域中的功能。

它的配置方式和选项跟 PodAffinity 是一样的，这里不再做详细解释，直接做一个测试案例。

1) 继续使用上个案例中目标 pod

```

[root@k8s-master01 ~]# kubectl get pods -n dev -o wide --show-labels
NAME                                READY   STATUS    RESTARTS   AGE   IP             NODE
LABELS
pod-podaffinity-required            1/1     Running   0           3m29s  10.244.1.38    node1
<none>
pod-podaffinity-target              1/1     Running   0           9m25s  10.244.1.37    node1
podenv=pro

```

2) 创建 pod-podantiaffinity-required.yaml，内容如下：

```

apiVersion: v1
kind: Pod
metadata:
  name: pod-podantiaffinity-required
  namespace: dev
spec:
  containers:
  - name: nginx
    image: nginx:1.17.1

```



```

affinity: #亲和性设置
  podAntiAffinity: #设置pod亲和性
    requiredDuringSchedulingIgnoredDuringExecution: # 硬限制
      - labelSelector:
          matchExpressions: # 匹配podenv的值在["pro"]中的标签
            - key: podenv
              operator: In
              values: ["pro"]
        topologyKey: kubernetes.io/hostname

```

上面配置表达的意思是：新Pod必须要与拥有标签nodeenv=pro的pod不在同一Node上，运行测试一下。

```

# 创建pod
[root@k8s-master01 ~]# kubectl create -f pod-podantiaffinity-required.yaml
pod/pod-podantiaffinity-required created

# 查看pod
# 发现调度到了node2上
[root@k8s-master01 ~]# kubectl get pods pod-podantiaffinity-required -n dev -o wide

```

NAME	READY	STATUS	RESTARTS	AGE	IP
node	..				
pod-podantiaffinity-required	1/1	Running	0	30s	10.244.1.96
node2	..				

5.4.3 污点和容忍

污点 (Taints)

前面的调度方式都是站在Pod的角度上，通过在Pod上添加属性，来确定Pod是否要调度到指定的Node上，其实我们也可以站在Node的角度上，通过在Node上添加**污点**属性，来决定是否允许Pod调度过来。

Node被设置上污点之后就与Pod之间存在了一种相斥的关系，进而拒绝Pod调度进来，甚至可以将已经存在的Pod驱逐出去。

污点的格式为：key=value:effect

key和value是污点的标签

- key=value的值可自己设置，后续用于Pod来容忍Node上的污点，来将Pod强行放在该Node上

effect描述污点的作用，支持如下三个选项：

- PreferNoSchedule: kubernetes将尽量避免把Pod调度到具有该污点的Node上，除非没有其他节点可调度
- NoSchedule: kubernetes将不会把Pod调度到具有该污点的Node上，但不会影响当前Node上已存在的Pod
- NoExecute: kubernetes将不会把Pod调度到具有该污点的Node上，同时也会将Node上已存在的Pod驱逐



使用kubectl设置和去除污点的命令示例如下:

```
# 设置污点
kubectl taint nodes node1 key=value:effect

# 去除污点
kubectl taint nodes node1 key:effect-

# 去除所有污点
kubectl taint nodes node1 key-
```

接下来, 演示下污点的效果:

1. 准备节点node1 (为了演示效果更加明显, 暂时停止node2节点)
2. 为node1节点设置一个污点: `tag=jax:PreferNoSchedule`; 然后创建pod1(pod1 可以)
3. 修改为node1节点设置一个污点: `tag=jax:NoSchedule`; 然后创建pod2(pod1 正常 pod2 失败)
4. 修改为node1节点设置一个污点: `tag=jax:NoExecute`; 然后创建pod3 (3个pod都失败)

```
# 为node1设置污点(PreferNoSchedule)
[root@k8s-master01 ~]# kubectl taint nodes node1 tag=jax:PreferNoSchedule

#可以通过 kubectl describe nodes node1 来查看污点是否打上

# 创建pod1
[root@k8s-master01 ~]# kubectl run taint1 --image=nginx:1.17.1 -n dev
[root@k8s-master01 ~]# kubectl get pods -n dev -o wide
NAME                                READY    STATUS    RESTARTS   AGE    IP              NODE
taint1-7665f7fd85-574h4             1/1      Running   0           2m24s  10.244.1.59    node1

# 为node1设置污点(取消PreferNoSchedule, 设置NoSchedule)
[root@k8s-master01 ~]# kubectl taint nodes node1 tag:PreferNoSchedule-
[root@k8s-master01 ~]# kubectl taint nodes node1 tag=jax:NoSchedule

# 创建pod2
[root@k8s-master01 ~]# kubectl run taint2 --image=nginx:1.17.1 -n dev
[root@k8s-master01 ~]# kubectl get pods taint2 -n dev -o wide
NAME                                READY    STATUS    RESTARTS   AGE    IP              NODE
taint1-7665f7fd85-574h4             1/1      Running   0           2m24s  10.244.1.59    node1
taint2-544694789-6zm1f              0/1      Pending   0           21s    <none>         <none>

# 为node1设置污点(取消NoSchedule, 设置NoExecute)
[root@k8s-master01 ~]# kubectl taint nodes node1 tag:NoSchedule-
```

```
[root@k8s-master01 ~]# kubectl taint nodes node1 tag=jax:NoExecute

# 创建pod3
[root@k8s-master01 ~]# kubectl run taint3 --image=nginx:1.17.1 -n dev
[root@k8s-master01 ~]# kubectl get pods -n dev -o wide
```

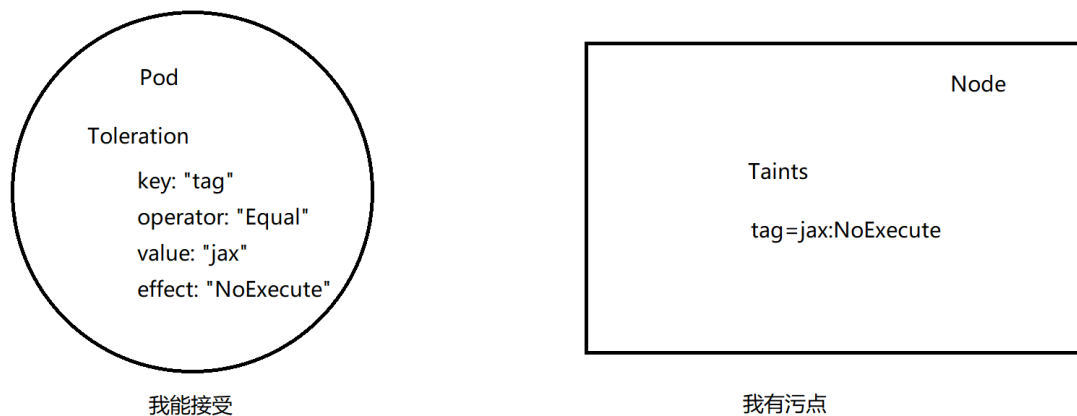
NAME	READY	STATUS	RESTARTS	AGE	IP	NODE
NOMINATED						
taint1-7665f7fd85-htkmp	0/1	Pending	0	35s	<none>	<none>
taint2-544694789-bn7wb	0/1	Pending	0	35s	<none>	<none>
taint3-6d78dbd749-tkktq	0/1	Pending	0	6s	<none>	<none>

小提示:

使用kubeadm搭建的集群，默认就会给master节点添加一个污点标记，所以pod就不会调度到master节点上。

容忍 (Toleration)

上面介绍了污点的作用，我们可以在node上添加污点用于拒绝pod调度上来，但是如果就是想将一个pod调度到一个有污点的node上去，这时候应该怎么做呢？这就要使用到**容忍**。



污点就是拒绝，容忍就是忽略，Node通过污点拒绝pod调度上去，Pod通过容忍忽略拒绝

下面先通过一个案例看下效果：

1. 上一小节，已经在node1节点上打上了 `NoExecute` 的污点，此时pod是调度不上去的
2. 本小节，可以通过给pod添加容忍，然后将其调度上去

创建pod-toleration.yaml,内容如下

```
apiVersion: v1
kind: Pod
metadata:
  name: pod-toleration
  namespace: dev
spec:
  containers:
    - name: nginx
      image: nginx:1.17.1
  tolerations: # 添加容忍
```

```
- key: "tag"          # 要容忍的污点的key
operator: "Equal"    # 操作符
value: "jax"         # 容忍的污点的value
effect: "NoExecute"  # 添加容忍的规则，这里必须和标记的污点规则相同
```

添加容忍之前的pod

```
[root@k8s-master01 ~]# kubectl get pods -n dev -o wide
```

NAME	READY	STATUS	RESTARTS	AGE	IP	NODE	NOMINATED
pod-toleration	0/1	Pending	0	3s	<none>	<none>	<none>

添加容忍之后的pod

```
[root@k8s-master01 ~]# kubectl get pods -n dev -o wide
```

NAME	READY	STATUS	RESTARTS	AGE	IP	NODE	NOMINATED
pod-toleration	1/1	Running	0	3s	10.244.1.62	node1	<none>

下面看一下容忍的详细配置:

```
[root@k8s-master01 ~]# kubectl explain pod.spec.tolerations
```

.....

FIELDS:

```
key          # 对应着要容忍的污点的键，空意味着匹配所有的键
value        # 对应着要容忍的污点的值
operator     # key-value的运算符，支持Equal和Exists（默认）
effect       # 对应污点的effect，空意味着匹配所有影响
tolerationSeconds # 容忍时间，当effect为NoExecute时生效，表示pod在Node上的停留时间
```

6. Pod控制器详解

6.1 Pod控制器介绍

Pod是kubernetes的最小管理单元，在kubernetes中，按照pod的创建方式可以将其分为两类：

- 自主式pod：kubernetes直接创建出来的Pod，这种pod删除后就没有了，也不会重建
- 控制器创建的pod：kubernetes通过控制器创建的pod，这种pod删除了之后还会自动重建

什么是Pod控制器

Pod控制器是管理pod的中间层，使用Pod控制器之后，只需要告诉Pod控制器，想要多少个什么样的Pod就可以了，它会创建出满足条件的Pod并确保每一个Pod资源处于用户期望的目标状态。如果Pod资源在运行中出现故障，它会基于指定策略重新编排Pod。

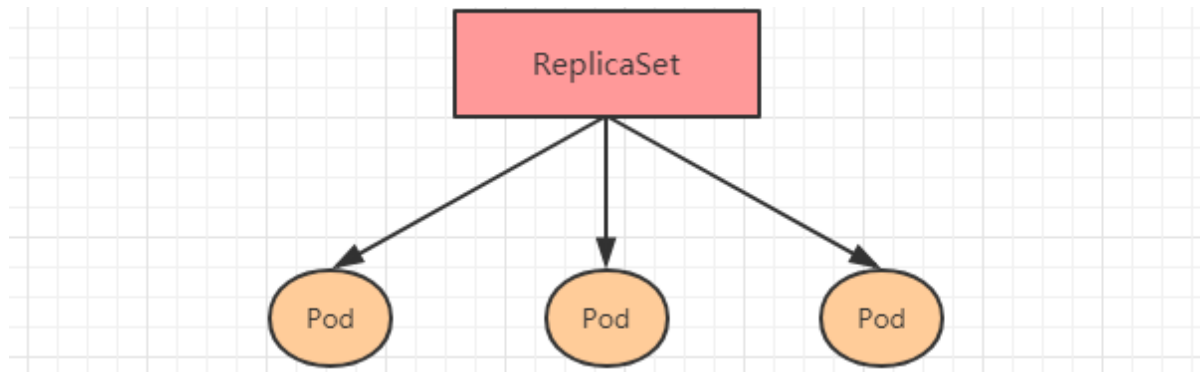
在kubernetes中，有很多类型的pod控制器，每种都有自己的适合的场景，常见的有下面这些：

- ReplicationController：比较原始的pod控制器，已经被废弃，由ReplicaSet替代
- ReplicaSet：保证副本数量一直维持在期望值，并支持pod数量扩缩容，镜像版本升级
- Deployment：通过控制ReplicaSet来控制Pod，并支持滚动升级、回退版本
- Horizontal Pod Autoscaler：可以根据集群负载自动水平调整Pod的数量，实现削峰填谷
- DaemonSet：在集群中的指定Node上运行且仅运行一个副本，一般用于守护进程类的任务

- Job: 它创建出来的pod只要完成任务就立即退出, 不需要重启或重建, 用于执行一次性任务
- Cronjob: 它创建的Pod负责周期性任务控制, 不需要持续后台运行
- StatefulSet: 管理有状态应用

6.2 ReplicaSet(RS)

ReplicaSet的主要作用是**保证一定数量的pod正常运行**, 它会持续监听这些Pod的运行状态, 一旦Pod发生故障, 就会重启或重建。同时它还支持对pod数量的扩缩容和镜像版本的升降级。



ReplicaSet的资源清单文件:

```

apiVersion: apps/v1 # 版本号
kind: ReplicaSet # 类型
metadata: # 元数据
  name: # rs名称
  namespace: # 所属命名空间
  labels: # 标签
  controller: rs
spec: # 详情描述
  replicas: 3 # 副本数量
  selector: # 选择器, 通过它指定该控制器管理哪些pod
    matchLabels: # Labels匹配规则
      app: nginx-pod
    matchExpressions: # Expressions匹配规则
      - {key: app, operator: In, values: [nginx-pod]}
  template: # 模板, 当副本数量不足时, 会根据下面的模板创建pod副本
    metadata:
      labels:
        app: nginx-pod
    spec:
      containers:
        - name: nginx
          image: nginx:1.17.1
          ports:
            - containerPort: 80
  
```

在这里面, 需要新了解的配置项就是 `spec` 下面几个选项:

- replicas: 指定副本数量, 其实就是当前rs创建出来的pod的数量, 默认为1
- selector: 选择器, 它的作用是建立pod控制器和pod之间的关联关系, 采用的Label Selector机制在pod模板上定义label, 在控制器上定义选择器, 就可以表明当前控制器能管理哪些pod了
- template: 模板, 就是当前控制器创建pod所使用的模板板, 里面其实就是前一章学过的pod的定义

创建ReplicaSet

创建pc-replicaset.yaml文件，内容如下：

```
apiVersion: apps/v1
kind: ReplicaSet
metadata:
  name: pc-replicaset
  namespace: dev
spec:
  replicas: 3
  selector:
    matchLabels:
      app: nginx-pod
  template:
    metadata:
      labels:
        app: nginx-pod
    spec:
      containers:
        - name: nginx
          image: nginx:1.17.1
```

创建rs

```
[root@k8s-master01 ~]# kubectl create -f pc-replicaset.yaml
replicaset.apps/pc-replicaset created
```

查看rs

DESIRED:期望副本数量

CURRENT:当前副本数量

READY:已经准备好提供服务的副本数量

```
[root@k8s-master01 ~]# kubectl get rs pc-replicaset -n dev -o wide
NAME                DESIRED   CURRENT  READY  AGE   CONTAINERS  IMAGES
SELECTOR
pc-replicaset 3          3        3      22s   nginx       nginx:1.17.1
app=nginx-pod
```

查看当前控制器创建出来的pod

这里发现控制器创建出来的pod的名称是在控制器名称后面拼接了-xxxxx随机码

```
[root@k8s-master01 ~]# kubectl get pod -n dev
NAME                READY  STATUS   RESTARTS  AGE
pc-replicaset-6vmvt 1/1    Running  0          54s
pc-replicaset-fmb8f 1/1    Running  0          54s
pc-replicaset-snrk2 1/1    Running  0          54s
```

扩缩容

编辑rs的副本数量，修改spec:replicas: 6即可

```
[root@k8s-master01 ~]# kubectl edit rs pc-replicaset -n dev
replicaset.apps/pc-replicaset edited
```

查看pod

```
[root@k8s-master01 ~]# kubectl get pods -n dev
NAME                READY  STATUS   RESTARTS  AGE
```

```
pc-replicaset-6vmvt 1/1 Running 0 114m
pc-replicaset-cftnp 1/1 Running 0 10s
pc-replicaset-fjlm6 1/1 Running 0 10s
pc-replicaset-fmb8f 1/1 Running 0 114m
pc-replicaset-s2whj 1/1 Running 0 10s
pc-replicaset-snrk2 1/1 Running 0 114m
```

当然也可以直接使用命令实现

使用scale命令实现扩缩容，后面--replicas=n直接指定目标数量即可

```
[root@k8s-master01 ~]# kubectl scale rs pc-replicaset --replicas=2 -n dev
replicaset.apps/pc-replicaset scaled
```

命令运行完毕，立即查看，发现已经有4个开始准备退出了

```
[root@k8s-master01 ~]# kubectl get pods -n dev
```

NAME	READY	STATUS	RESTARTS	AGE
pc-replicaset-6vmvt	0/1	Terminating	0	118m
pc-replicaset-cftnp	0/1	Terminating	0	4m17s
pc-replicaset-fjlm6	0/1	Terminating	0	4m17s
pc-replicaset-fmb8f	1/1	Running	0	118m
pc-replicaset-s2whj	0/1	Terminating	0	4m17s
pc-replicaset-snrk2	1/1	Running	0	118m

#稍等片刻，就只剩下2个了

```
[root@k8s-master01 ~]# kubectl get pods -n dev
```

NAME	READY	STATUS	RESTARTS	AGE
pc-replicaset-fmb8f	1/1	Running	0	119m
pc-replicaset-snrk2	1/1	Running	0	119m

镜像升级

编辑rs的容器镜像 - image: nginx:1.17.2

```
[root@k8s-master01 ~]# kubectl edit rs pc-replicaset -n dev
replicaset.apps/pc-replicaset edited
```

再次查看，发现镜像版本已经变更了

```
[root@k8s-master01 ~]# kubectl get rs -n dev -o wide
```

NAME	DESIRED	CURRENT	READY	AGE	CONTAINERS	IMAGES
...						
pc-replicaset	2	2	2	140m	nginx	nginx:1.17.2
...						

同样的道理，也可以使用命令完成这个工作

kubectl set image rs rs名称 容器=镜像版本 -n namespace

```
[root@k8s-master01 ~]# kubectl set image rs pc-replicaset nginx=nginx:1.17.1 -n dev
replicaset.apps/pc-replicaset image updated
```

再次查看，发现镜像版本已经变更了

```
[root@k8s-master01 ~]# kubectl get rs -n dev -o wide
```

NAME	DESIRED	CURRENT	READY	AGE	CONTAINERS	IMAGES
...						
pc-replicaset	2	2	2	145m	nginx	nginx:1.17.1
...						

删除ReplicaSet

```
# 使用kubectl delete命令会删除此RS以及它管理的Pod
# 在kubernetes删除RS前，会将RS的replicas调整为0，等待所有的Pod被删除后，在执行RS对象的删除

[root@k8s-master01 ~]# kubectl delete rs pc-replicaset -n dev
replicaset.apps "pc-replicaset" deleted
[root@k8s-master01 ~]# kubectl get pod -n dev -o wide
No resources found in dev namespace.

# 如果希望仅仅删除RS对象（保留Pod），可以使用kubectl delete命令时添加--cascade=false选项（不推荐）。

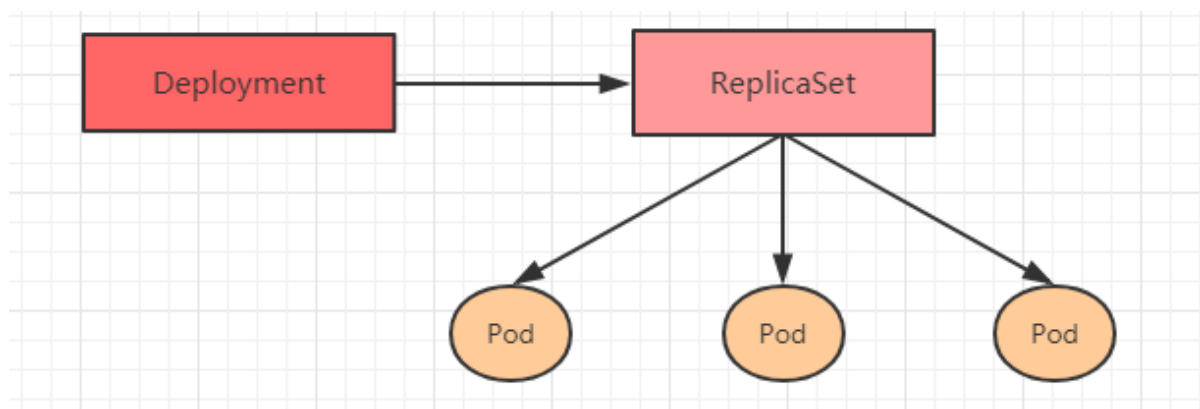
[root@k8s-master01 ~]# kubectl delete rs pc-replicaset -n dev --cascade=false
replicaset.apps "pc-replicaset" deleted
[root@k8s-master01 ~]# kubectl get pods -n dev
```

NAME	READY	STATUS	RESTARTS	AGE
pc-replicaset-cl82j	1/1	Running	0	75s
pc-replicaset-dslhb	1/1	Running	0	75s

```
# 也可以使用yaml直接删除(推荐)
[root@k8s-master01 ~]# kubectl delete -f pc-replicaset.yaml
replicaset.apps "pc-replicaset" deleted
```

6.3 Deployment(Deploy)

为了更好的解决服务编排的问题，kubernetes在V1.2版本开始，引入了Deployment控制器。值得一提的是，这种控制器并不直接管理pod，而是通过管理ReplicaSet来简介管理Pod，即：Deployment管理ReplicaSet，ReplicaSet管理Pod。所以Deployment比ReplicaSet功能更加强大。



Deployment主要功能有下面几个：

- 支持ReplicaSet的所有功能
- 支持发布的停止、继续
- 支持滚动升级和回滚版本

Deployment的资源清单文件：

```
apiVersion: apps/v1 # 版本号
kind: Deployment # 类型
metadata: # 元数据
  name: # rs名称
  namespace: # 所属命名空间
  labels: # 标签
  controller: deploy
spec: # 详情描述
```



```

replicas: 3 # 副本数量
revisionHistoryLimit: 3 # 保留历史版本
paused: false # 暂停部署，默认是false，为false时，创建出deployment后，马上创建pod
progressDeadlineSeconds: 600 # 部署超时时间（s），默认是600
strategy: # 策略
  type: RollingUpdate # 滚动更新策略
  rollingUpdate: # 滚动更新
    maxSurge: 30% # 最大额外可以存在的副本数，可以为百分比，也可以为整数
    maxUnavailable: 30% # 最大不可用状态的 Pod 的最大值，可以为百分比，也可以为整数
selector: # 选择器，通过它指定该控制器管理哪些pod
  matchLabels: # Labels匹配规则
    app: nginx-pod
  matchExpressions: # Expressions匹配规则
    - {key: app, operator: In, values: [nginx-pod]}
template: # 模板，当副本数量不足时，会根据下面的模板创建pod副本
  metadata:
    labels:
      app: nginx-pod
  spec:
    containers:
      - name: nginx
        image: nginx:1.17.1
        ports:
          - containerPort: 80

```

6.3.1 创建deployment

创建pc-deployment.yaml，内容如下：

```

apiVersion: apps/v1
kind: Deployment
metadata:
  name: pc-deployment
  namespace: dev
spec:
  replicas: 3
  selector:
    matchLabels:
      app: nginx-pod
  template:
    metadata:
      labels:
        app: nginx-pod
    spec:
      containers:
        - name: nginx
          image: nginx:1.17.1

```

创建deployment

```
[root@k8s-master01 ~]# kubectl create -f pc-deployment.yaml --record=true
deployment.apps/pc-deployment created
```

查看deployment

```
# UP-TO-DATE 最新版本的pod的数量
# AVAILABLE 当前可用的pod的数量
```

```
[root@k8s-master01 ~]# kubectl get deploy pc-deployment -n dev
```

NAME	READY	UP-TO-DATE	AVAILABLE	AGE
pc-deployment	3/3	3	3	15s

查看rs

发现rs的名称是在原来deployment的名字后面添加了一个10位数的随机串

```
[root@k8s-master01 ~]# kubectl get rs -n dev
```

NAME	DESIRED	CURRENT	READY	AGE
pc-deployment-6696798b78	3	3	3	23s

查看pod

```
[root@k8s-master01 ~]# kubectl get pods -n dev
```

NAME	READY	STATUS	RESTARTS	AGE
pc-deployment-6696798b78-d2c8n	1/1	Running	0	107s
pc-deployment-6696798b78-smpvp	1/1	Running	0	107s
pc-deployment-6696798b78-wvjd8	1/1	Running	0	107s

6.3.2 扩缩容

变更副本数量为5个

```
[root@k8s-master01 ~]# kubectl scale deploy pc-deployment --replicas=5 -n dev
deployment.apps/pc-deployment scaled
```

查看deployment

```
[root@k8s-master01 ~]# kubectl get deploy pc-deployment -n dev
```

NAME	READY	UP-TO-DATE	AVAILABLE	AGE
pc-deployment	5/5	5	5	2m

查看pod

```
[root@k8s-master01 ~]# kubectl get pods -n dev
```

NAME	READY	STATUS	RESTARTS	AGE
pc-deployment-6696798b78-d2c8n	1/1	Running	0	4m19s
pc-deployment-6696798b78-jxmdq	1/1	Running	0	94s
pc-deployment-6696798b78-mktqv	1/1	Running	0	93s
pc-deployment-6696798b78-smpvp	1/1	Running	0	4m19s
pc-deployment-6696798b78-wvjd8	1/1	Running	0	4m19s

编辑deployment的副本数量，修改spec:replicas: 4即可

```
[root@k8s-master01 ~]# kubectl edit deploy pc-deployment -n dev
deployment.apps/pc-deployment edited
```

查看pod

```
[root@k8s-master01 ~]# kubectl get pods -n dev
```

NAME	READY	STATUS	RESTARTS	AGE
pc-deployment-6696798b78-d2c8n	1/1	Running	0	5m23s
pc-deployment-6696798b78-jxmdq	1/1	Running	0	2m38s
pc-deployment-6696798b78-smpvp	1/1	Running	0	5m23s
pc-deployment-6696798b78-wvjd8	1/1	Running	0	5m23s

镜像更新

deployment支持两种更新策略: 重建更新 和 滚动更新, 可以通过 strategy 指定策略类型, 支持两个属性:

strategy: 指定新的Pod替换旧的Pod的策略， 支持两个属性：

type: 指定策略类型，支持两种策略

Recreate: 在创建出新的Pod之前会先杀掉所有已存在的Pod

RollingUpdate: 滚动更新，就是杀死一部分，就启动一部分，在更新过程中，存在两个版本Pod

rollingUpdate: 当type为RollingUpdate时生效，用于为RollingUpdate设置参数，支持两个属性：

maxUnavailable: 用来指定在升级过程中不可用Pod的最大数量，默认为25%，也就是关闭几个老pod

maxSurge: 用来指定在升级过程中可以超过期望的Pod的最大数量，默认为25%，也就是创建几个新pod

重建更新

1. 编辑pc-deployment.yaml,在spec节点下添加更新策略

spec:

strategy: # 策略

type: Recreate # 重建更新

2. 创建deploy进行验证

变更镜像 **kubectl set image ...**

[root@k8s-master01 ~]# **kubectl set image deployment pc-deployment**

nginx=nginx:1.17.2 -n dev

deployment.apps/pc-deployment image updated

观察升级过程

[root@k8s-master01 ~]# **kubectl get pods -n dev -w**

NAME	READY	STATUS	RESTARTS	AGE
pc-deployment-5d89bdfbf9-65qcw	1/1	Running	0	31s
pc-deployment-5d89bdfbf9-w5nzv	1/1	Running	0	31s
pc-deployment-5d89bdfbf9-xpt7w	1/1	Running	0	31s
pc-deployment-5d89bdfbf9-xpt7w	1/1	Terminating	0	41s
pc-deployment-5d89bdfbf9-65qcw	1/1	Terminating	0	41s
pc-deployment-5d89bdfbf9-w5nzv	1/1	Terminating	0	41s
pc-deployment-675d469f8b-grn8z	0/1	Pending	0	0s
pc-deployment-675d469f8b-hb14v	0/1	Pending	0	0s
pc-deployment-675d469f8b-67nz2	0/1	Pending	0	0s
pc-deployment-675d469f8b-grn8z	0/1	ContainerCreating	0	0s
pc-deployment-675d469f8b-hb14v	0/1	ContainerCreating	0	0s
pc-deployment-675d469f8b-67nz2	0/1	ContainerCreating	0	0s
pc-deployment-675d469f8b-grn8z	1/1	Running	0	1s
pc-deployment-675d469f8b-67nz2	1/1	Running	0	1s
pc-deployment-675d469f8b-hb14v	1/1	Running	0	2s

滚动更新

1. 编辑pc-deployment.yaml,在spec节点下添加更新策略

```
spec:
  strategy: # 策略
    type: RollingUpdate # 滚动更新策略
  rollingUpdate:
    maxSurge: 25%
    maxUnavailable: 25%
```

2. 创建deploy进行验证

变更镜像

```
[root@k8s-master01 ~]# kubectl set image deployment pc-deployment
nginx=nginx:1.17.3 -n dev
deployment.apps/pc-deployment image updated
```

观察升级过程

```
[root@k8s-master01 ~]# kubectl get pods -n dev -w
```

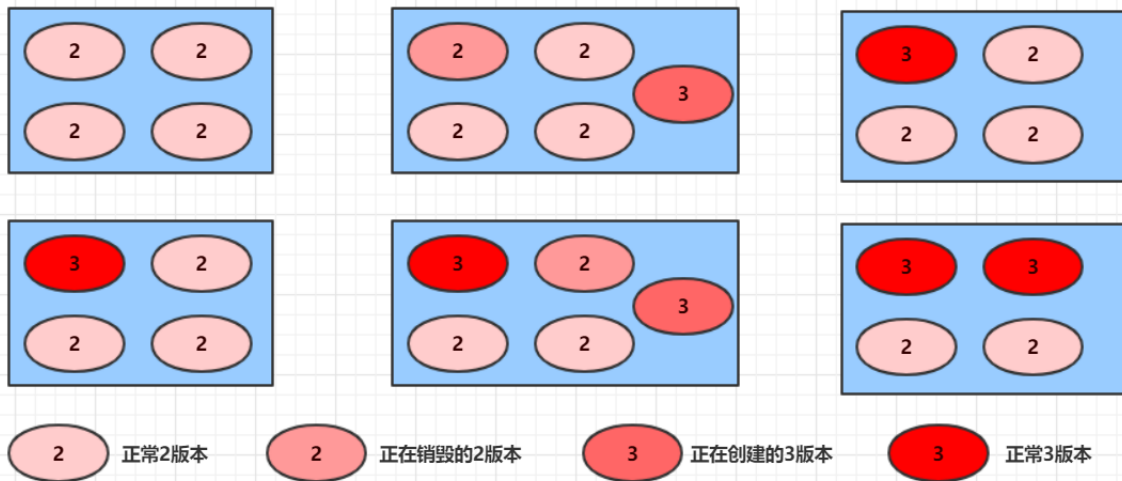
NAME	READY	STATUS	RESTARTS	AGE
pc-deployment-c848d767-8rbzt	1/1	Running	0	31m
pc-deployment-c848d767-h4p68	1/1	Running	0	31m
pc-deployment-c848d767-h1mz4	1/1	Running	0	31m
pc-deployment-c848d767-rrqcn	1/1	Running	0	31m
pc-deployment-966bf7f44-226rx	0/1	Pending	0	0s
pc-deployment-966bf7f44-226rx	0/1	ContainerCreating	0	0s
pc-deployment-966bf7f44-226rx	1/1	Running	0	1s
pc-deployment-c848d767-h4p68	0/1	Terminating	0	34m
pc-deployment-966bf7f44-cnd44	0/1	Pending	0	0s
pc-deployment-966bf7f44-cnd44	0/1	ContainerCreating	0	0s
pc-deployment-966bf7f44-cnd44	1/1	Running	0	2s
pc-deployment-c848d767-h1mz4	0/1	Terminating	0	34m
pc-deployment-966bf7f44-px48p	0/1	Pending	0	0s
pc-deployment-966bf7f44-px48p	0/1	ContainerCreating	0	0s
pc-deployment-966bf7f44-px48p	1/1	Running	0	0s
pc-deployment-c848d767-8rbzt	0/1	Terminating	0	34m
pc-deployment-966bf7f44-dkmqp	0/1	Pending	0	0s
pc-deployment-966bf7f44-dkmqp	0/1	ContainerCreating	0	0s
pc-deployment-966bf7f44-dkmqp	1/1	Running	0	2s
pc-deployment-c848d767-rrqcn	0/1	Terminating	0	34m

至此，新版本的pod创建完毕，就版本的pod销毁完毕

中间过程是滚动进行的，也就是边销毁边创建

滚动更新的过程：

滚动更新：每一轮,销毁一个2版本的Pod, 创建一个3版本的Pod



镜像更新中rs的变化

```
# 查看rs,发现原来的rs的依旧存在,只是pod数量变为了0,而后又新产生了一个rs, pod数量为4
```

```
# 其实这就是deployment能够进行版本回退的奥妙所在,后面会详细解释
```

```
[root@k8s-master01 ~]# kubectl get rs -n dev
```

NAME	DESIRED	CURRENT	READY	AGE
pc-deployment-6696798b78	0	0	0	7m37s
pc-deployment-6696798b11	0	0	0	5m37s
pc-deployment-c848d76789	4	4	4	72s

6.3.3 版本回退

deployment支持版本升级过程中的暂停、继续功能以及版本回退等诸多功能,下面具体来看.

kubectl rollout: 版本升级相关功能,支持下面的选项:

- status 显示当前升级状态
- history 显示 升级历史记录
- pause 暂停版本升级过程
- resume 继续已经暂停的版本升级过程
- restart 重启版本升级过程
- undo 回滚到上一级版本 (可以使用--to-revision回滚到指定版本)

```
# 查看当前升级版本的状态
```

```
[root@k8s-master01 ~]# kubectl rollout status deploy pc-deployment -n dev
deployment "pc-deployment" successfully rolled out
```

```
# 查看升级历史记录
```

```
[root@k8s-master01 ~]# kubectl rollout history deploy pc-deployment -n dev
deployment.apps/pc-deployment
```

```
REVISION  CHANGE-CAUSE
```

```
1          kubectl create --filename=pc-deployment.yaml --record=true
2          kubectl create --filename=pc-deployment.yaml --record=true
3          kubectl create --filename=pc-deployment.yaml --record=true
```

```
# 可以发现有三版本记录,说明完成过两次升级
```

```
# 版本回滚
```

```
# 这里直接使用--to-revision=1回滚到了1版本, 如果省略这个选项,就是回退到上个版本,就是2版本
```

```
[root@k8s-master01 ~]# kubectl rollout undo deployment pc-deployment --to-revision=1 -n dev
deployment.apps/pc-deployment rolled back

# 查看发现，通过nginx镜像版本可以发现到了第一版
[root@k8s-master01 ~]# kubectl get deploy -n dev -o wide
NAME                                READY    UP-TO-DATE    AVAILABLE    AGE    CONTAINERS    IMAGES
pc-deployment                       4/4      4              4            74m    nginx         nginx:1.17.1

# 查看rs，发现第一个rs中有4个pod运行，后面两个版本的rs中pod为运行
# 其实deployment之所以可是实现版本的回滚，就是通过记录下历史rs来实现的，
# 一旦想回滚到哪个版本，只需要将当前版本pod数量降为0，然后将回滚版本的pod提升为目标数量就可以了
[root@k8s-master01 ~]# kubectl get rs -n dev
NAME                                DESIRED    CURRENT    READY    AGE
pc-deployment-6696798b78            4          4          4        78m
pc-deployment-966bf7f44              0          0          0        37m
pc-deployment-c848d767              0          0          0        71m
```

6.3.4 金丝雀发布

Deployment控制器支持控制更新过程中的控制，如“暂停(pause)”或“继续(resume)”更新操作。

比如有一批新的Pod资源创建完成后立即暂停更新过程，此时，仅存在一部分新版本的应用，主体部分还是旧的版本。然后，再筛选一小部分的用户请求路由到新版本的Pod应用，继续观察能否稳定地按期望的方式运行。确定没问题之后再继续完成余下的Pod资源滚动更新，否则立即回滚更新操作。这就是所谓的金丝雀发布。

```
# 更新deployment的版本，并配置暂停deployment
[root@k8s-master01 ~]# kubectl set image deploy pc-deployment nginx=nginx:1.17.4 -n dev && kubectl rollout pause deployment pc-deployment -n dev
deployment.apps/pc-deployment image updated
deployment.apps/pc-deployment paused

#观察更新状态
[root@k8s-master01 ~]# kubectl rollout status deploy pc-deployment -n dev
waiting for deployment "pc-deployment" rollout to finish: 2 out of 4 new replicas have been updated...

# 监控更新的过程，可以看到已经新增了一个资源，但是并未按照预期的状态去删除一个旧的资源，就是因为使用了pause暂停命令

[root@k8s-master01 ~]# kubectl get rs -n dev -o wide
NAME                                DESIRED    CURRENT    READY    AGE    CONTAINERS    IMAGES
pc-deployment-5d89bdfbf9            3          3          3        19m    nginx         nginx:1.17.1
pc-deployment-675d469f8b            0          0          0        14m    nginx         nginx:1.17.2
pc-deployment-6c9f56fcfb            2          2          2        3m16s    nginx         nginx:1.17.4
[root@k8s-master01 ~]# kubectl get pods -n dev
NAME                                READY    STATUS    RESTARTS    AGE
pc-deployment-5d89bdfbf9-rj8sq      1/1     Running   0            7m33s
```

```
pc-deployment-5d89bdfbf9-ttwgg 1/1 Running 0 7m35s
pc-deployment-5d89bdfbf9-v4wvc 1/1 Running 0 7m34s
pc-deployment-6c9f56fcfb-996rt 1/1 Running 0 3m31s
pc-deployment-6c9f56fcfb-j2gtj 1/1 Running 0 3m31s
```

确保更新的pod没问题了，继续更新

```
[root@k8s-master01 ~]# kubectl rollout resume deploy pc-deployment -n dev
deployment.apps/pc-deployment resumed
```

查看最后的更新情况

```
[root@k8s-master01 ~]# kubectl get rs -n dev -o wide
```

NAME	DESIRED	CURRENT	READY	AGE	CONTAINERS
pc-deployment-5d89bdfbf9	0	0	0	21m	nginx
pc-deployment-675d469f8b	0	0	0	16m	nginx
pc-deployment-6c9f56fcfb	4	4	4	5m11s	nginx

```
[root@k8s-master01 ~]# kubectl get pods -n dev
```

NAME	READY	STATUS	RESTARTS	AGE
pc-deployment-6c9f56fcfb-7bfbh	1/1	Running	0	37s
pc-deployment-6c9f56fcfb-996rt	1/1	Running	0	5m27s
pc-deployment-6c9f56fcfb-j2gtj	1/1	Running	0	5m27s
pc-deployment-6c9f56fcfb-rf84v	1/1	Running	0	37s

删除Deployment

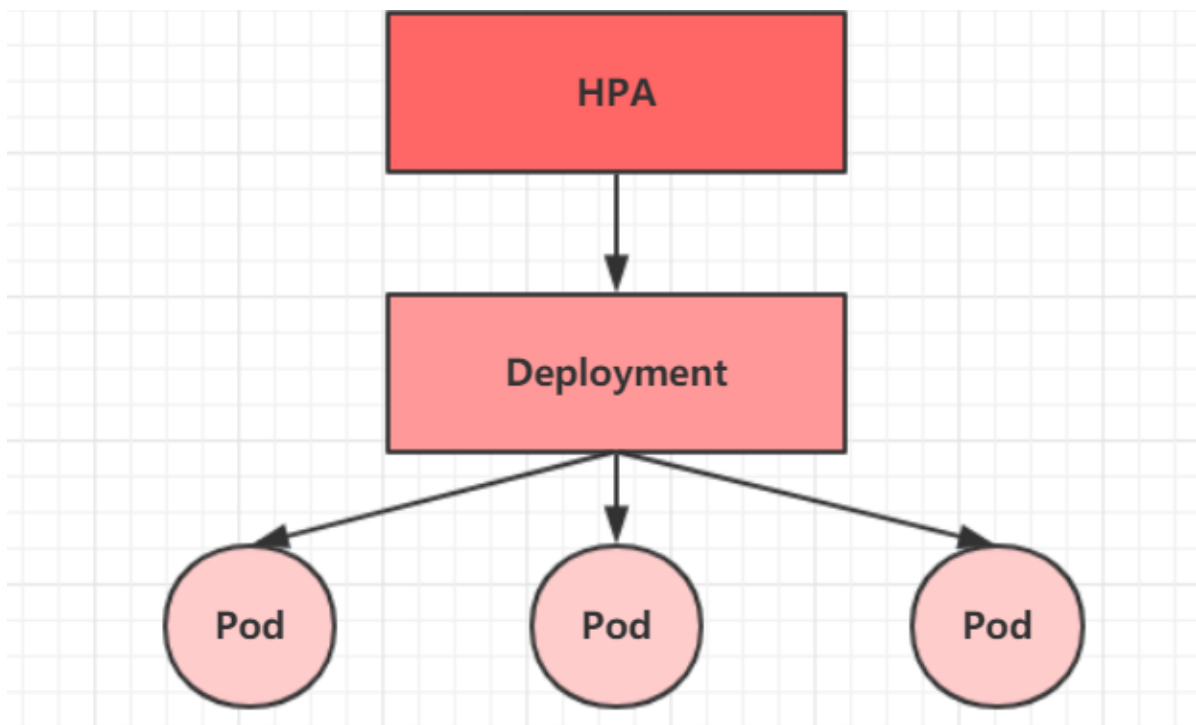
删除deployment，其下的rs和pod也将被删除

```
[root@k8s-master01 ~]# kubectl delete -f pc-deployment.yaml
deployment.apps "pc-deployment" deleted
```

6.4 Horizontal Pod Autoscaler(HPA)

在前面的课程中，我们已经可以实现通过手工执行 `kubectl scale` 命令实现Pod扩容或缩容，但是这显然不符合Kubernetes的定位目标--自动化、智能化。Kubernetes期望可以实现通过监测Pod的使用情况，实现pod数量的自动调整，于是就产生了Horizontal Pod Autoscaler（HPA）这种控制器。

HPA可以获取每个Pod利用率，然后和HPA中定义的指标进行对比，同时计算出需要伸缩的具体值，最后实现Pod的数量的调整。其实HPA与之前的Deployment一样，也属于一种Kubernetes资源对象，它通过追踪分析RC控制的所有目标Pod的负载变化情况，来确定是否需要针对性地调整目标Pod的副本数，这是HPA的实现原理。



接下来，我们来做一个实验

6.4.1 安装metrics-server

metrics-server可以用来收集集群中的资源使用情况

```
# 安装git
[root@k8s-master01 ~]# yum install git -y
# 获取metrics-server，注意使用的版本
[root@k8s-master01 ~]# git clone -b v0.3.6 https://github.com/kubernetes-incubator/metrics-server
# 修改deployment，注意修改的是镜像和初始化参数
[root@k8s-master01 ~]# cd /root/metrics-server/deploy/1.8+/
[root@k8s-master01 1.8+]# vim metrics-server-deployment.yaml
按图中添加下面选项
hostNetwork: true
image: registry.cn-hangzhou.aliyuncs.com/google_containers/metrics-server-amd64:v0.3.6
args:
- --kubelet-insecure-tls
- --kubelet-preferred-address-types=InternalIP,Hostname,InternalDNS,ExternalDNS,ExternalIP
```

```
hostNetwork: true
serviceAccountName: metrics-server
volumes:
# mount in tmp so we can safely use from-scratch images and/or read-only containers
- name: tmp-dir
  emptyDir: {}
containers:
- name: metrics-server
  image: registry.cn-hangzhou.aliyuncs.com/google_containers/metrics-server-amd64:v0.3.6
  imagePullPolicy: Always
  args:
  - --kubelet-insecure-tls
  - --kubelet-preferred-address-types=InternalIP,Hostname,InternalDNS,ExternalDNS,ExternalIP
  volumeMounts:
```

```
# 安装metrics-server
[root@k8s-master01 1.8+]# kubectl apply -f ./
```



```
# 查看pod运行情况
[root@k8s-master01 1.8+]# kubectl get pod -n kube-system
metrics-server-6b976979db-2xwbj    1/1    Running    0          90s

# 使用kubectl top node 查看资源使用情况
[root@k8s-master01 1.8+]# kubectl top node
NAME             CPU(cores)    CPU%    MEMORY(bytes)    MEMORY%
k8s-master01     289m         14%     1582Mi           54%
k8s-node01       81m          4%      1195Mi           40%
k8s-node02       72m          3%      1211Mi           41%
[root@k8s-master01 1.8+]# kubectl top pod -n kube-system
NAME                                CPU(cores)    MEMORY(bytes)
coredns-6955765f44-7ptsb           3m            9Mi
coredns-6955765f44-vcwr5           3m            8Mi
etcd-master                         14m           145Mi
...
```

至此,metrics-server安装完成

6.4.2 准备deployment和servie

创建pc-hpa-pod.yaml文件, 内容如下:

```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: nginx
  namespace: dev
spec:
  strategy: # 策略
    type: RollingUpdate # 滚动更新策略
  replicas: 1
  selector:
    matchLabels:
      app: nginx-pod
  template:
    metadata:
      labels:
        app: nginx-pod
    spec:
      containers:
        - name: nginx
          image: nginx:1.17.1
          resources: # 资源配额
            limits: # 限制资源 (上限)
              cpu: "1" # CPU限制, 单位是core数
            requests: # 请求资源 (下限)
              cpu: "100m" # CPU限制, 单位是core数
```

```
# 创建deployment
[root@k8s-master01 1.8+]# kubectl run nginx --image=nginx:1.17.1 --
requests=cpu=100m -n dev
# 创建service
[root@k8s-master01 1.8+]# kubectl expose deployment nginx --type=NodePort --
port=80 -n dev
```

```
# 查看
[root@k8s-master01 1.8+]# kubectl get deployment,pod,svc -n dev
```

NAME	READY	UP-TO-DATE	AVAILABLE	AGE
deployment.apps/nginx	1/1	1	1	47s

NAME	READY	STATUS	RESTARTS	AGE
pod/nginx-7df9756ccc-bh8dr	1/1	Running	0	47s

NAME	TYPE	CLUSTER-IP	EXTERNAL-IP	PORT(S)	AGE
service/nginx	NodePort	10.101.18.29	<none>	80:31830/TCP	35s

6.4.3 部署HPA

创建pc-hpa.yaml文件，内容如下：

```
apiVersion: autoscaling/v1
kind: HorizontalPodAutoscaler
metadata:
  name: pc-hpa
  namespace: dev
spec:
  minReplicas: 1 #最小pod数量
  maxReplicas: 10 #最大pod数量
  targetCPUUtilizationPercentage: 3 # CPU使用率指标
  scaleTargetRef: # 指定要控制的nginx信息
    apiVersion: /v1
    kind: Deployment
    name: nginx
```

```
# 创建hpa
[root@k8s-master01 1.8+]# kubectl create -f pc-hpa.yaml
horizontalpodautoscaler.autoscaling/pc-hpa created

# 查看hpa
[root@k8s-master01 1.8+]# kubectl get hpa -n dev
```

NAME	REFERENCE	TARGETS	MINPODS	MAXPODS	REPLICAS	AGE
pc-hpa	Deployment/nginx	0%/3%	1	10	1	62s

6.4.4 测试

使用压测工具对service地址 192.168.5.4:31830 进行压测，然后通过控制台查看hpa和pod的变化

hpa变化

```
[root@k8s-master01 ~]# kubectl get hpa -n dev -w
```

NAME	REFERENCE	TARGETS	MINPODS	MAXPODS	REPLICAS	AGE
pc-hpa	Deployment/nginx	0%/3%	1	10	1	4m11s
pc-hpa	Deployment/nginx	0%/3%	1	10	1	5m19s
pc-hpa	Deployment/nginx	22%/3%	1	10	1	6m50s
pc-hpa	Deployment/nginx	22%/3%	1	10	4	7m5s
pc-hpa	Deployment/nginx	22%/3%	1	10	8	7m21s
pc-hpa	Deployment/nginx	6%/3%	1	10	8	7m51s
pc-hpa	Deployment/nginx	0%/3%	1	10	8	9m6s
pc-hpa	Deployment/nginx	0%/3%	1	10	8	13m
pc-hpa	Deployment/nginx	0%/3%	1	10	1	14m

deployment变化

```
[root@k8s-master01 ~]# kubectl get deployment -n dev -w
```

NAME	READY	UP-TO-DATE	AVAILABLE	AGE
nginx	1/1	1	1	11m
nginx	1/4	1	1	13m
nginx	1/4	1	1	13m
nginx	1/4	1	1	13m
nginx	1/4	4	1	13m
nginx	1/8	4	1	14m
nginx	1/8	4	1	14m
nginx	1/8	4	1	14m
nginx	1/8	8	1	14m
nginx	2/8	8	2	14m
nginx	3/8	8	3	14m
nginx	4/8	8	4	14m
nginx	5/8	8	5	14m
nginx	6/8	8	6	14m
nginx	7/8	8	7	14m
nginx	8/8	8	8	15m
nginx	8/1	8	8	20m
nginx	8/1	8	8	20m
nginx	1/1	1	1	20m

pod变化

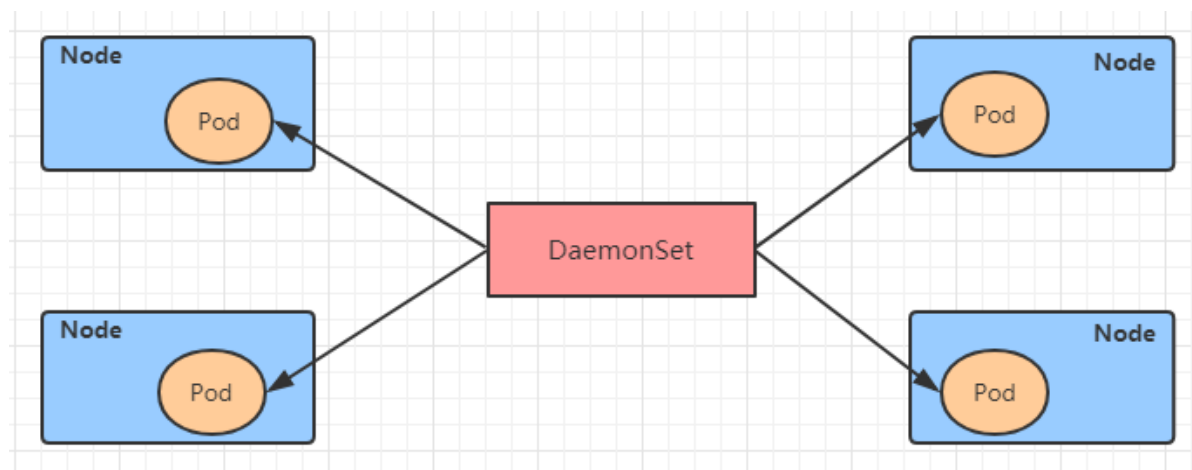
```
[root@k8s-master01 ~]# kubectl get pods -n dev -w
```

NAME	READY	STATUS	RESTARTS	AGE
nginx-7df9756ccc-bh8dr	1/1	Running	0	11m
nginx-7df9756ccc-cpgrv	0/1	Pending	0	0s
nginx-7df9756ccc-8zhwk	0/1	Pending	0	0s
nginx-7df9756ccc-rr9bn	0/1	Pending	0	0s
nginx-7df9756ccc-cpgrv	0/1	ContainerCreating	0	0s
nginx-7df9756ccc-8zhwk	0/1	ContainerCreating	0	0s
nginx-7df9756ccc-rr9bn	0/1	ContainerCreating	0	0s
nginx-7df9756ccc-m9gsj	0/1	Pending	0	0s
nginx-7df9756ccc-g56qb	0/1	Pending	0	0s
nginx-7df9756ccc-s19c6	0/1	Pending	0	0s
nginx-7df9756ccc-fgst7	0/1	Pending	0	0s
nginx-7df9756ccc-g56qb	0/1	ContainerCreating	0	0s
nginx-7df9756ccc-m9gsj	0/1	ContainerCreating	0	0s
nginx-7df9756ccc-s19c6	0/1	ContainerCreating	0	0s

nginx-7df9756ccc-fgst7	0/1	ContainerCreating	0	0s
nginx-7df9756ccc-8zhwk	1/1	Running	0	19s
nginx-7df9756ccc-rr9bn	1/1	Running	0	30s
nginx-7df9756ccc-m9gsj	1/1	Running	0	21s
nginx-7df9756ccc-cpgrv	1/1	Running	0	47s
nginx-7df9756ccc-sl9c6	1/1	Running	0	33s
nginx-7df9756ccc-g56qb	1/1	Running	0	48s
nginx-7df9756ccc-fgst7	1/1	Running	0	66s
nginx-7df9756ccc-fgst7	1/1	Terminating	0	6m50s
nginx-7df9756ccc-8zhwk	1/1	Terminating	0	7m5s
nginx-7df9756ccc-cpgrv	1/1	Terminating	0	7m5s
nginx-7df9756ccc-g56qb	1/1	Terminating	0	6m50s
nginx-7df9756ccc-rr9bn	1/1	Terminating	0	7m5s
nginx-7df9756ccc-m9gsj	1/1	Terminating	0	6m50s
nginx-7df9756ccc-sl9c6	1/1	Terminating	0	6m50s

6.5 DaemonSet(DS)

DaemonSet类型的控制器可以保证在集群中的每一台（或指定）节点上都运行一个副本。一般适用于日志收集、节点监控等场景。也就是说，如果一个Pod提供的功能是节点级别的（每个节点都需要且只需要一个），那么这类Pod就适合使用DaemonSet类型的控制器创建。



DaemonSet控制器的特点：

- 每当向集群中添加一个节点时，指定的 Pod 副本也将添加到该节点上
- 当节点从集群中移除时，Pod 也就被垃圾回收了

下面先来看下DaemonSet的资源清单文件

```

apiVersion: apps/v1 # 版本号
kind: DaemonSet # 类型
metadata: # 元数据
  name: # rs名称
  namespace: # 所属命名空间
  labels: #标签
    controller: daemonset
spec: # 详情描述
  revisionHistoryLimit: 3 # 保留历史版本
  updateStrategy: # 更新策略
    type: RollingUpdate # 滚动更新策略
    rollingUpdate: # 滚动更新
      maxUnavailable: 1 # 最大不可用状态的 Pod 的最大值，可以为百分比，也可以为整数

```

```

selector: # 选择器, 通过它指定该控制器管理哪些pod
  matchLabels: # Labels匹配规则
    app: nginx-pod
  matchExpressions: # Expressions匹配规则
    - {key: app, operator: In, values: [nginx-pod]}
template: # 模板, 当副本数量不足时, 会根据下面的模板创建pod副本
  metadata:
    labels:
      app: nginx-pod
  spec:
    containers:
      - name: nginx
        image: nginx:1.17.1
        ports:
          - containerPort: 80

```

创建pc-daemonset.yaml, 内容如下:

```

apiVersion: apps/v1
kind: DaemonSet
metadata:
  name: pc-daemonset
  namespace: dev
spec:
  selector:
    matchLabels:
      app: nginx-pod
  template:
    metadata:
      labels:
        app: nginx-pod
    spec:
      containers:
        - name: nginx
          image: nginx:1.17.1

```

创建daemonset

```

[root@k8s-master01 ~]# kubectl create -f pc-daemonset.yaml
daemonset.apps/pc-daemonset created

```

查看daemonset

```

[root@k8s-master01 ~]# kubectl get ds -n dev -o wide
NAME           DESIRED  CURRENT  READY  UP-TO-DATE  AVAILABLE  AGE    CONTAINERS
IMAGES
pc-daemonset   2        2        2      2           2          24s    nginx
nginx:1.17.1

```

查看pod, 发现在每个Node上都运行一个pod

```

[root@k8s-master01 ~]# kubectl get pods -n dev -o wide
NAME                READY  STATUS   RESTARTS  AGE  IP            NODE
pc-daemonset-9bck8  1/1    Running  0         37s  10.244.1.43   node1
pc-daemonset-k224w  1/1    Running  0         37s  10.244.2.74   node2

```

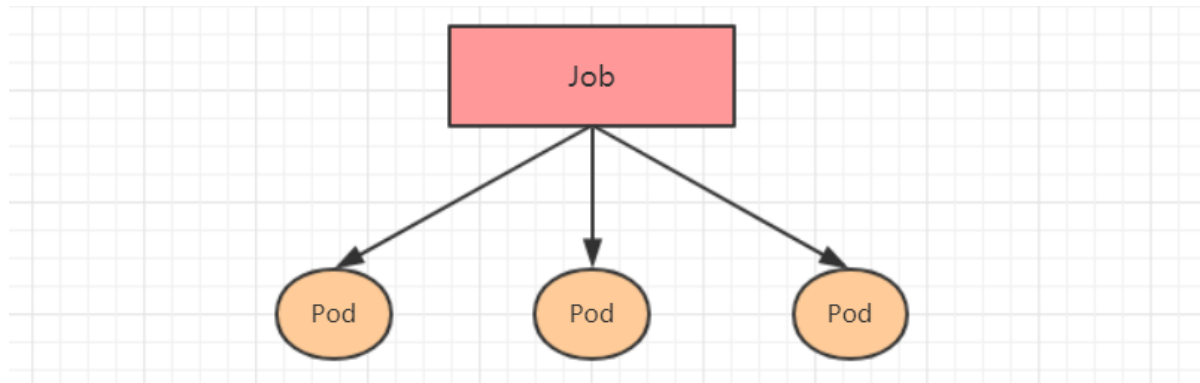
删除daemonset

```
[root@k8s-master01 ~]# kubectl delete -f pc-daemonset.yaml
daemonset.apps "pc-daemonset" deleted
```

6.6 Job

Job，主要用于负责**批量处理(一次要处理指定数量任务)**短暂的一次性(每个任务仅运行一次就结束)任务。Job特点如下：

- 当Job创建的pod执行成功结束时，Job将记录成功结束的pod数量
- 当成功结束的pod达到指定的数量时，Job将完成执行



Job的资源清单文件：

```
apiVersion: batch/v1 # 版本号
kind: Job # 类型
metadata: # 元数据
  name: # rs名称
  namespace: # 所属命名空间
  labels: # 标签
  controller: job
spec: # 详情描述
  completions: 1 # 指定job需要成功运行Pods的次数。默认值：1
  parallelism: 1 # 指定job在任一时刻应该并发运行Pods的数量。默认值：1
  activeDeadlineSeconds: 30 # 指定job可运行的时间期限，超过时间还未结束，系统将会尝试进行终止。
  backoffLimit: 6 # 指定job失败后进行重试的次数。默认是6
  manualSelector: true # 是否可以使用selector选择器选择pod，默认是false
  selector: # 选择器，通过它指定该控制器管理哪些pod
    matchLabels: # Labels匹配规则
      app: counter-pod
    matchExpressions: # Expressions匹配规则
      - {key: app, operator: In, values: [counter-pod]}
  template: # 模板，当副本数量不足时，会根据下面的模板创建pod副本
    metadata:
      labels:
        app: counter-pod
    spec:
      restartPolicy: Never # 重启策略只能设置为Never或者OnFailure
      containers:
        - name: counter
          image: busybox:1.30
```

```
command: ["bin/sh","-c","for i in 9 8 7 6 5 4 3 2 1; do echo $i;sleep 2;done"]
```

关于重启策略设置的说明:

如果指定为**OnFailure**, 则**job**会在**pod**出现故障时重启容器, 而不是创建**pod**, **failed**次数不变

如果指定为**Never**, 则**job**会在**pod**出现故障时创建新的**pod**, 并且故障**pod**不会消失, 也不会重启, **failed**次数加1

如果指定为**Always**的话, 就意味着一直重启, 意味着**job**任务会重复去执行了, 当然不对, 所以不能设置为**Always**

创建pc-job.yaml, 内容如下:

```
apiVersion: batch/v1
kind: Job
metadata:
  name: pc-job
  namespace: dev
spec:
  manualSelector: true
  selector:
    matchLabels:
      app: counter-pod
  template:
    metadata:
      labels:
        app: counter-pod
    spec:
      restartPolicy: Never
      containers:
        - name: counter
          image: busybox:1.30
          command: ["bin/sh","-c","for i in 9 8 7 6 5 4 3 2 1; do echo $i;sleep 3;done"]
```

创建job

```
[root@k8s-master01 ~]# kubectl create -f pc-job.yaml
job.batch/pc-job created
```

查看job

```
[root@k8s-master01 ~]# kubectl get job -n dev -o wide -w
```

NAME	COMPLETIONS	DURATION	AGE	CONTAINERS	IMAGES	SELECTOR
pc-job	0/1	21s	21s	counter	busybox:1.30	app=counter-pod
pc-job	1/1	31s	79s	counter	busybox:1.30	app=counter-pod

通过观察pod状态可以看到, pod在运行完毕任务后, 就会变成Completed状态

```
[root@k8s-master01 ~]# kubectl get pods -n dev -w
```

NAME	READY	STATUS	RESTARTS	AGE
pc-job-rxg96	1/1	Running	0	29s
pc-job-rxg96	0/1	Completed	0	33s

接下来, 调整下pod运行的总数量和并行数量 即: 在spec下设置下面两个选项

completions: 6 # 指定job需要成功运行Pods的次数为6

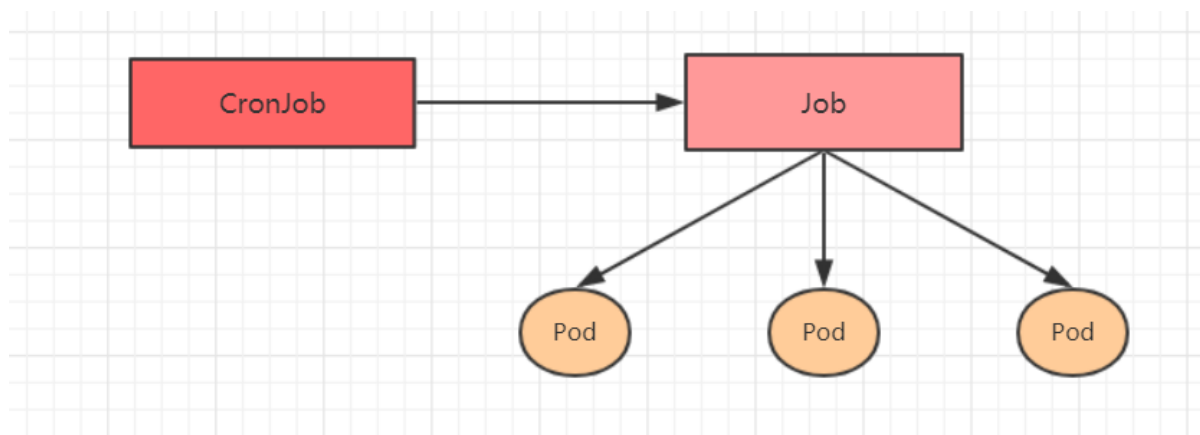
```
# parallelism: 3 # 指定job并发运行Pods的数量为3
# 然后重新运行job，观察效果，此时会发现，job会每次运行3个pod，总共执行了6个pod
[root@k8s-master01 ~]# kubectl get pods -n dev -w
```

NAME	READY	STATUS	RESTARTS	AGE
pc-job-684ft	1/1	Running	0	5s
pc-job-jhj49	1/1	Running	0	5s
pc-job-pfcvh	1/1	Running	0	5s
pc-job-684ft	0/1	Completed	0	11s
pc-job-v7rhr	0/1	Pending	0	0s
pc-job-v7rhr	0/1	Pending	0	0s
pc-job-v7rhr	0/1	ContainerCreating	0	0s
pc-job-jhj49	0/1	Completed	0	11s
pc-job-fhwf7	0/1	Pending	0	0s
pc-job-fhwf7	0/1	Pending	0	0s
pc-job-pfcvh	0/1	Completed	0	11s
pc-job-5vg2j	0/1	Pending	0	0s
pc-job-fhwf7	0/1	ContainerCreating	0	0s
pc-job-5vg2j	0/1	Pending	0	0s
pc-job-5vg2j	0/1	ContainerCreating	0	0s
pc-job-fhwf7	1/1	Running	0	2s
pc-job-v7rhr	1/1	Running	0	2s
pc-job-5vg2j	1/1	Running	0	3s
pc-job-fhwf7	0/1	Completed	0	12s
pc-job-v7rhr	0/1	Completed	0	12s
pc-job-5vg2j	0/1	Completed	0	12s

```
# 删除job
[root@k8s-master01 ~]# kubectl delete -f pc-job.yaml
job.batch "pc-job" deleted
```

6.7 CronJob(CJ)

CronJob控制器以Job控制器资源为其管控对象，并借助它管理pod资源对象，Job控制器定义的作业任务在其控制器资源创建之后便会立即执行，但CronJob可以以类似于Linux操作系统的周期性任务作业计划的方式控制其运行**时间点**及**重复运行**的方式。也就是说，**CronJob可以在特定的时间点(反复的)去运行job任务。**



CronJob的资源清单文件:

```
apiVersion: batch/v1beta1 # 版本号
kind: CronJob # 类型
metadata: # 元数据
  name: # rs名称
```



```

namespace: # 所属命名空间
labels: # 标签
controller: cronjob
spec: # 详情描述
  schedule: # cron格式的调度运行时间点,用于控制任务在什么时间执行
  concurrencyPolicy: # 并发执行策略,用于定义前一次作业运行尚未完成时是否以及如何运行后一次的作业
  failedJobHistoryLimit: # 为失败的任务执行保留的历史记录数,默认为1
  successfulJobHistoryLimit: # 为成功的任务执行保留的历史记录数,默认为3
  startingDeadlineSeconds: # 启动作业错误的超时时长
  jobTemplate: # job控制器模板,用于为cronjob控制器生成job对象;下面其实就是job的定义
    metadata:
    spec:
      completions: 1
      parallelism: 1
      activeDeadlineSeconds: 30
      backoffLimit: 6
      manualSelector: true
      selector:
        matchLabels:
          app: counter-pod
        matchExpressions: 规则
          - {key: app, operator: In, values: [counter-pod]}
      template:
        metadata:
          labels:
            app: counter-pod
        spec:
          restartPolicy: Never
          containers:
            - name: counter
              image: busybox:1.30
              command: ["bin/sh", "-c", "for i in 9 8 7 6 5 4 3 2 1; do echo $i;sleep
20;done"]

```

需要重点解释的几个选项:

schedule: cron表达式,用于指定任务的执行时间

```

*/1 * * * *
<分钟> <小时> <日> <月份> <星期>

```

分钟 值从 0 到 59.

小时 值从 0 到 23.

日 值从 1 到 31.

月 值从 1 到 12.

星期 值从 0 到 6, 0 代表星期日

多个时间可以用逗号隔开; 范围可以用连字符给出; *可以作为通配符; /表示每...

concurrencyPolicy:

Allow: 允许Jobs并发运行(默认)

Forbid: 禁止并发运行,如果上一次运行尚未完成,则跳过下一次运行

Replace: 替换,取消当前正在运行的作业并用新作业替换它

创建pc-cronjob.yaml,内容如下:

```

apiVersion: batch/v1beta1

```

```

kind: CronJob
metadata:
  name: pc-cronjob
  namespace: dev
  labels:
    controller: cronjob
spec:
  schedule: "*/1 * * * *"
  jobTemplate:
    metadata:
    spec:
      template:
        spec:
          restartPolicy: Never
          containers:
            - name: counter
              image: busybox:1.30
              command: ["bin/sh", "-c", "for i in 9 8 7 6 5 4 3 2 1; do echo $i;sleep
3;done"]

```

创建cronjob

```

[root@k8s-master01 ~]# kubectl create -f pc-cronjob.yaml
cronjob.batch/pc-cronjob created

```

查看cronjob

```

[root@k8s-master01 ~]# kubectl get cronjobs -n dev

```

NAME	SCHEDULE	SUSPEND	ACTIVE	LAST SCHEDULE	AGE
pc-cronjob	*/1 * * * *	False	0	<none>	6s

查看job

```

[root@k8s-master01 ~]# kubectl get jobs -n dev

```

NAME	COMPLETIONS	DURATION	AGE
pc-cronjob-1592587800	1/1	28s	3m26s
pc-cronjob-1592587860	1/1	28s	2m26s
pc-cronjob-1592587920	1/1	28s	86s

查看pod

```

[root@k8s-master01 ~]# kubectl get pods -n dev

```

NAME	COMPLETIONS	STATUS	AGE
pc-cronjob-1592587800-x4tsm	0/1	Completed	0
pc-cronjob-1592587860-r5gv4	0/1	Completed	0
pc-cronjob-1592587920-9dxxq	1/1	Running	0

删除cronjob

```

[root@k8s-master01 ~]# kubectl delete -f pc-cronjob.yaml
cronjob.batch "pc-cronjob" deleted

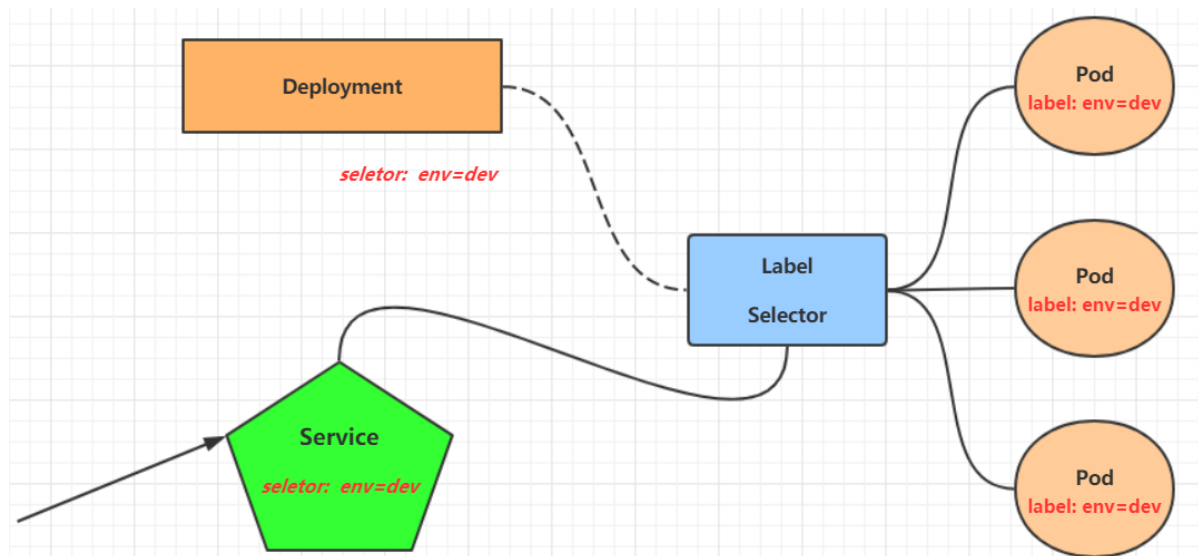
```

7. Service详解

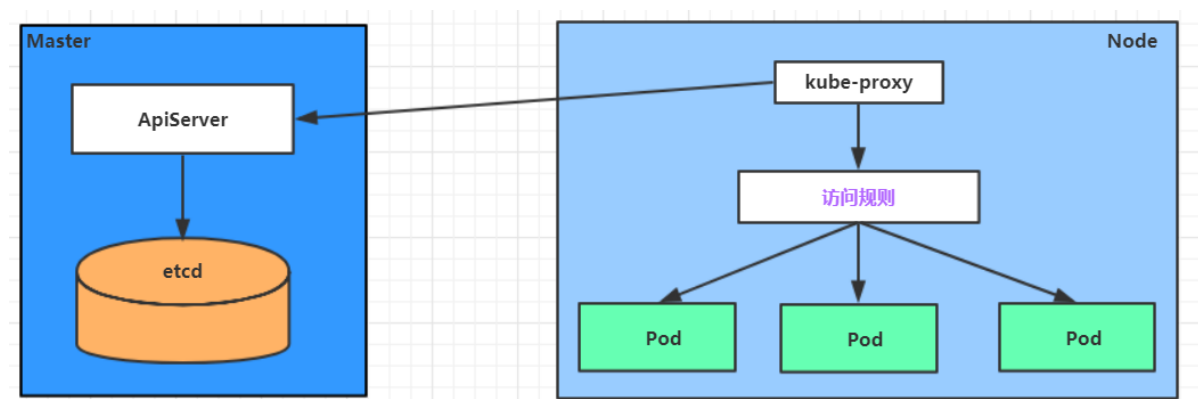
7.1 Service介绍

在kubernetes中，pod是应用程序的载体，我们可以通过pod的ip来访问应用程序，但是pod的ip地址不是固定的，这也就意味着不方便直接采用pod的ip对服务进行访问。

为了解决这个问题，kubernetes提供了Service资源，Service会对提供同一个服务的多个pod进行聚合，并且提供一个统一的入口地址。通过访问Service的入口地址就能访问到后面的pod服务。



Service在很多情况下只是一个概念，真正起作用的其实是kube-proxy服务进程，每个Node节点上都运行着一个kube-proxy服务进程。当创建Service的时候会通过api-server向etcd写入创建的service的信息，而kube-proxy会基于监听的机制发现这种Service的变动，然后**它会将最新的Service信息转换成对应的访问规则**。

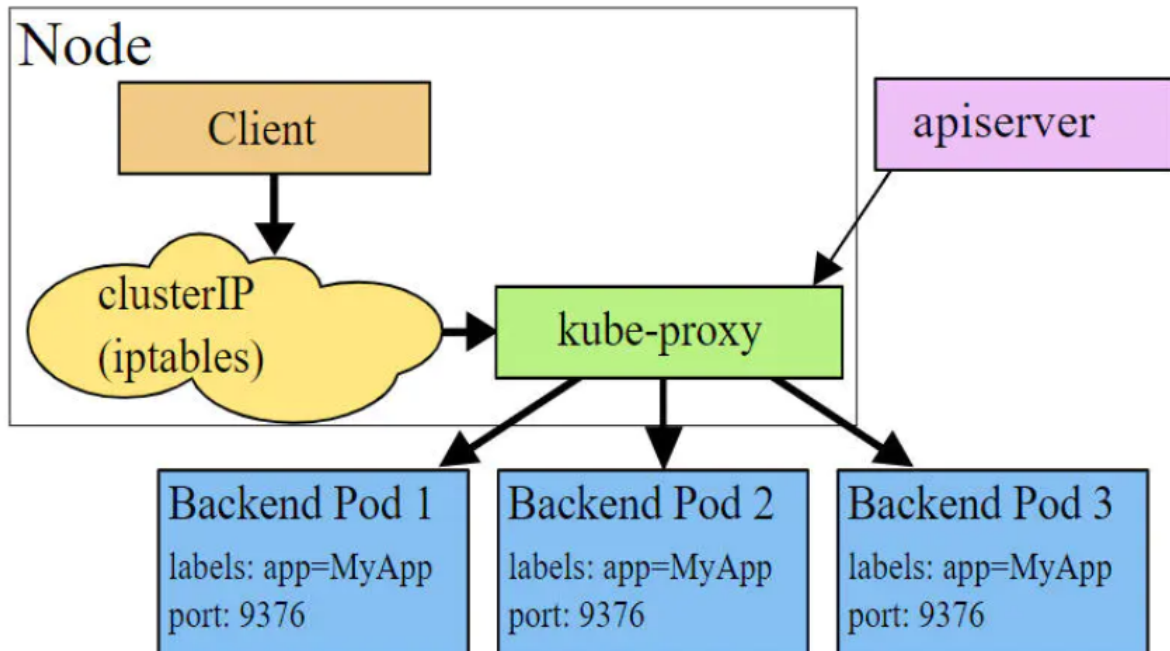


```
# 10.97.97.97:80 是service提供的访问入口
# 当访问这个入口的时候，可以发现后面有三个pod的服务在等待调用，
# kube-proxy会基于rr（轮询）的策略，将请求分发到其中一个pod上去
# 这个规则会同时在集群内的所有节点上都生成，所以在任何一个节点上访问都可以。
[root@node1 ~]# ipvsadm -Ln
IP Virtual Server version 1.2.1 (size=4096)
Prot LocalAddress:Port Scheduler Flags
  -> RemoteAddress:Port           Forward Weight ActiveConn InActConn
TCP  10.97.97.97:80 rr
  -> 10.244.1.39:80               Masq    1      0      0
  -> 10.244.1.40:80               Masq    1      0      0
  -> 10.244.2.33:80               Masq    1      0      0
```

kube-proxy目前支持三种工作模式:

7.1.1 userspace 模式

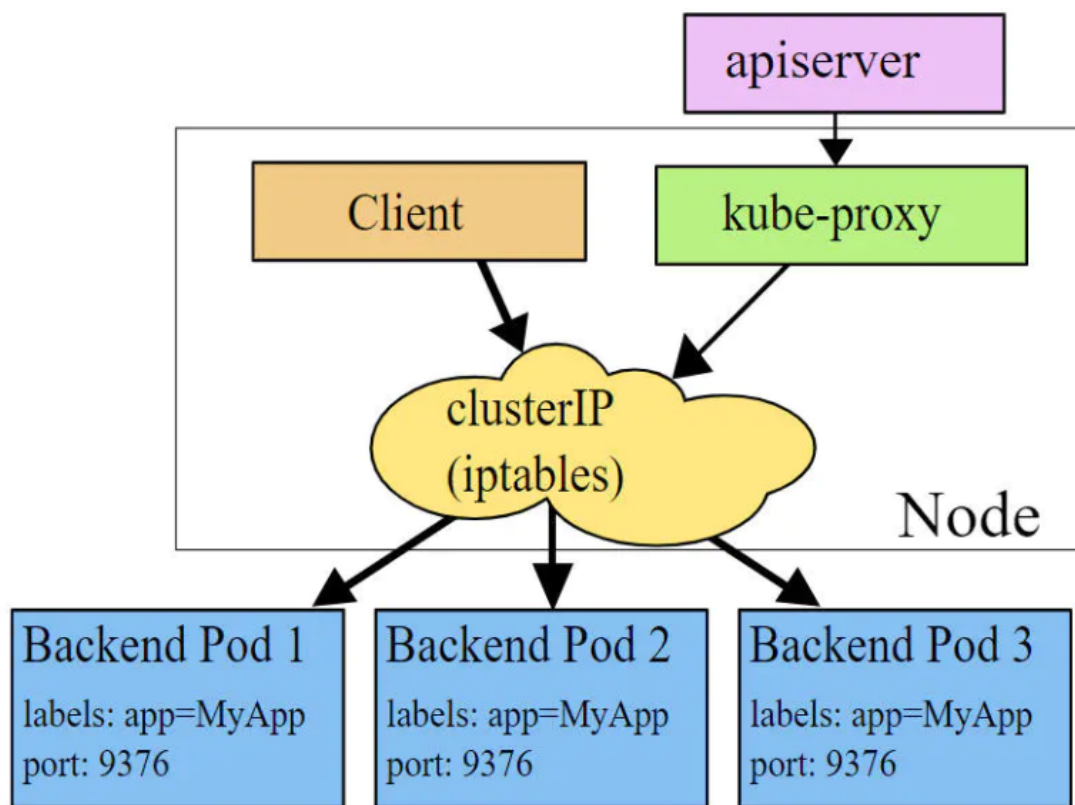
userspace模式下, kube-proxy会为每一个Service创建一个监听端口, 发向Cluster IP的请求被Iptables规则重定向到kube-proxy监听的端口上, kube-proxy根据LB算法选择一个提供服务的Pod并和其建立连接, 以将请求转发到Pod上。该模式下, kube-proxy充当了一个四层负责均衡器的角色。由于kube-proxy运行在userspace中, 在进行转发处理时会增加内核和用户空间之间的数据拷贝, 虽然比较稳定, 但是效率比较低。



Kube-proxy userspace模式

7.1.2 iptables 模式

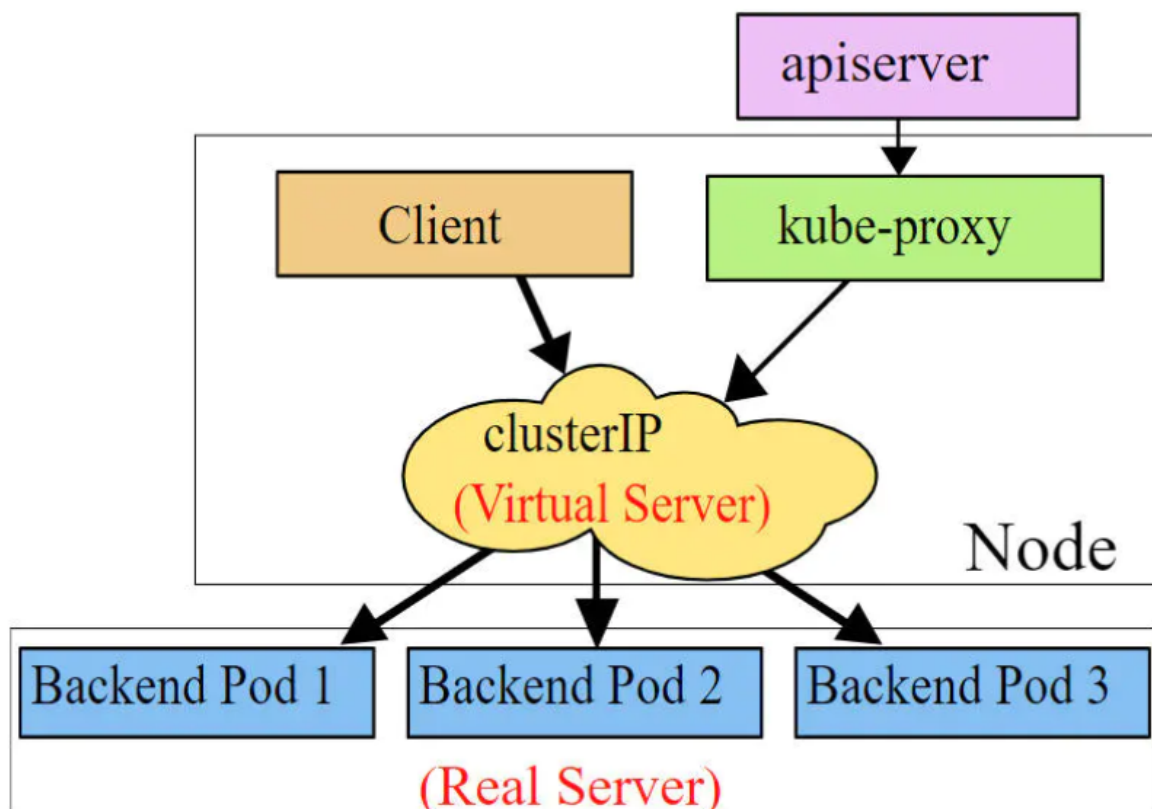
iptables模式下, kube-proxy为service后端的每个Pod创建对应的iptables规则, 直接将发向Cluster IP的请求重定向到一个Pod IP。该模式下kube-proxy不承担四层负责均衡器的角色, 只负责创建iptables规则。该模式的优点是较userspace模式效率更高, 但不能提供灵活的LB策略, 当后端Pod不可用时也无法进行重试。



Kube-proxy iptables模式

7.1.3 ipvs 模式

ipvs模式和iptables类似，kube-proxy监控Pod的变化并创建相应的ipvs规则。ipvs相对iptables转发效率更高。除此以外，ipvs支持更多的LB算法。



Kube-proxy ipvs模式

```
# 此模式必须安装ipvs内核模块，否则会降级为iptables
# 开启ipvs
[root@k8s-master01 ~]# kubectl edit cm kube-proxy -n kube-system
# 修改mode: "ipvs"
[root@k8s-master01 ~]# kubectl delete pod -l k8s-app=kube-proxy -n kube-system
[root@node1 ~]# ipvsadm -Ln
IP Virtual Server version 1.2.1 (size=4096)
Prot LocalAddress:Port Scheduler Flags
  -> RemoteAddress:Port           Forward Weight ActiveConn InActConn
TCP  10.97.97.97:80 rr
  -> 10.244.1.39:80                Masq    1      0          0
  -> 10.244.1.40:80                Masq    1      0          0
  -> 10.244.2.33:80                Masq    1      0          0
```

7.2 Service类型

Service的资源清单文件:

```
kind: Service # 资源类型
apiVersion: v1 # 资源版本
metadata: # 元数据
  name: service # 资源名称
  namespace: dev # 命名空间
spec: # 描述
  selector: # 标签选择器，用于确定当前service代理哪些pod
    app: nginx
  type: # Service类型，指定service的访问方式
  clusterIP: # 虚拟服务的ip地址
  sessionAffinity: # session亲和性，支持ClientIP、None两个选项
  ports: # 端口信息
    - protocol: TCP
      port: 3017 # service端口
      targetPort: 5003 # pod端口
      nodePort: 31122 # 主机端口
```

- ClusterIP: 默认值，它是Kubernetes系统自动分配的虚拟IP，只能在集群内部访问
- NodePort: 将Service通过指定的Node上的端口暴露给外部，通过此方法，就可以在集群外部访问服务
- LoadBalancer: 使用外接负载均衡器完成到服务的负载分发，注意此模式需要外部云环境支持
- ExternalName: 把集群外部的服务引入集群内部，直接使用

7.3 Service使用

7.3.1 实验环境准备

在使用service之前，首先利用Deployment创建出3个pod，注意要为pod设置 app=nginx-pod 的标签
创建deployment.yaml，内容如下:

```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: pc-deployment
```

```

namespace: dev
spec:
  replicas: 3
  selector:
    matchLabels:
      app: nginx-pod
  template:
    metadata:
      labels:
        app: nginx-pod
    spec:
      containers:
        - name: nginx
          image: nginx:1.17.1
          ports:
            - containerPort: 80

```

```

[root@k8s-master01 ~]# kubectl create -f deployment.yaml
deployment.apps/pc-deployment created

```

查看pod详情

```

[root@k8s-master01 ~]# kubectl get pods -n dev -o wide --show-labels

```

NAME	READY	STATUS	IP	NODE	LABELS
pc-deployment-66cb59b984-8p84h	1/1	Running	10.244.1.39	node1	app=nginx-pod
pc-deployment-66cb59b984-vx8vx	1/1	Running	10.244.2.33	node2	app=nginx-pod
pc-deployment-66cb59b984-wnncx	1/1	Running	10.244.1.40	node1	app=nginx-pod

为了方便后面的测试，修改下三台nginx的index.html页面（三台修改的IP地址不一致）

```

# kubectl exec -it pc-deployment-66cb59b984-8p84h -n dev /bin/sh

```

```

# echo "10.244.1.39" > /usr/share/nginx/html/index.html

```

#修改完毕之后，访问测试

```

[root@k8s-master01 ~]# curl 10.244.1.39

```

```

10.244.1.39

```

```

[root@k8s-master01 ~]# curl 10.244.2.33

```

```

10.244.2.33

```

```

[root@k8s-master01 ~]# curl 10.244.1.40

```

```

10.244.1.40

```

7.3.2 ClusterIP类型的Service

创建service-clusterip.yaml文件

```

apiVersion: v1
kind: Service
metadata:
  name: service-clusterip
  namespace: dev
spec:
  #sessionAffinity: ClientIP #每个kehud每次访问都是固定的属于自己的pod
  selector:
    app: nginx-pod

```

```
clusterIP: 10.97.97.97 # service的ip地址, 如果不写, 默认会生成一个
type: ClusterIP
ports:
- port: 80 # Service端口
  targetPort: 80 # pod端口
```

```
# 创建service
[root@k8s-master01 ~]# kubectl create -f service-clusterip.yaml
service/service-clusterip created

# 查看service
[root@k8s-master01 ~]# kubectl get svc -n dev -o wide
NAME                                TYPE             CLUSTER-IP      EXTERNAL-IP      PORT(S)          AGE
SELECTOR
service-clusterip                  ClusterIP         10.97.97.97      <none>           80/TCP           13s
app=nginx-pod

# 查看service的详细信息
# 在这里有一个Endpoints列表, 里面就是当前service可以负载到的服务入口
[root@k8s-master01 ~]# kubectl describe svc service-clusterip -n dev
Name:                               service-clusterip
Namespace:                           dev
Labels:                               <none>
Annotations:                           <none>
Selector:                             app=nginx-pod
Type:                                  ClusterIP
IP:                                    10.97.97.97
Port:                                  <unset> 80/TCP
TargetPort:                           80/TCP
Endpoints:                             10.244.1.39:80,10.244.1.40:80,10.244.2.33:80
Session Affinity:                     None
Events:                               <none>

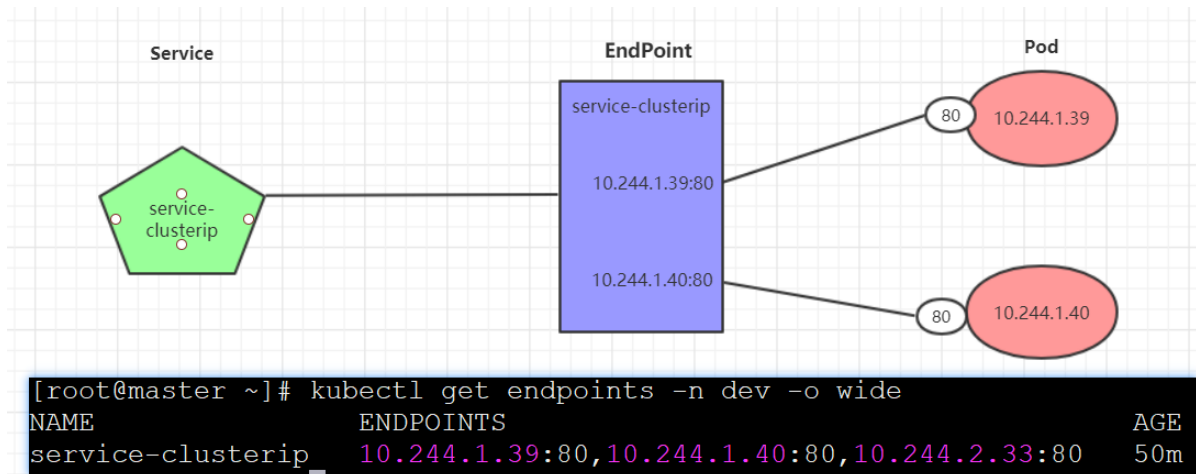
# 查看ipvs的映射规则
[root@k8s-master01 ~]# ipvsadm -Ln
TCP 10.97.97.97:80 rr
-> 10.244.1.39:80           Masq    1      0      0
-> 10.244.1.40:80          Masq    1      0      0
-> 10.244.2.33:80          Masq    1      0      0

# 访问10.97.97.97:80观察效果
[root@k8s-master01 ~]# curl 10.97.97.97:80
10.244.2.33
```

7.3.3 Endpoint

Endpoint是kubernetes中的一个资源对象, 存储在etcd中, 用来记录一个service对应的所有pod的访问地址, 它是根据service配置文件中selector描述产生的。

一个Service由一组Pod组成, 这些Pod通过Endpoints暴露出来, **Endpoints是实现实际服务的端点集合**。换句话说, service和pod之间的联系是通过endpoints实现的。



负载分发策略

对Service的访问被分发到了后端的Pod上去，目前kubernetes提供了两种负载分发策略：

- 如果不定义，默认使用kube-proxy的策略，比如随机、轮询
- 基于客户端地址的会话保持模式，即来自同一个客户端发起的所有请求都会转发到固定的一个Pod上

此模式可以在spec中添加 `sessionAffinity: ClientIP` 选项

查看ipvs的映射规则【rr 轮询】

```
[root@k8s-master01 ~]# ipvsadm -Ln
```

```

TCP 10.97.97.97:80 rr
-> 10.244.1.39:80      Masq    1      0      0
-> 10.244.1.40:80      Masq    1      0      0
-> 10.244.2.33:80      Masq    1      0      0

```

循环访问测试

```

[root@k8s-master01 ~]# while true;do curl 10.97.97.97:80; sleep 5; done;
10.244.1.40
10.244.1.39
10.244.2.33
10.244.1.40
10.244.1.39
10.244.2.33

```

修改分发策略-----sessionAffinity:ClientIP

查看ipvs规则【persistent 代表持久】

```
[root@k8s-master01 ~]# ipvsadm -Ln
```

```

TCP 10.97.97.97:80 rr persistent 10800
-> 10.244.1.39:80      Masq    1      0      0
-> 10.244.1.40:80      Masq    1      0      0
-> 10.244.2.33:80      Masq    1      0      0

```

循环访问测试

```

[root@k8s-master01 ~]# while true;do curl 10.97.97.97; sleep 5; done;
10.244.2.33
10.244.2.33
10.244.2.33

```

删除service

```
[root@k8s-master01 ~]# kubectl delete -f service-clusterip.yaml
```

```
service "service-clusterip" deleted
```

7.3.4 Headliness类型的Service

在某些场景中，开发人员可能不想使用Service提供的负载均衡功能，而希望自己来控制负载均衡策略，针对这种情况，kubernetes提供了Headliness Service，这类Service不会分配Cluster IP，如果想要访问service，只能通过service的域名进行查询。

创建service-headliness.yaml

```
apiVersion: v1
kind: Service
metadata:
  name: service-headliness
  namespace: dev
spec:
  selector:
    app: nginx-pod
  clusterIP: None # 将clusterIP设置为None，即可创建headliness Service
  type: ClusterIP
  ports:
    - port: 80
      targetPort: 80
```

创建service

```
[root@k8s-master01 ~]# kubectl create -f service-headliness.yaml
service/service-headliness created
```

获取service，发现CLUSTER-IP未分配

```
[root@k8s-master01 ~]# kubectl get svc service-headliness -n dev -o wide
```

NAME	TYPE	CLUSTER-IP	EXTERNAL-IP	PORT(S)	AGE
service-headliness	ClusterIP	None	<none>	80/TCP	11s

app=nginx-pod

查看service详情

```
[root@k8s-master01 ~]# kubectl describe svc service-headliness -n dev
```

Name: service-headliness
Namespace: dev
Labels: <none>
Annotations: <none>
Selector: app=nginx-pod
Type: ClusterIP
IP: None
Port: <unset> 80/TCP
TargetPort: 80/TCP
Endpoints: 10.244.1.39:80,10.244.1.40:80,10.244.2.33:80
Session Affinity: None
Events: <none>

查看域名的解析情况

```
[root@k8s-master01 ~]# kubectl exec -it pc-deployment-66cb59b984-8p84h -n dev /bin/bash
/ # cat /etc/resolv.conf
nameserver 10.96.0.10
```

```
search dev.svc.cluster.local svc.cluster.local cluster.local
```

```
[root@k8s-master01 ~]# dig @10.96.0.10 service-headliness.dev.svc.cluster.local
service-headliness.dev.svc.cluster.local. 30 IN A 10.244.1.40
service-headliness.dev.svc.cluster.local. 30 IN A 10.244.1.39
service-headliness.dev.svc.cluster.local. 30 IN A 10.244.2.33
```

```
# dig @10.96.0.10 service-headliness.dev.svc.cluster.local 超时间问题
```

```
[root@server1 ~]# dig -t A myapp-svc.default.svc.cluster.local. @10.96.0.1
```

```
; <<>> DiG 9.9.4-RedHat-9.9.4-37.el7 <<>> -t A myapp-
svc.default.svc.cluster.local. @10.96.0.1
;; global options: +cmd
;; connection timed out; no servers could be reached
```

```
# 域名解析出错，查看 coredns组件
```

```
# kubectl get pod -n kube-system
```

```
coredns-9d85f5447-256lb          0/1      Running   0          2m2s
coredns-9d85f5447-79t6q          0/1      Running   0          91s
```

```
# kubectl describe pod coredns-9d85f5447-256lb -n kube-system
```

```
shabox异常, network open /run/flannel/subnet.env
```

```
#经过查找, /run/flannel/subnet.env 是存在的, 使用 ls -l查看文件权限
```

```
[root@master flannel]# ls -l
```

```
总用量 4
```

```
-rw-r--r-- 1 root root 96 4月 12 14:21 subnet.env
```

```
# 文件权限没问题, 考虑删除 coredns, 重启试试
```

```
[root@master ~]# kubectl get pod -n kube-system
```

```
NAME                                READY   STATUS    RESTARTS   AGE
coredns-9d85f5447-ndj5n             0/1     Running   3          9d
coredns-9d85f5447-r4kpx             0/1     Running   3          9d
...
```

```
[root@master ~]# kubectl delete pod coredns-9d85f5447-ndj5n -n kube-system
pod "coredns-9d85f5447-ndj5n" deleted
```

```
[root@master ~]# kubectl delete pod coredns-9d85f5447-r4kpx -n kube-system
pod "coredns-9d85f5447-r4kpx" deleted
```

```
[root@master ~]# kubectl get pod -n kube-system
```

```
NAME                                READY   STATUS    RESTARTS   AGE
coredns-9d85f5447-256lb             1/1     Running   0          59s
coredns-9d85f5447-79t6q             1/1     Running   0          28s
```

```
#重启完毕, 发现 coredns启动成功
```

```
[root@master ~]# dig @10.96.0.10 service-headliness.dev.svc.cluster.local
```

```
; <<>> DiG 9.11.4-P2-RedHat-9.11.4-9.P2.el7 <<>> @10.96.0.10 service-
headliness.dev.svc.cluster.local
; (1 server found)
;; global options: +cmd
;; Got answer:
;; WARNING: .local is reserved for Multicast DNS
;; You are currently testing what happens when an mDNS query is leaked to DNS
;; ->>HEADER<<- opcode: QUERY, status: NOERROR, id: 60063
;; flags: qr aa rd; QUERY: 1, ANSWER: 3, AUTHORITY: 0, ADDITIONAL: 1
;; WARNING: recursion requested but not available
```

```
;; OPT PSEUDOSECTION:
; EDNS: version: 0, flags:; udp: 4096
;; QUESTION SECTION:
;service-headliness.dev.svc.cluster.local. IN A

;; ANSWER SECTION:
service-headliness.dev.svc.cluster.local. 30 IN A 10.244.1.10
service-headliness.dev.svc.cluster.local. 30 IN A 10.244.2.9
service-headliness.dev.svc.cluster.local. 30 IN A 10.244.1.9

;; Query time: 0 msec
;; SERVER: 10.96.0.10#53(10.96.0.10)
;; WHEN: 二 4月 12 14:39:04 CST 2022
;; MSG SIZE rcvd: 237
```

#至此，dns解析成功

在master上配置dns，即使生效，重启eth33，会重写

```
[root@master ~]# vi /etc/resolv.conf
```

```
# Generated by NetworkManager
```

```
#nameserver 223.5.5.5
```

```
nameserver 10.96.0.10
```

#如何通过域名访问

```
[root@master ~]# curl service-headliness.dev.svc.cluster.local
```

```
10.244.2.9:
```

```
[root@master ~]# curl service-headliness.dev.svc.cluster.local
```

```
10.244.2.9:
```

```
[root@master ~]# curl service-headliness.dev.svc.cluster.local
```

```
10.244.2.9:
```

```
[root@master ~]# curl service-headliness.dev.svc.cluster.local
```

```
10.244.1.10:
```

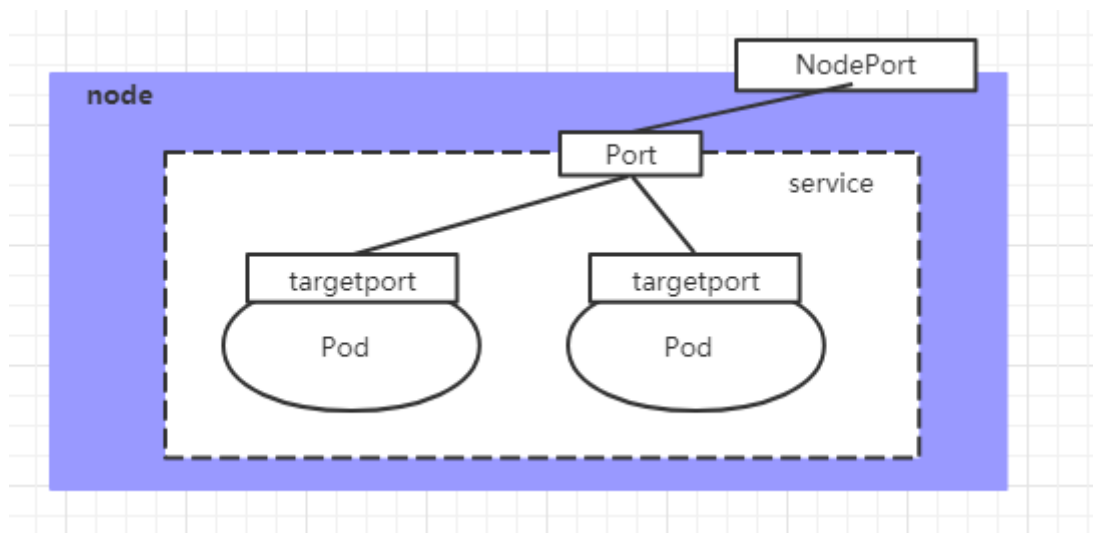
```
[root@master ~]# curl service-headliness.dev.svc.cluster.local
```

```
10.244.2.9:
```

#ps: 测试完，记得把上面的dns改回来，不改也没事，重启的时候，eth33会重写 resolv.conf文件

7.3.5 NodePort类型的Service

在之前的样例中，创建的Service的ip地址只有集群内部才可以访问，如果希望将Service暴露给集群外部使用，那么就要使用到另外一种类型的Service，称为NodePort类型。NodePort的工作原理其实就是将service的端口映射到Node的一个端口上，然后就可以通过 NodeIp:NodePort 来访问service了。



创建service-nodeport.yaml

```
apiVersion: v1
kind: Service
metadata:
  name: service-nodeport
  namespace: dev
spec:
  selector:
    app: nginx-pod
  type: NodePort # service类型
  ports:
    - port: 80
      nodePort: 30002 # 指定绑定的node的端口(默认的取值范围是: 30000-32767), 如果不指定, 会默认分配
      targetPort: 80
```

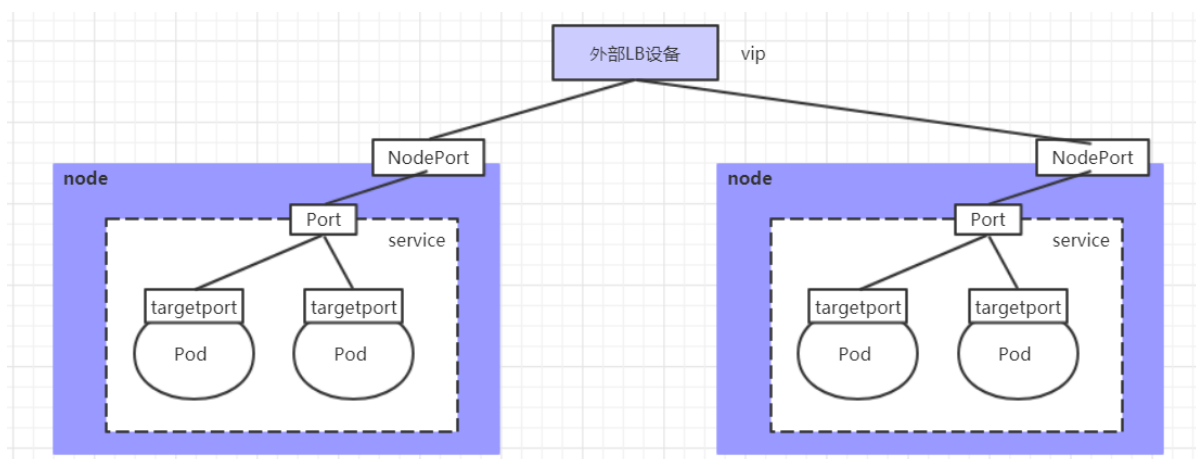
```
# 创建service
[root@k8s-master01 ~]# kubectl create -f service-nodeport.yaml
service/service-nodeport created

# 查看service
[root@k8s-master01 ~]# kubectl get svc -n dev -o wide
NAME                                TYPE        CLUSTER-IP    EXTERNAL-IP    PORT(S)
SELECTOR
service-nodeport                    NodePort    10.105.64.191 <none>         80:30002/TCP
app=nginx-pod

# 接下来可以通过电脑主机的浏览器去访问集群中任意一个nodeip的30002端口, 即可访问到pod
```

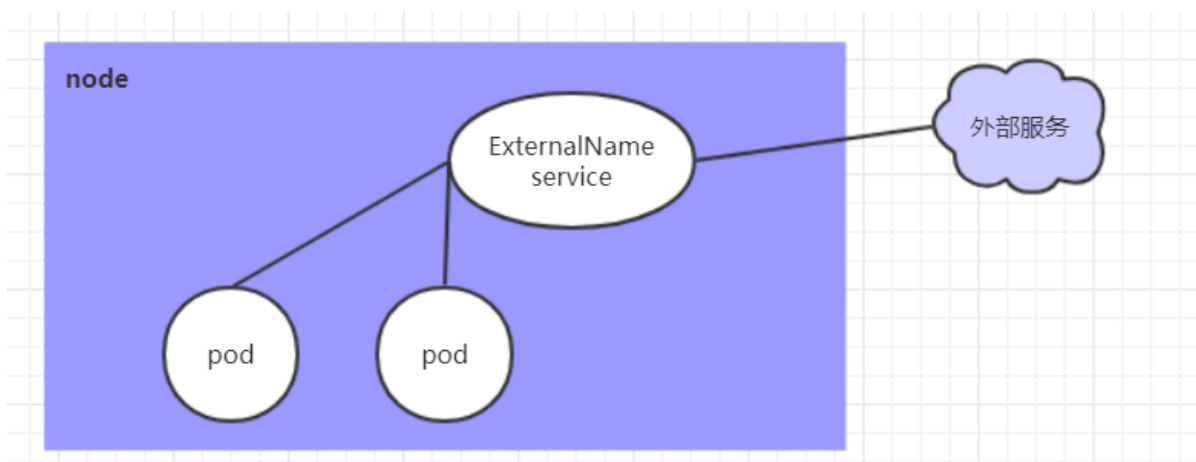
7.3.6 LoadBalancer类型的Service

LoadBalancer和NodePort很相似, 目的都是向外部暴露一个端口, 区别在于LoadBalancer会在集群的外部再来做一个负载均衡设备, 而这个设备需要外部环境支持的, 外部服务发送到这个设备上的请求, 会被设备负载之后转发到集群中。



7.3.7 ExternalName类型的Service

ExternalName类型的Service用于引入集群外部的服务，它通过 `externalName` 属性指定外部一个服务的地址，然后在集群内部访问此service就可以访问到外部的服务了。



```
apiVersion: v1
kind: Service
metadata:
  name: service-externalname
  namespace: dev
spec:
  type: ExternalName # service类型
  externalName: www.baidu.com #改成ip地址也可以
```

创建service

```
[root@k8s-master01 ~]# kubectl create -f service-externalname.yaml
service/service-externalname created
```

域名解析

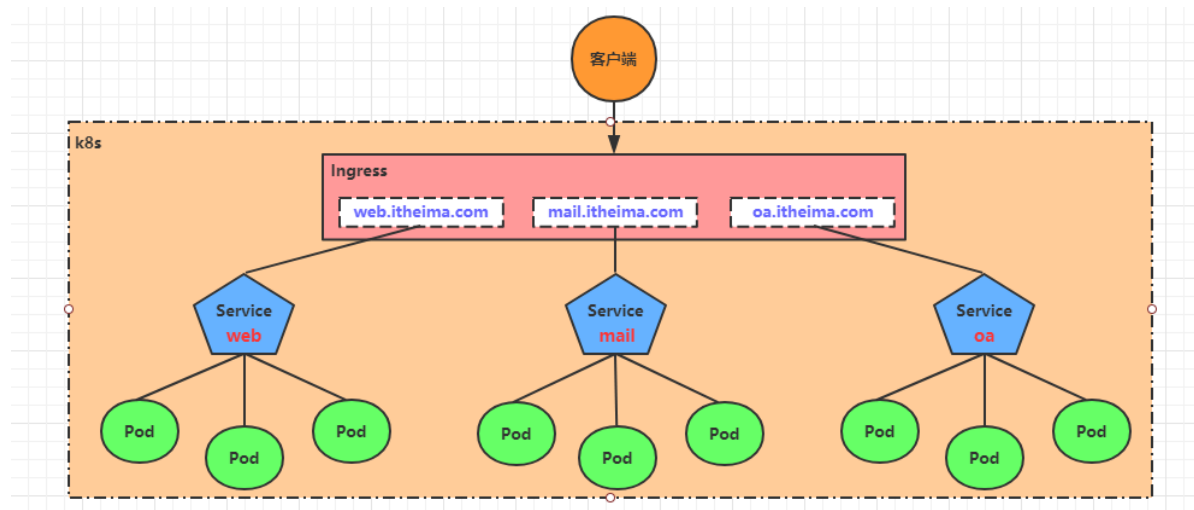
```
[root@k8s-master01 ~]# dig @10.96.0.10 service-externalname.dev.svc.cluster.local
service-externalname.dev.svc.cluster.local. 30 IN CNAME www.baidu.com.
www.baidu.com. 30 IN CNAME www.a.shifen.com.
www.a.shifen.com. 30 IN A 39.156.66.18
www.a.shifen.com. 30 IN A 39.156.66.14
```

7.4 Ingress介绍

在前面课程中已经提到，Service对集群之外暴露服务的主要方式有两种：NodePort和LoadBalancer，但是这两种方式，都有一定的缺点：

- NodePort方式的缺点是会占用很多集群机器的端口，那么当集群服务变多的时候，这个缺点就愈发明显
- LB方式的缺点是每个service需要一个LB，浪费、麻烦，并且需要kubernetes之外设备的支持

基于这种现状，kubernetes提供了Ingress资源对象，Ingress只需要一个NodePort或者一个LB就可以满足暴露多个Service的需求。工作机制大致如下图所示：

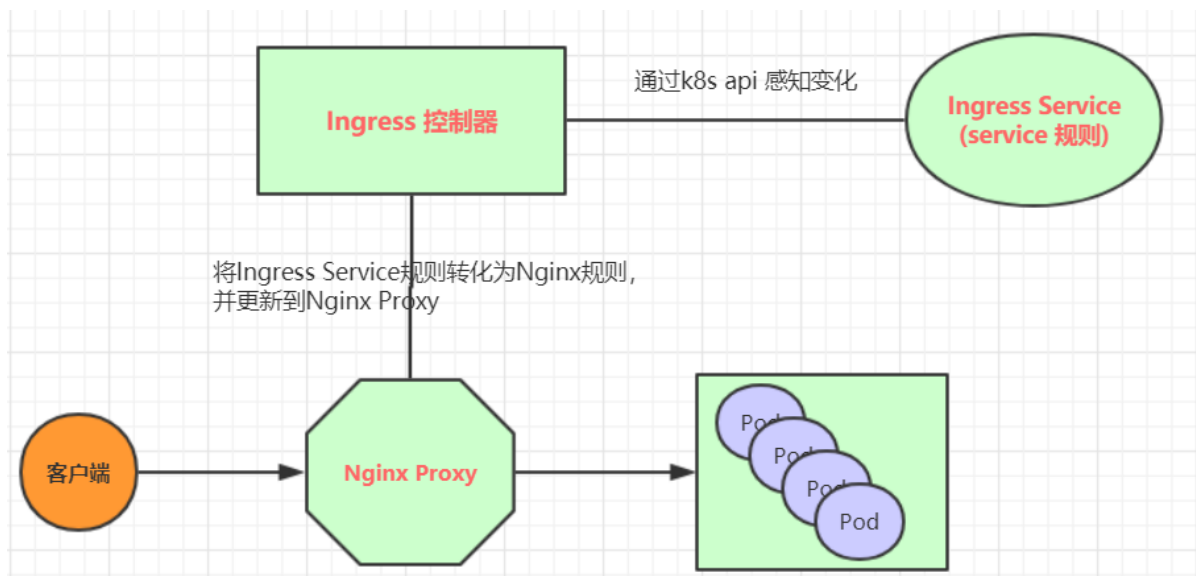


实际上，Ingress相当于一个7层的负载均衡器，是kubernetes对反向代理的一个抽象，它的工作原理类似于Nginx，可以理解成在Ingress里建立诸多映射规则，Ingress Controller通过监听这些配置规则并转化成Nginx的反向代理配置，然后对外部提供服务。在这里有两个核心概念：

- ingress：kubernetes中的一个对象，作用是定义请求如何转发到service的规则
- ingress controller：具体实现反向代理及负载均衡的程序，对ingress定义的规则进行解析，根据配置的规则来实现请求转发，实现方式有很多，比如Nginx, Contour, Haproxy等等

Ingress（以Nginx为例）的工作原理如下：

1. 用户编写Ingress规则，说明哪个域名对应kubernetes集群中的哪个Service
2. Ingress控制器动态感知Ingress服务规则的变化，然后生成一段对应的Nginx反向代理配置
3. Ingress控制器会将生成的Nginx配置写入到一个运行着的Nginx服务中，并动态更新
4. 到此为止，其实真正在工作的就是一个Nginx了，内部配置了用户定义请求转发规则



7.5 Ingress使用

7.5.1 环境准备 搭建ingress环境

```
# 创建文件夹
[root@k8s-master01 ~]# mkdir ingress-controller
[root@k8s-master01 ~]# cd ingress-controller/

# 获取ingress-nginx, 本次案例使用的是0.30版本
[root@k8s-master01 ingress-controller]# wget
https://raw.githubusercontent.com/kubernetes/ingress-nginx/nginx-
0.30.0/deploy/static/mandatory.yaml
[root@k8s-master01 ingress-controller]# wget
https://raw.githubusercontent.com/kubernetes/ingress-nginx/nginx-
0.30.0/deploy/static/provider/baremetal/service-nodeport.yaml

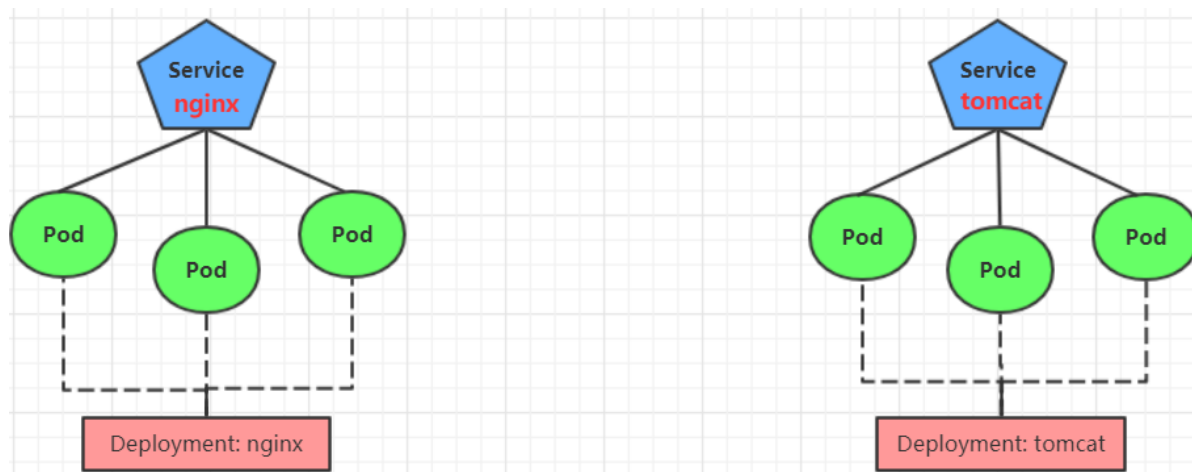
# 修改mandatory.yaml文件中的仓库
# 修改quay.io/kubernetes-ingress-controller/nginx-ingress-controller:0.30.0
# 为quay-mirror.qiniu.com/kubernetes-ingress-controller/nginx-ingress-
controller:0.30.0
# 创建ingress-nginx
[root@k8s-master01 ingress-controller]# kubectl apply -f ./

# 查看ingress-nginx
[root@k8s-master01 ingress-controller]# kubectl get pod -n ingress-nginx
NAME                                READY   STATUS    RESTARTS   AGE
pod/nginx-ingress-controller-fbf967dd5-4qbbp   1/1     Running   0          12h

# 查看service
[root@master ~]# kubectl get svc -n ingress-nginx
NAME                TYPE        CLUSTER-IP   EXTERNAL-IP   PORT(S)
AGE
ingress-nginx       NodePort    10.99.13.74   <none>        80:31773/TCP,443:30476/TCP
9m21s
```


7.5.2 准备service和pod

为了后面的实验比较方便，创建如下图所示的模型



创建tomcat-nginx.yaml

```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: nginx-deployment
  namespace: dev
spec:
  replicas: 3
  selector:
    matchLabels:
      app: nginx-pod
  template:
    metadata:
      labels:
        app: nginx-pod
    spec:
      containers:
        - name: nginx
          image: nginx:1.17.1
          ports:
            - containerPort: 80
```

```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: tomcat-deployment
  namespace: dev
spec:
  replicas: 3
  selector:
    matchLabels:
      app: tomcat-pod
  template:
    metadata:
      labels:
        app: tomcat-pod
```

```

spec:
  containers:
  - name: tomcat
    image: tomcat:8.5-jre10-slim
    ports:
    - containerPort: 8080

```

```

apiVersion: v1
kind: Service
metadata:
  name: nginx-service
  namespace: dev
spec:
  selector:
    app: nginx-pod
  clusterIP: None
  type: ClusterIP
  ports:
  - port: 80
    targetPort: 80

```

```

apiVersion: v1
kind: Service
metadata:
  name: tomcat-service
  namespace: dev
spec:
  selector:
    app: tomcat-pod
  clusterIP: None
  type: ClusterIP
  ports:
  - port: 8080
    targetPort: 8080

```

创建

```
[root@k8s-master01 ~]# kubectl create -f tomcat-nginx.yaml
```

查看

```
[root@k8s-master01 ~]# kubectl get svc -n dev
```

NAME	TYPE	CLUSTER-IP	EXTERNAL-IP	PORT(S)	AGE
nginx-service	ClusterIP	None	<none>	80/TCP	48s
tomcat-service	ClusterIP	None	<none>	8080/TCP	48s

7.5.3 Http代理

创建ingress-http.yaml

```

apiVersion: extensions/v1beta1
kind: Ingress
metadata:

```

```

name: ingress-http
namespace: dev
spec:
  rules:
  - host: nginx.jax.com
    http:
      paths:
      - path: /
        backend:
          serviceName: nginx-service
          servicePort: 80
  - host: tomcat.jax.com
    http:
      paths:
      - path: /
        backend:
          serviceName: tomcat-service
          servicePort: 8080

```

创建

```

[root@master ~]# vi ingress-http.yaml
[root@master ~]# kubectl create -f ingress-http.yaml
ingress.extensions/ingress-http created
[root@master ~]# kubectl get ing ingress-http -n dev

```

NAME	HOSTS	ADDRESS	PORTS	AGE
ingress-http	nginx.jax.com,tomcat.jax.com	10.99.13.74	80	10s

查看详情

```

[root@k8s-master01 ~]# kubectl describe ing ingress-http -n dev

```

```

Name:          ingress-http
Namespace:     dev
Address:       10.99.13.74
Default backend: default-http-backend:80 (<none>)
Rules:
  Host          Path  Backends
  ----          -
  nginx.jax.com /    nginx-service:80
(10.244.1.18:80,10.244.1.20:80,10.244.1.21:80 + 3 more...)
  tomcat.jax.com /    tomcat-service:8080
(10.244.1.22:8080,10.244.1.23:8080,10.244.2.13:8080)

```

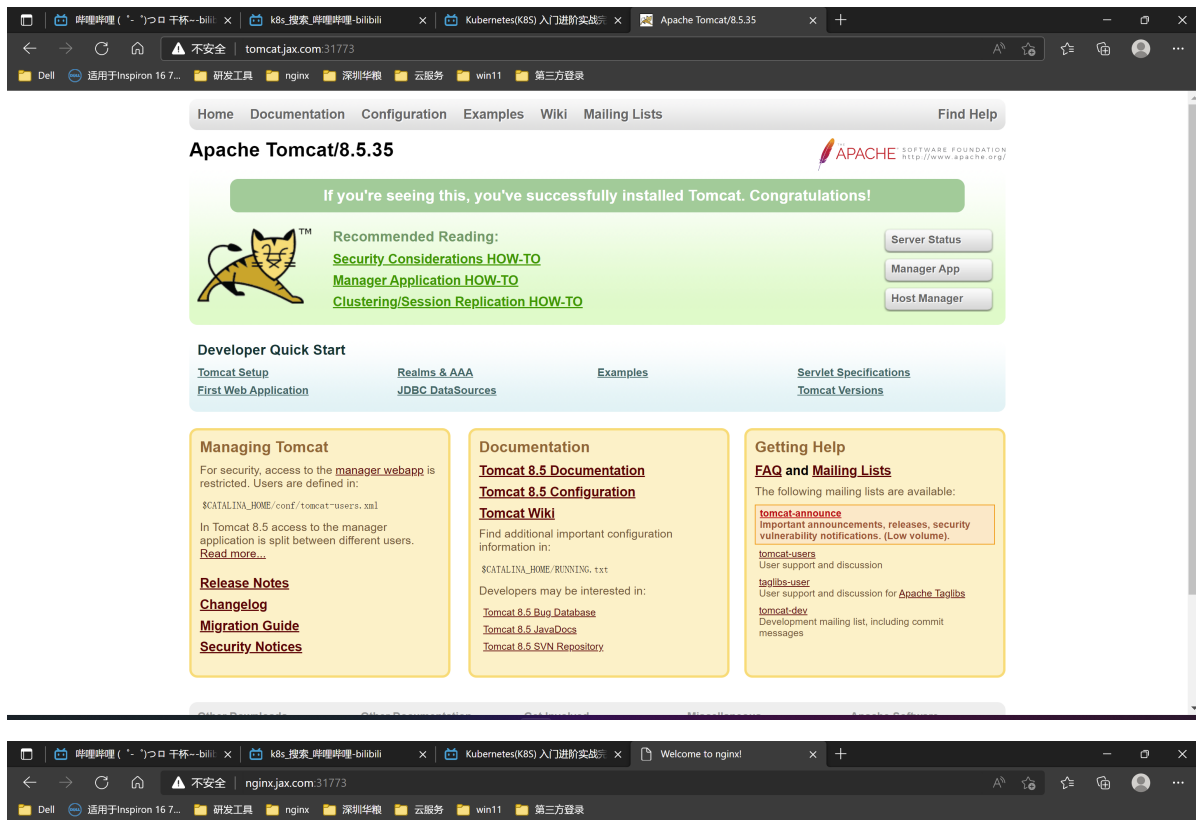
Annotations:

Events:

Type	Reason	Age	From	Message
Normal	CREATE	2m45s	nginx-ingress-controller	Ingress dev/ingress-http
Normal	UPDATE	2m41s	nginx-ingress-controller	Ingress dev/ingress-http

接下来,在本地电脑上配置host文件,解析上面的两个域名到192.168.5.100(master)上

然后,就可以分别访问tomcat.jax.com:31773 和 nginx.jax.com:31773 查看效果了



7.5.4 Https代理

创建证书

```
# 生成证书
openssl req -x509 -sha256 -nodes -days 365 -newkey rsa:2048 -keyout tls.key -out
tls.crt -subj "/C=CN/ST=BJ/L=BJ/O=nginx/CN=jax.com"

# 创建密钥
kubectl create secret tls tls-secret --key tls.key --cert tls.crt
```

创建ingress-https.yaml

```
apiVersion: extensions/v1beta1
kind: Ingress
metadata:
```

```

name: ingress-https
namespace: dev
spec:
  tls:
    - hosts:
        - nginx.jax.com
        - tomcat.jax.com
      secretName: tls-secret # 指定密钥
  rules:
    - host: nginx.jax.com
      http:
        paths:
          - path: /
            backend:
              serviceName: nginx-service
              servicePort: 80
    - host: tomcat.itjax.com
      http:
        paths:
          - path: /
            backend:
              serviceName: tomcat-service
              servicePort: 8080

```

创建

```

[root@k8s-master01 ~]# kubectl create -f ingress-https.yaml
ingress.extensions/ingress-https created

```

查看

```

[root@k8s-master01 ~]# kubectl get ing ingress-https -n dev
NAME                                HOSTS                                ADDRESS          PORTS          AGE
ingress-https    nginx.jax.com,tomcat.jax.com    10.104.184.38    80, 443        2m42s

```

查看详情

```

[root@k8s-master01 ~]# kubectl describe ing ingress-https -n dev
...

```

TLS:

tls-secret terminates nginx.jax.com,tomcat.jax.com

Rules:

Host	Path	Backends
nginx.jax.com	/	nginx-service:80
tomcat.jax.com	/	tomcat-service:8080

nginx.jax.com / nginx-service:80
(10.244.1.97:80,10.244.1.98:80,10.244.2.119:80)

tomcat.jax.com / tomcat-service:8080(10.244.1.99:8080,10.244.2.117:8080,10.244.2.120:8080)

...

下面可以通过浏览器访问<https://nginx.jax.com:31335> 和 <https://tomcat.jax.com:31335>来查看了

8. 数据存储

在前面已经提到，容器的生命周期可能很短，会被频繁地创建和销毁。那么容器在销毁时，保存在容器中的数据也会被清除。这种结果对用户来说，在某些情况下是不乐意看到的。为了持久化保存容器的数据，kubernetes引入了Volume的概念。

Volume是Pod中能够被多个容器访问的共享目录，它被定义在Pod上，然后被一个Pod里的多个容器挂载到具体的文件目录下，kubernetes通过Volume实现同一个Pod中不同容器之间的数据共享以及数据的持久化存储。Volume的生命周期不与Pod中单个容器的生命周期相关，当容器终止或者重启时，Volume中的数据也不会丢失。

kubernetes的Volume支持多种类型，比较常见的有下面几个：

- 简单存储：EmptyDir、HostPath、NFS
- 高级存储：PV、PVC
- 配置存储：ConfigMap、Secret

8.1 基本存储

8.1.1 EmptyDir

EmptyDir是最基础的Volume类型，一个EmptyDir就是Host上的一个空目录。

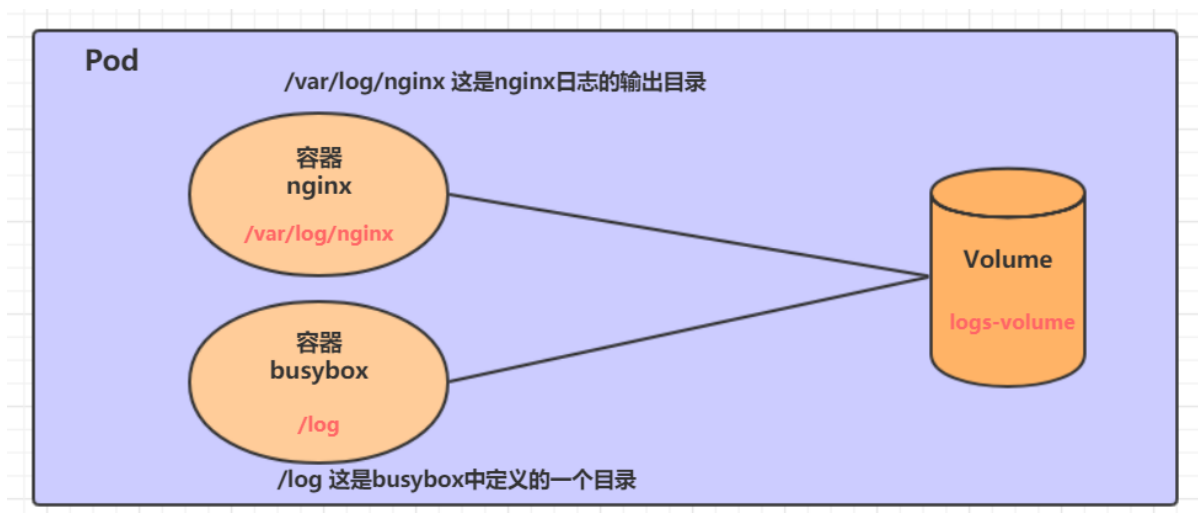
EmptyDir是在Pod被分配到Node时创建的，它的初始内容为空，并且无须指定宿主机上对应的目录文件，因为kubernetes会自动分配一个目录，当Pod销毁时，EmptyDir中的数据也会被永久删除。

EmptyDir用途如下：

- 临时空间，例如用于某些应用程序运行时所需的临时目录，且无须永久保留
- 一个容器需要从另一个容器中获取数据的目录（多容器共享目录）

接下来，通过一个容器之间文件共享的案例来使用一下EmptyDir。

在一个Pod中准备两个容器nginx和busybox，然后声明一个Volume分别挂在到两个容器的目录中，然后nginx容器负责向Volume中写日志，busybox中通过命令将日志内容读到控制台。



创建一个volume-emptydir.yaml

```
apiVersion: v1
kind: Pod
metadata:
  name: volume-emptydir
  namespace: dev
```

```
spec:
  containers:
  - name: nginx
    image: nginx:1.17.1
    ports:
    - containerPort: 80
    volumeMounts: # 将logs-volume挂在到nginx容器中，对应的目录为 /var/log/nginx
    - name: logs-volume
      mountPath: /var/log/nginx
  - name: busybox
    image: busybox:1.30
    command: ["/bin/sh", "-c", "tail -f /logs/access.log"] # 初始命令，动态读取指定文件中内容
    volumeMounts: # 将logs-volume 挂在到busybox容器中，对应的目录为 /logs
    - name: logs-volume
      mountPath: /logs
  volumes: # 声明volume， name为logs-volume，类型为emptyDir
  - name: logs-volume
    emptyDir: {}
```

```
# 创建Pod
[root@k8s-master01 ~]# kubectl create -f volume-emptydir.yaml
pod/volume-emptydir created

# 查看pod
[root@k8s-master01 ~]# kubectl get pods volume-emptydir -n dev -o wide
NAME                                READY    STATUS    RESTARTS   AGE      IP             NODE
.....
volume-emptydir                    2/2      Running   0           97s      10.42.2.9      node1
.....

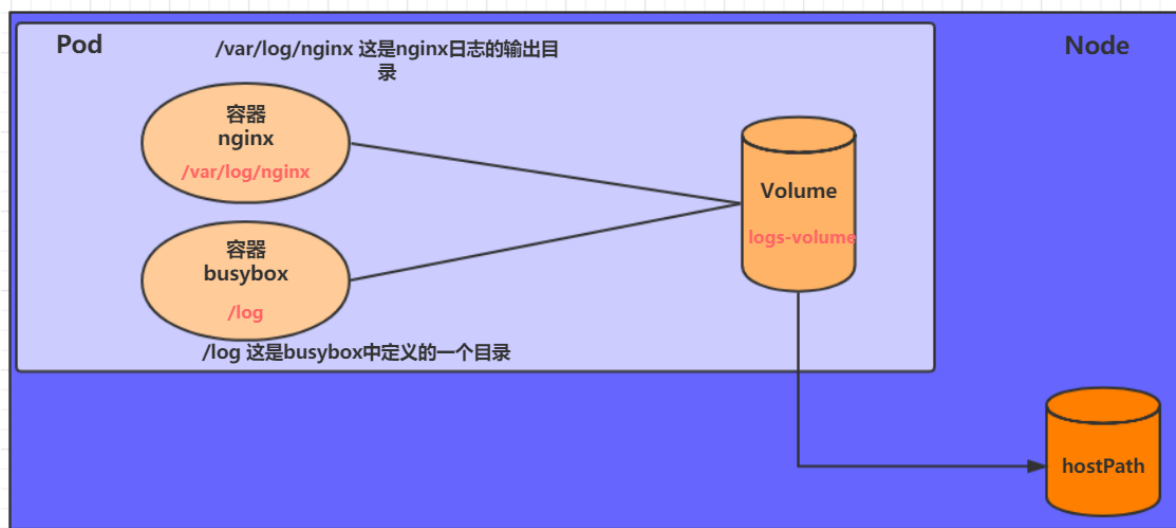
# 通过podIp访问nginx
[root@k8s-master01 ~]# curl 10.42.2.9
.....

# 通过kubectl logs命令查看指定容器的标准输出
[root@k8s-master01 ~]# kubectl logs -f volume-emptydir -n dev -c busybox
10.42.1.0 - - [27/Jun/2021:15:08:54 +0000] "GET / HTTP/1.1" 200 612 "-"
"curl/7.29.0" "-"
```

8.1.2 HostPath

上节课提到，EmptyDir中数据不会被持久化，它会随着Pod的结束而销毁，如果想简单的将数据持久化到主机中，可以选择HostPath。

HostPath就是将Node主机中一个实际目录挂在到Pod中，以供容器使用，这样的设计就可以保证Pod销毁了，但是数据依据可以存在于Node主机上。



创建一个volume-hostpath.yaml:

```
apiVersion: v1
kind: Pod
metadata:
  name: volume-hostpath
  namespace: dev
spec:
  containers:
    - name: nginx
      image: nginx:1.17.1
      ports:
        - containerPort: 80
      volumeMounts:
        - name: logs-volume
          mountPath: /var/log/nginx
    - name: busybox
      image: busybox:1.30
      command: ["/bin/sh", "-c", "tail -f /logs/access.log"]
      volumeMounts:
        - name: logs-volume
          mountPath: /logs
  volumes:
    - name: logs-volume
      hostPath:
        path: /root/logs
        type: DirectoryOrCreate # 目录存在就使用，不存在就先创建后使用
```

关于type的值的一点说明:

- DirectoryOrCreate** 目录存在就使用，不存在就先创建后使用
- Directory** 目录必须存在
- FileOrCreate** 文件存在就使用，不存在就先创建后使用
- File** 文件必须存在
- Socket** unix套接字必须存在
- CharDevice** 字符设备必须存在
- BlockDevice** 块设备必须存在

创建Pod

```
[root@k8s-master01 ~]# kubectl create -f volume-hostpath.yaml
```



```
pod/volume-hostpath created
```

```
# 查看Pod
```

```
[root@k8s-master01 ~]# kubectl get pods volume-hostpath -n dev -o wide
```

NAME	READY	STATUS	RESTARTS	AGE	IP	NODE
.....						
pod-volume-hostpath	2/2	Running	0	16s	10.42.2.10	node1
.....						

```
#访问nginx
```

```
[root@k8s-master01 ~]# curl 10.42.2.10
```

```
[root@k8s-master01 ~]# kubectl logs -f volume-emptydir -n dev -c busybox
```

```
# 接下来就可以去host的/root/logs目录下查看存储的文件了
```

```
### 注意：下面的操作需要到Pod所在的节点运行（案例中是node1）
```

```
[root@node1 ~]# ls /root/logs/
```

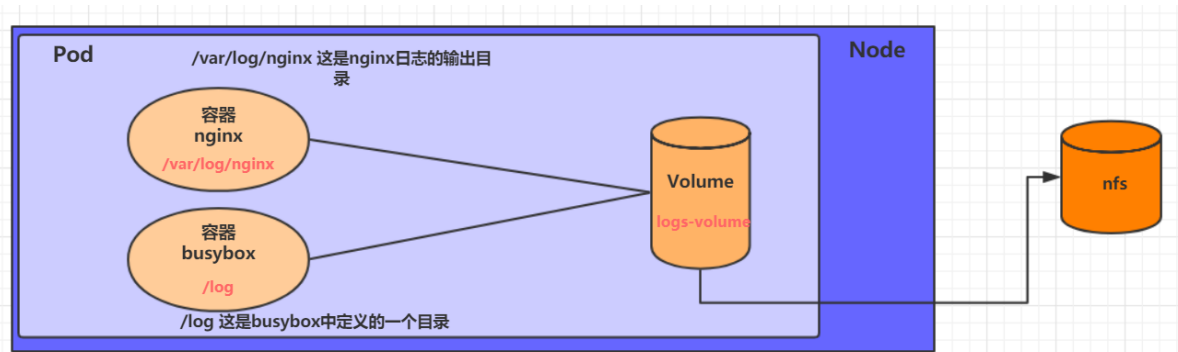
```
access.log  error.log
```

```
# 同样的道理，如果在此目录下创建一个文件，到容器中也是可以看到的
```

8.1.3 NFS

HostPath可以解决数据持久化的问题，但是一旦Node节点故障了，Pod如果转移到了别的节点，又会出现问题了，此时需要准备单独的网络存储系统，比较常用的用NFS、CIFS。

NFS是一个网络文件存储系统，可以搭建一台NFS服务器，然后将Pod中的存储直接连接到NFS系统上，这样的话，无论Pod在节点上怎么转移，只要Node跟NFS的对接没问题，数据就可以成功访问。



1) 首先要准备nfs的服务器，这里为了简单，直接是master节点做nfs服务器

```
# 在nfs上安装nfs服务
```

```
[root@nfs ~]# yum install nfs-utils -y
```

```
# 准备一个共享目录
```

```
[root@nfs ~]# mkdir /root/data/nfs -pv
```

```
# 将共享目录以读写权限暴露给192.168.5.0/24网段中的所有主机
```

```
[root@nfs ~]# vim /etc/exports
```

```
[root@nfs ~]# more /etc/exports
```

```
/root/data/nfs 192.168.5.0/24(rw,no_root_squash)
```

```
# 启动nfs服务
```

```
[root@nfs ~]# systemctl restart nfs
```

2) 接下来, 要在每个node节点上都安装下nfs, 这样的目的是为了node节点可以驱动nfs设备

```
# 在node上安装nfs服务, 注意不需要启动
[root@k8s-master01 ~]# yum install nfs-utils -y
```

3) 接下来, 就可以编写pod的配置文件了, 创建volume-nfs.yaml

```
apiVersion: v1
kind: Pod
metadata:
  name: volume-nfs
  namespace: dev
spec:
  containers:
  - name: nginx
    image: nginx:1.17.1
    ports:
    - containerPort: 80
    volumeMounts:
    - name: logs-volume
      mountPath: /var/log/nginx
  - name: busybox
    image: busybox:1.30
    command: ["/bin/sh", "-c", "tail -f /logs/access.log"]
    volumeMounts:
    - name: logs-volume
      mountPath: /logs
  volumes:
  - name: logs-volume
    nfs:
      server: 192.168.5.6 #nfs服务器地址
      path: /root/data/nfs #共享文件路径
```

4) 最后, 运行下pod, 观察结果

```
# 创建pod
[root@k8s-master01 ~]# kubectl create -f volume-nfs.yaml
pod/volume-nfs created

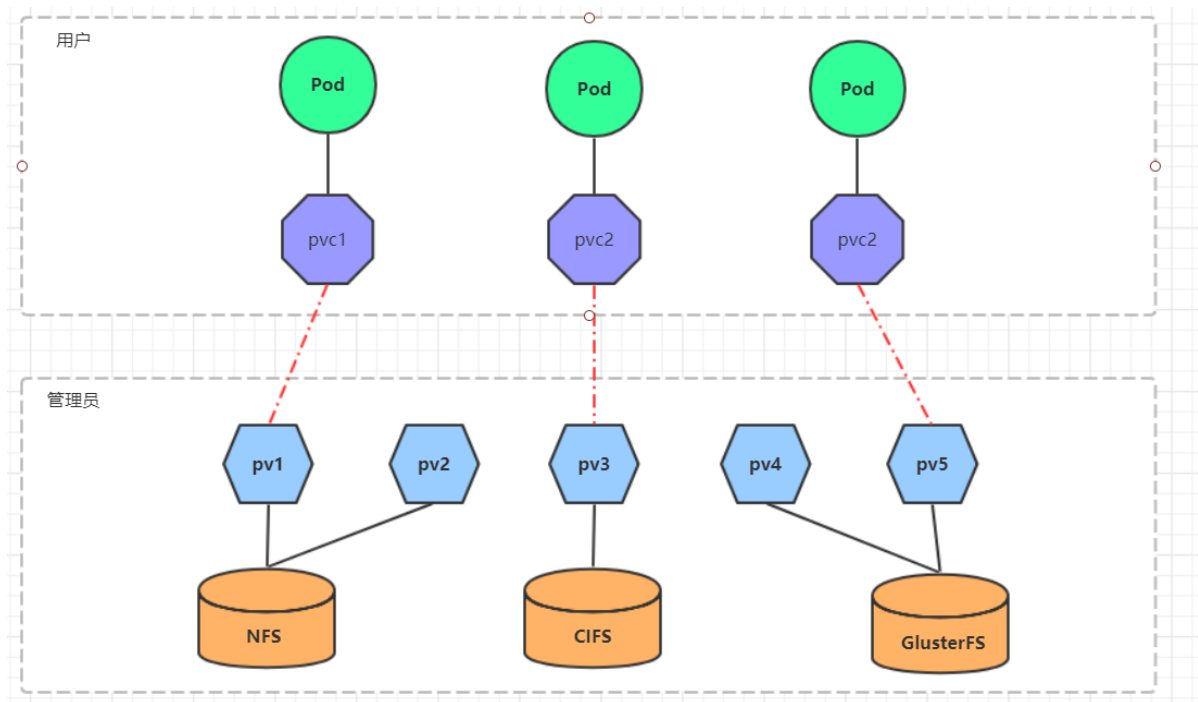
# 查看pod
[root@k8s-master01 ~]# kubectl get pods volume-nfs -n dev
NAME                READY   STATUS    RESTARTS   AGE
volume-nfs          2/2     Running   0           2m9s

# 查看nfs服务器上的共享目录, 发现已经有文件了
[root@k8s-master01 ~]# ls /root/data/
access.log  error.log
```

8.2 高级存储

前面已经学习了使用NFS提供存储，此时就要求用户会搭建NFS系统，并且会在yaml配置nfs。由于kubernetes支持的存储系统有很多，要求客户全都掌握，显然不现实。为了能够屏蔽底层存储实现的细节，方便用户使用，kubernetes引入PV和PVC两种资源对象。

- PV (Persistent Volume) 是持久化卷的意思，是对底层的共享存储的一种抽象。一般情况下PV由kubernetes管理员进行创建和配置，它与底层具体的共享存储技术有关，并通过插件完成与共享存储的对接。
- PVC (Persistent Volume Claim) 是持久卷声明的意思，是用户对于存储需求的一种声明。换句话说，PVC其实就是用户向kubernetes系统发出的一种资源需求申请。



使用了PV和PVC之后，工作可以得到进一步的细分：

- 存储：存储工程师维护
- PV：kubernetes管理员维护
- PVC：kubernetes用户维护

8.2.1 PV

PV是存储资源的抽象，下面是资源清单文件：

```
apiVersion: v1
kind: PersistentVolume
metadata:
  name: pv2
spec:
  nfs: # 存储类型，与底层真正存储对应
  capacity: # 存储能力，目前只支持存储空间设置
    storage: 2Gi
  accessModes: # 访问模式
  storageClassName: # 存储类别
  persistentVolumeReclaimPolicy: # 回收策略
```

PV 的关键配置参数说明：

- **存储类型**

底层实际存储的类型，kubernetes支持多种存储类型，每种存储类型的配置都有所差异

- **存储能力 (capacity)**

目前只支持存储空间的设置(storage=1Gi), 不过未来可能会加入IOPS、吞吐量等指标的配置

- **访问模式 (accessModes)**

用于描述用户应用对存储资源的访问权限，访问权限包括下面几种方式：

- ReadWriteOnce (RWO)：读写权限，但是只能被单个节点挂载
- ReadOnlyMany (ROX)：只读权限，可以被多个节点挂载
- ReadWriteMany (RWX)：读写权限，可以被多个节点挂载

需要注意的是，底层不同的存储类型可能支持的访问模式不同

- **回收策略 (persistentVolumeReclaimPolicy)**

当PV不再被使用了之后，对其的处理方式。目前支持三种策略：

- Retain（保留）保留数据，需要管理员手工清理数据
- Recycle（回收）清除 PV 中的数据，效果相当于执行 `rm -rf /thevolume/*`
- Delete（删除）与 PV 相连的后端存储完成 volume 的删除操作，当然这常见于云服务商的存储服务

需要注意的是，底层不同的存储类型可能支持的回收策略不同

- **存储类别**

PV可以通过storageClassName参数指定一个存储类别

- 具有特定类别的PV只能与请求了该类别的PVC进行绑定
- 未设定类别的PV则只能与不请求任何类别的PVC进行绑定

- **状态 (status)**

一个 PV 的生命周期中，可能会处于4中不同的阶段：

- Available（可用）：表示可用状态，还未被任何 PVC 绑定
- Bound（已绑定）：表示 PV 已经被 PVC 绑定
- Released（已释放）：表示 PVC 被删除，但是资源还未被集群重新声明
- Failed（失败）：表示该 PV 的自动回收失败

实验

使用NFS作为存储，来演示PV的使用，创建3个PV，对应NFS中的3个暴露的路径。

1. 准备NFS环境

```
# 创建目录
[root@nfs ~]# mkdir /root/data/{pv1,pv2,pv3} -pv

# 暴露服务
[root@nfs ~]# more /etc/exports
/root/data/pv1      192.168.5.0/24(rw,no_root_squash)
/root/data/pv2      192.168.5.0/24(rw,no_root_squash)
/root/data/pv3      192.168.5.0/24(rw,no_root_squash)

# 重启服务
[root@nfs ~]# systemctl restart nfs
```

2. 创建pv.yaml

```
apiVersion: v1
kind: PersistentVolume
metadata:
  name: pv1
spec:
  capacity:
    storage: 1Gi
  accessModes:
    - ReadWriteMany
  persistentVolumeReclaimPolicy: Retain
  nfs:
    path: /root/data/pv1
    server: 192.168.5.6

---

apiVersion: v1
kind: PersistentVolume
metadata:
  name: pv2
spec:
  capacity:
    storage: 2Gi
  accessModes:
    - ReadWriteMany
  persistentVolumeReclaimPolicy: Retain
  nfs:
    path: /root/data/pv2
    server: 192.168.5.6

---

apiVersion: v1
kind: PersistentVolume
metadata:
  name: pv3
spec:
  capacity:
    storage: 3Gi
  accessModes:
```

```
- ReadwriteMany
persistentVolumeReclaimPolicy: Retain
nfs:
  path: /root/data/pv3
  server: 192.168.5.6
```

```
# 创建 pv
[root@k8s-master01 ~]# kubectl create -f pv.yaml
persistentvolume/pv1 created
persistentvolume/pv2 created
persistentvolume/pv3 created

# 查看pv
[root@k8s-master01 ~]# kubectl get pv -o wide
```

NAME	CAPACITY	ACCESS MODES	RECLAIM POLICY	STATUS	AGE	VOLUMEMODE
pv1	1Gi	RWX	Retain	Available	10s	Filesystem
pv2	2Gi	RWX	Retain	Available	10s	Filesystem
pv3	3Gi	RWX	Retain	Available	9s	Filesystem

8.2.2 PVC

PVC是资源的申请，用来声明对存储空间、访问模式、存储类别需求信息。下面是资源清单文件：

```
apiVersion: v1
kind: PersistentVolumeClaim
metadata:
  name: pvc
  namespace: dev
spec:
  accessModes: # 访问模式
  selector: # 采用标签对PV选择
  storageClassName: # 存储类别
  resources: # 请求空间
    requests:
      storage: 5Gi
```

PVC 的关键配置参数说明：

- **访问模式 (accessModes)**

用于描述用户应用对存储资源的访问权限

- **选择条件 (selector)**

通过Label Selector的设置，可使PVC对于系统中已存在的PV进行筛选

- **存储类别 (storageClassName)**

PVC在定义时可以设定需要的后端存储的类别，只有设置了该class的pv才能被系统选出

- **资源请求 (Resources)**

描述对存储资源的请求

实验

1. 创建pvc.yaml，申请pv

```

apiVersion: v1
kind: PersistentVolumeClaim
metadata:
  name: pvc1
  namespace: dev
spec:
  accessModes:
    - ReadWriteMany
  resources:
    requests:
      storage: 1Gi
---
apiVersion: v1
kind: PersistentVolumeClaim
metadata:
  name: pvc2
  namespace: dev
spec:
  accessModes:
    - ReadWriteMany
  resources:
    requests:
      storage: 1Gi
---
apiVersion: v1
kind: PersistentVolumeClaim
metadata:
  name: pvc3
  namespace: dev
spec:
  accessModes:
    - ReadWriteMany
  resources:
    requests:
      storage: 1Gi

```

创建pvc

```

[root@k8s-master01 ~]# kubectl create -f pvc.yaml
persistentvolumeclaim/pvc1 created
persistentvolumeclaim/pvc2 created
persistentvolumeclaim/pvc3 created

```

查看pvc

```

[root@k8s-master01 ~]# kubectl get pvc -n dev -o wide

```

NAME	STATUS	VOLUME	CAPACITY	ACCESS MODES	STORAGECLASS	AGE
pvc1	Bound	pv1	1Gi	RWX		15s
Filesystem						
pvc2	Bound	pv2	2Gi	RWX		15s
Filesystem						
pvc3	Bound	pv3	3Gi	RWX		15s
Filesystem						

查看pv

```

[root@k8s-master01 ~]# kubectl get pv -o wide

```

NAME	CAPACITY	ACCESS MODES	RECLAIM POLICY	STATUS	CLAIM	AGE
VOLUME MODE						
pv1	1Gi	RWx	Retain	Bound	dev/pvc1	3h37m
Filesystem						
pv2	2Gi	RWX	Retain	Bound	dev/pvc2	3h37m
Filesystem						
pv3	3Gi	RWX	Retain	Bound	dev/pvc3	3h37m
Filesystem						

2. 创建pods.yaml, 使用pv

```

apiVersion: v1
kind: Pod
metadata:
  name: pod1
  namespace: dev
spec:
  containers:
  - name: busybox
    image: busybox:1.30
    command: ["/bin/sh", "-c", "while true;do echo pod1 >> /root/out.txt; sleep 10;
done;"]
    volumeMounts:
    - name: volume
      mountPath: /root/
  volumes:
  - name: volume
    persistentVolumeClaim:
      claimName: pvc1
      readOnly: false
---
apiVersion: v1
kind: Pod
metadata:
  name: pod2
  namespace: dev
spec:
  containers:
  - name: busybox
    image: busybox:1.30
    command: ["/bin/sh", "-c", "while true;do echo pod2 >> /root/out.txt; sleep 10;
done;"]
    volumeMounts:
    - name: volume
      mountPath: /root/
  volumes:
  - name: volume
    persistentVolumeClaim:
      claimName: pvc2
      readOnly: false

```

创建pod

```

[root@k8s-master01 ~]# kubectl create -f pods.yaml
pod/pod1 created

```



```
pod/pod2 created
```

```
# 查看pod
```

```
[root@k8s-master01 ~]# kubectl get pods -n dev -o wide
```

NAME	READY	STATUS	RESTARTS	AGE	IP	NODE
pod1	1/1	Running	0	14s	10.244.1.69	node1
pod2	1/1	Running	0	14s	10.244.1.70	node1

```
# 查看pvc
```

```
[root@k8s-master01 ~]# kubectl get pvc -n dev -o wide
```

NAME	STATUS	VOLUME	CAPACITY	ACCESS MODES	AGE	VOLUMEMODE
pvc1	Bound	pv1	1Gi	RWX	94m	Filesystem
pvc2	Bound	pv2	2Gi	RWX	94m	Filesystem
pvc3	Bound	pv3	3Gi	RWX	94m	Filesystem

```
# 查看pv
```

```
[root@k8s-master01 ~]# kubectl get pv -n dev -o wide
```

NAME	CAPACITY	ACCESS MODES	RECLAIM POLICY	STATUS	CLAIM	AGE
pv1	1Gi	RWX	Retain	Bound	dev/pvc1	5h11m
pv2	2Gi	RWX	Retain	Bound	dev/pvc2	5h11m
pv3	3Gi	RWX	Retain	Bound	dev/pvc3	5h11m

```
# 查看nfs中的文件存储
```

```
[root@nfs ~]# more /root/data/pv1/out.txt
```

```
node1
```

```
node1
```

```
[root@nfs ~]# more /root/data/pv2/out.txt
```

```
node2
```

```
node2
```

8.2.3 生命周期

PVC和PV是一一对应的，PV和PVC之间的相互作用遵循以下生命周期：

- **资源供应：**管理员手动创建底层存储和PV

- **资源绑定：**用户创建PVC，kubernetes负责根据PVC的声明去寻找PV，并绑定

在用户定义好PVC之后，系统将根据PVC对存储资源的请求在已存在的PV中选择一个满足条件的

- 一旦找到，就将该PV与用户定义的PVC进行绑定，用户的应用就可以使用这个PVC了
- 如果找不到，PVC则会无限期处于Pending状态，直到等到系统管理员创建了一个符合其要求的PV

PV一旦绑定到某个PVC上，就会被这个PVC独占，不能再与其他PVC进行绑定了

- **资源使用：**用户可在pod中像volume一样使用pvc

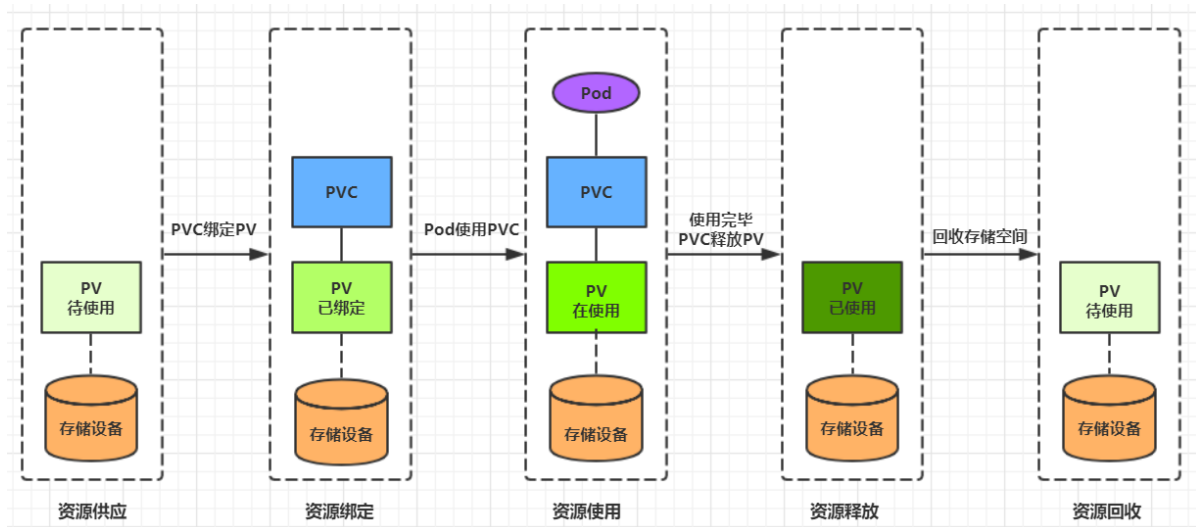
Pod使用Volume的定义，将PVC挂载到容器内的某个路径进行使用。

- **资源释放：**用户删除pvc来释放pv

当存储资源使用完毕后，用户可以删除PVC，与该PVC绑定的PV将会被标记为“已释放”，但还不能立刻与其他PVC进行绑定。通过之前PVC写入的数据可能还被留在存储设备上，只有在清除之后该PV才能再次使用。

- **资源回收**: kubernetes根据pv设置的回收策略进行资源的回收

对于PV，管理员可以设定回收策略，用于设置与之绑定的PVC释放资源之后如何处理遗留数据的问题。只有PV的存储空间完成回收，才能供新的PVC绑定和使用



8.3 配置存储

8.3.1 ConfigMap

ConfigMap是一种比较特殊的存储卷，它的主要作用是用来存储配置信息的。

创建configmap.yaml，内容如下：

```
apiVersion: v1
kind: ConfigMap
metadata:
  name: configmap
  namespace: dev
data:
  info: |
    username:admin
    password:123456
```

接下来，使用此配置文件创建configmap

```
# 创建configmap
[root@k8s-master01 ~]# kubectl create -f configmap.yaml
configmap/configmap created

# 查看configmap详情
[root@k8s-master01 ~]# kubectl describe cm configmap -n dev
Name:         configmap
Namespace:    dev
Labels:       <none>
Annotations:  <none>

Data
====
info:
----
username:admin
```

```
password:123456
```

```
Events: <none>
```

接下来创建一个pod-configmap.yaml, 将上面创建的configmap挂载进去

```
apiVersion: v1
kind: Pod
metadata:
  name: pod-configmap
  namespace: dev
spec:
  containers:
  - name: nginx
    image: nginx:1.17.1
    volumeMounts: # 将configmap挂载到目录
  - name: config
    mountPath: /configmap/config
  volumes: # 引用configmap
  - name: config
    configMap:
      name: configmap
```

```
# 创建pod
```

```
[root@k8s-master01 ~]# kubectl create -f pod-configmap.yaml
pod/pod-configmap created
```

```
# 查看pod
```

```
[root@k8s-master01 ~]# kubectl get pod pod-configmap -n dev
```

NAME	READY	STATUS	RESTARTS	AGE
pod-configmap	1/1	Running	0	6s

```
#进入容器
```

```
[root@k8s-master01 ~]# kubectl exec -it pod-configmap -n dev /bin/sh
# cd /configmap/config/
```

```
# ls
```

```
info
```

```
# more info
```

```
username:admin
```

```
password:123456
```

```
# 可以看到映射已经成功, 每个configmap都映射成了一个目录
```

```
# key---->文件      value---->文件中的内容
```

```
# 此时如果更新configmap的内容, 容器中的值也会动态更新
```

8.3.2 Secret

在kubernetes中, 还存在一种和ConfigMap非常类似的对象, 称为Secret对象。它主要用于存储敏感信息, 例如密码、秘钥、证书等等。

1. 首先使用base64对数据进行编码

```
[root@k8s-master01 ~]# echo -n 'admin' | base64 #准备username
YWRTaw4=
[root@k8s-master01 ~]# echo -n '123456' | base64 #准备password
MTIzNDU2
```

2. 接下来编写secret.yaml, 并创建Secret

```
apiVersion: v1
kind: Secret
metadata:
  name: secret
  namespace: dev
type: Opaque
data:
  username: YWRTaw4=
  password: MTIzNDU2
```

```
# 创建secret
[root@k8s-master01 ~]# kubectl create -f secret.yaml
secret/secret created

# 查看secret详情
[root@k8s-master01 ~]# kubectl describe secret secret -n dev
Name:          secret
Namespace:     dev
Labels:        <none>
Annotations:   <none>
Type:          Opaque
Data
====
password:      6 bytes
username:      5 bytes
```

3. 创建pod-secret.yaml, 将上面创建的secret挂载进去:

```
apiVersion: v1
kind: Pod
metadata:
  name: pod-secret
  namespace: dev
spec:
  containers:
    - name: nginx
      image: nginx:1.17.1
      volumeMounts: # 将secret挂载到目录
        - name: config
          mountPath: /secret/config
  volumes:
    - name: config
      secret:
        secretName: secret
```

```
# 创建pod
```

```
[root@k8s-master01 ~]# kubectl create -f pod-secret.yaml
pod/pod-secret created

# 查看pod
[root@k8s-master01 ~]# kubectl get pod pod-secret -n dev
NAME          READY   STATUS    RESTARTS   AGE
pod-secret    1/1     Running   0           2m28s

# 进入容器，查看secret信息，发现已经自动解码了
[root@k8s-master01 ~]# kubectl exec -it pod-secret /bin/sh -n dev
/ # ls /secret/config/
password  username
/ # more /secret/config/username
admin
/ # more /secret/config/password
123456
```

至此，已经实现了利用secret实现了信息的编码。

9. 安全认证

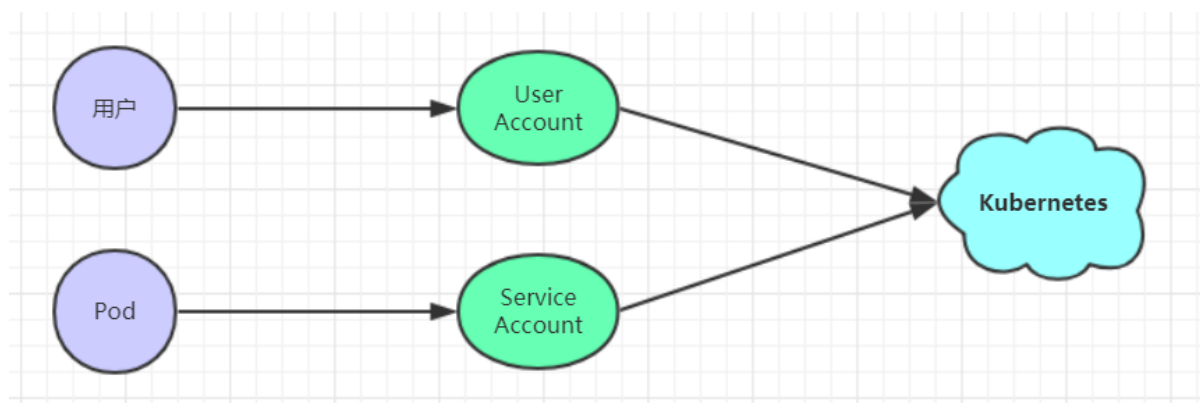
9.1 访问控制概述

Kubernetes作为一个分布式集群的管理工具，保证集群的安全性是其一个重要的任务。所谓的安全性其实就是保证对Kubernetes的各种**客户端**进行**认证和鉴权**操作。

客户端

在Kubernetes集群中，客户端通常有两类：

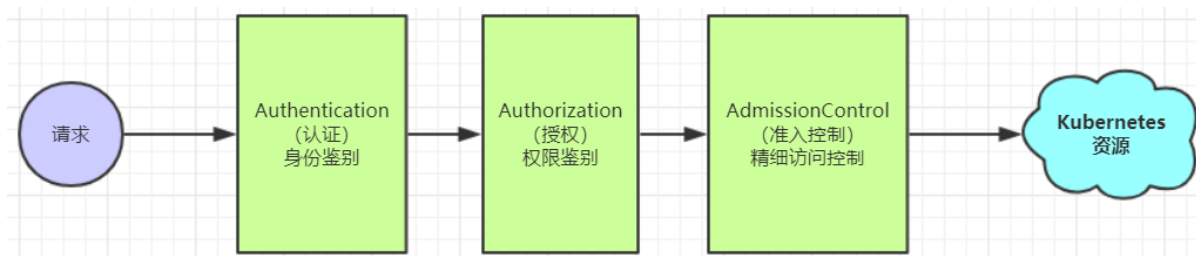
- **User Account**：一般是独立于kubernetes之外的其他服务管理的用户账号。
- **Service Account**：kubernetes管理的账号，用于为Pod中的服务进程在访问Kubernetes时提供身份标识。



认证、授权与准入控制

ApiServer是访问及管理资源对象的唯一入口。任何一个请求访问ApiServer，都要经过下面三个流程：

- **Authentication (认证)**：身份鉴别，只有正确的账号才能够通过认证
- **Authorization (授权)**：判断用户是否有权限对访问的资源执行特定的动作
- **Admission Control (准入控制)**：用于补充授权机制以实现更加精细的访问控制功能。



9.2 认证管理

Kubernetes集群安全的最关键点在于如何识别并认证客户端身份，它提供了3种客户端身份认证方式：

- HTTP Base认证：通过用户名+密码的方式认证

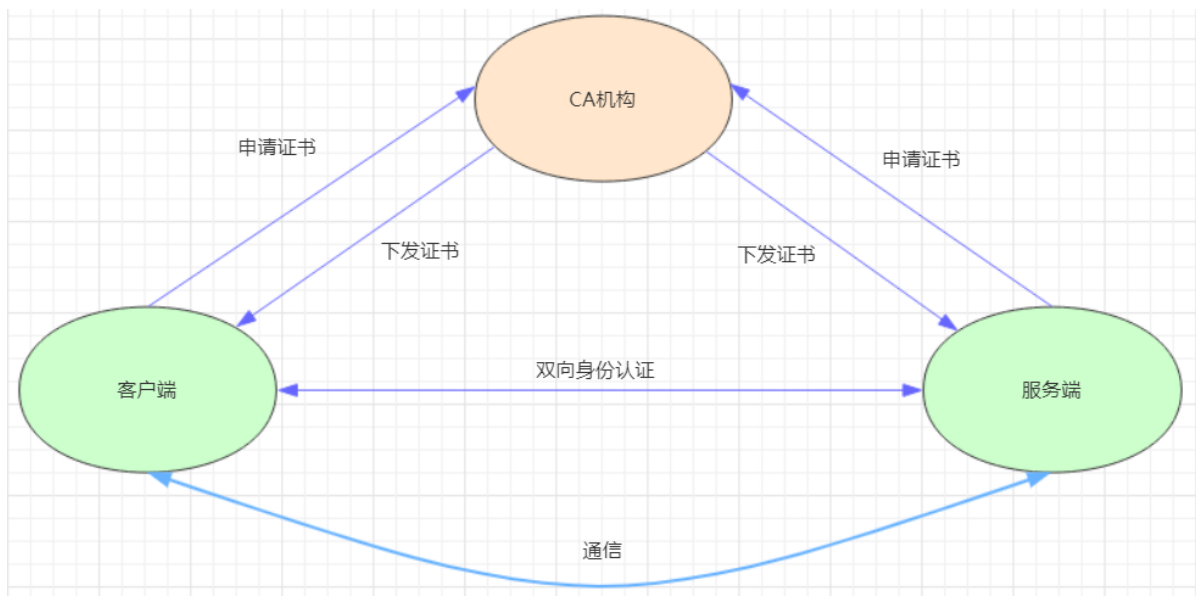
这种认证方式是把“用户名:密码”用BASE64算法进行编码后的字符串放在HTTP请求中的Header **Authorization**域里发送给服务端。服务端收到后进行解码，获取用户名及密码，然后进行用户身份认证的过程。

- HTTP Token认证：通过一个Token来识别合法用户

这种认证方式是用一个很长的难以被模仿的字符串--**Token**来表明客户身份的一种方式。每个**Token**对应一个用户名，当客户端发起API调用请求时，需要在HTTP Header里放入**Token**，API Server接到**Token**后会跟服务器中保存的**token**进行比对，然后进行用户身份认证的过程。

- HTTPS证书认证：基于CA根证书签名的双向数字证书认证方式

这种认证方式是安全性最高的一种方式，但是同时也是操作起来最麻烦的一种方式。



HTTPS认证大体分为3个过程：

1. 证书申请和下发

HTTPS通信双方的服务器向CA机构申请证书，CA机构下发根证书、服务端证书及私钥给申请者

2. 客户端和服务端的双向认证

- 1> 客户端向服务器端发起请求，服务器端下发自己的证书给客户端，
客户端接收到证书后，通过私钥解密证书，在证书中获得服务器端的公钥，
客户端利用服务器端的公钥认证证书中的信息，如果一致，则认可这个服务器
- 2> 客户端发送自己的证书给服务器端，服务器端接收到证书后，通过私钥解密证书，
在证书中获得客户端的公钥，并用该公钥认证证书信息，确认客户端是否合法

3. 服务器端和客户端进行通信

服务器端和客户端协商好加密方案后，客户端会产生一个随机的密钥并加密，然后发送到服务器端。
服务器端接收这个密钥后，双方接下来通信的所有内容都通过该随机密钥加密

注意: Kubernetes允许同时配置多种认证方式，只要其中任意一个方式认证通过即可

9.3 授权管理

授权发生在认证成功之后，通过认证就可以知道请求用户是谁，然后Kubernetes会根据事先定义的授权策略来决定用户是否有权限访问，这个过程就称为授权。

每个发送到ApiServer的请求都带上了用户和资源的信息：比如发送请求的用户、请求的路径、请求的动作等，授权就是根据这些信息和授权策略进行比较，如果符合策略，则认为授权通过，否则会返回错误。

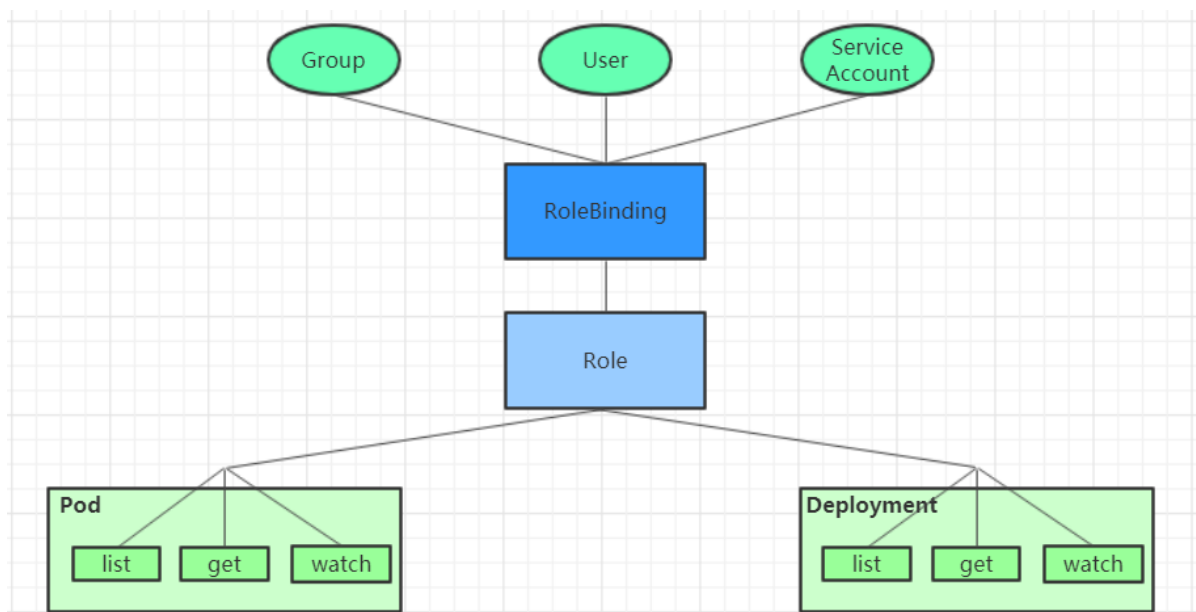
API Server目前支持以下几种授权策略：

- AlwaysDeny：表示拒绝所有请求，一般用于测试
- AlwaysAllow：允许接收所有请求，相当于集群不需要授权流程（Kubernetes默认的策略）
- ABAC：基于属性的访问控制，表示使用用户配置的授权规则对用户请求进行匹配和控制
- Webhook：通过调用外部REST服务对用户进行授权
- Node：是一种专用模式，用于对kubelet发出的请求进行访问控制
- RBAC：基于角色的访问控制（kubeadm安装方式下的默认选项）

RBAC(Role-Based Access Control) 基于角色的访问控制，主要是在描述一件事情：**给哪些对象授予了哪些权限**

其中涉及到了下面几个概念：

- 对象：User、Groups、ServiceAccount
- 角色：代表着一组定义在资源上的可操作动作(权限)的集合
- 绑定：将定义好的角色跟用户绑定在一起



RBAC引入了4个顶级资源对象：

- Role、ClusterRole：角色，用于指定一组权限
- RoleBinding、ClusterRoleBinding：角色绑定，用于将角色（权限）赋予给对象

Role、ClusterRole

一个角色就是一组权限的集合，这里的权限都是许可形式的（白名单）。

```
# Role只能对命名空间内的资源进行授权，需要指定namespace
kind: Role
apiVersion: rbac.authorization.k8s.io/v1beta1
metadata:
  namespace: dev
  name: authorization-role
rules:
- apiGroups: [""] # 支持的API组列表，"" 空字符串，表示核心API群
  resources: ["pods"] # 支持的资源对象列表
  verbs: ["get", "watch", "list"] # 允许的对资源对象的操作方法列表
```

```
# ClusterRole可以对集群范围内资源、跨namespaces的范围资源、非资源类型进行授权
kind: ClusterRole
apiVersion: rbac.authorization.k8s.io/v1beta1
metadata:
  name: authorization-clusterrole
rules:
- apiGroups: [""]
  resources: ["pods"]
  verbs: ["get", "watch", "list"]
```

需要详细说明的是，rules中的参数：

- apiGroups: 支持的API组列表

```
"" , "apps", "autoscaling", "batch"
```

- resources: 支持的资源对象列表


```
"services", "endpoints",
"pods", "secrets", "configmaps", "crontabs", "deployments", "jobs",
"nodes", "rolebindings", "clusterroles", "daemonsets", "replicasets", "statefulsets",
"horizontalpodautoscalers", "replicationcontrollers", "cronjobs"
```

- verbs: 对资源对象的操作方法列表

```
"get", "list", "watch", "create", "update", "patch", "delete", "exec"
```

RoleBinding、ClusterRoleBinding

角色绑定用来把一个角色绑定到一个目标对象上，绑定目标可以是User、Group或者ServiceAccount。

```
# RoleBinding可以将同一namespace中的subject绑定到某个Role下，则此subject即具有该Role定义的权限
kind: RoleBinding
apiVersion: rbac.authorization.k8s.io/v1beta1
metadata:
  name: authorization-role-binding
  namespace: dev
subjects:
- kind: User
  name: jax
  apiGroup: rbac.authorization.k8s.io
roleRef:
  kind: Role
  name: authorization-role
  apiGroup: rbac.authorization.k8s.io
```

```
# ClusterRoleBinding在整个集群级别和所有namespaces将特定的subject与ClusterRole绑定，授予权限
kind: ClusterRoleBinding
apiVersion: rbac.authorization.k8s.io/v1beta1
metadata:
  name: authorization-clusterrole-binding
subjects:
- kind: User
  name: jax
  apiGroup: rbac.authorization.k8s.io
roleRef:
  kind: ClusterRole
  name: authorization-clusterrole
  apiGroup: rbac.authorization.k8s.io
```

RoleBinding引用ClusterRole进行授权

RoleBinding可以引用ClusterRole，对属于同一命名空间内ClusterRole定义的资源主体进行授权。

一种很常用的做法就是，集群管理员为集群范围预定义好一组角色（ClusterRole），然后在多个命名空间中重复使用这些ClusterRole。这样可以大幅提高授权管理工作效率，也使得各个命名空间下的基础性授权规则与使用体验保持一致。

```
# 虽然authorization-clusterrole是一个集群角色，但是因为使用了RoleBinding
```

```
# 所以jax只能读取dev命名空间中的资源
kind: RoleBinding
apiVersion: rbac.authorization.k8s.io/v1beta1
metadata:
  name: authorization-role-binding-ns
  namespace: dev
subjects:
- kind: User
  name: jax
  apiGroup: rbac.authorization.k8s.io
roleRef:
  kind: ClusterRole
  name: authorization-clusterrole
  apiGroup: rbac.authorization.k8s.io
```

实战：创建一个只能管理dev空间下Pods资源的账号

1. 创建账号

```
# 1) 创建证书
[root@k8s-master01 pki]# cd /etc/kubernetes/pki/
[root@k8s-master01 pki]# (umask 077;openssl genrsa -out devman.key 2048)

# 2) 用apiserver的证书去签署
# 2-1) 签名申请，申请的用户是devman,组是devgroup
[root@k8s-master01 pki]# openssl req -new -key devman.key -out devman.csr -subj
"/CN=devman/O=devgroup"
# 2-2) 签署证书
[root@k8s-master01 pki]# openssl x509 -req -in devman.csr -CA ca.crt -CAkey
ca.key -CAcreateserial -out devman.crt -days 3650

# 3) 设置集群、用户、上下文信息
[root@k8s-master01 pki]# kubectl config set-cluster kubernetes --embed-certs=true
--certificate-authority=/etc/kubernetes/pki/ca.crt --
server=https://192.168.109.100:6443

[root@k8s-master01 pki]# kubectl config set-credentials devman --embed-certs=true
--client-certificate=/etc/kubernetes/pki/devman.crt --client-
key=/etc/kubernetes/pki/devman.key

[root@k8s-master01 pki]# kubectl config set-context devman@kubernetes --
cluster=kubernetes --user=devman

# 切换账户到devman
[root@k8s-master01 pki]# kubectl config use-context devman@kubernetes
Switched to context "devman@kubernetes".

# 查看dev下pod，发现没有权限
[root@k8s-master01 pki]# kubectl get pods -n dev
Error from server (Forbidden): pods is forbidden: User "devman" cannot list
resource "pods" in API group "" in the namespace "dev"

# 切换到admin账户
[root@k8s-master01 pki]# kubectl config use-context kubernetes-admin@kubernetes
Switched to context "kubernetes-admin@kubernetes".
```

2) 创建Role和RoleBinding, 为devman用户授权

```
kind: Role
apiVersion: rbac.authorization.k8s.io/v1beta1
metadata:
  namespace: dev
  name: dev-role
rules:
- apiGroups: ["" ]
  resources: ["pods"]
  verbs: ["get", "watch", "list"]

---

kind: RoleBinding
apiVersion: rbac.authorization.k8s.io/v1beta1
metadata:
  name: authorization-role-binding
  namespace: dev
subjects:
- kind: User
  name: devman
  apiGroup: rbac.authorization.k8s.io
roleRef:
  kind: Role
  name: dev-role
  apiGroup: rbac.authorization.k8s.io
```

```
[root@k8s-master01 pki]# kubectl create -f dev-role.yaml
role.rbac.authorization.k8s.io/dev-role created
rolebinding.rbac.authorization.k8s.io/authorization-role-binding created
```

3. 切换账户, 再次验证

```
# 切换账户到devman
[root@k8s-master01 pki]# kubectl config use-context devman@kubernetes
Switched to context "devman@kubernetes".

# 再次查看
[root@k8s-master01 pki]# kubectl get pods -n dev
```

NAME	READY	STATUS	RESTARTS	AGE
nginx-deployment-66cb59b984-8wp2k	1/1	Running	0	4d1h
nginx-deployment-66cb59b984-dc46j	1/1	Running	0	4d1h
nginx-deployment-66cb59b984-thfck	1/1	Running	0	4d1h

```
# 为了不影响后面的学习,切回admin账户
[root@k8s-master01 pki]# kubectl config use-context kubernetes-admin@kubernetes
Switched to context "kubernetes-admin@kubernetes".
```

9.4 准入控制

通过了前面的认证和授权之后，还需要经过准入控制处理通过之后，apiserver才会处理这个请求。

准入控制是一个可配置的控制器列表，可以通过在Api-Server上通过命令行设置选择执行哪些准入控制器：

```
--admission-  
control=NamespaceLifecycle,LimitRanger,ServiceAccount,PersistentVolumeLabel,  
DefaultStorageClass,ResourceQuota,DefaultTolerationSeconds
```

只有当所有的准入控制器都检查通过之后，apiserver才执行该请求，否则返回拒绝。

当前可配置的Admission Control准入控制如下：

- AlwaysAdmit：允许所有请求
- AlwaysDeny：禁止所有请求，一般用于测试
- AlwaysPullImages：在启动容器之前总去下载镜像
- DenyExecOnPrivileged：它会拦截所有想在Privileged Container上执行命令的请求
- ImagePolicyWebhook：这个插件将允许后端的一个Webhook程序来完成admission controller的功能。
- Service Account：实现ServiceAccount实现了自动化
- SecurityContextDeny：这个插件将使用SecurityContext的Pod中的定义全部失效
- ResourceQuota：用于资源配额管理目的，观察所有请求，确保在namespace上的配额不会超标
- LimitRanger：用于资源限制管理，作用于namespace上，确保对Pod进行资源限制
- InitialResources：为未设置资源请求与限制的Pod，根据其镜像的历史资源的使用情况进行设置
- NamespaceLifecycle：如果尝试在一个不存在的namespace中创建资源对象，则该创建请求将被拒绝。当删除一个namespace时，系统将会删除该namespace中所有对象。
- DefaultStorageClass：为了实现共享存储的动态供应，为未指定StorageClass或PV的PVC尝试匹配默认的StorageClass，尽可能减少用户在申请PVC时所需了解的后端存储细节
- DefaultTolerationSeconds：这个插件为那些没有设置forgiveness tolerations并具有notready:NoExecute和unreachable:NoExecute两种taints的Pod设置默认的“容忍”时间，为5min
- PodSecurityPolicy：这个插件用于在创建或修改Pod时决定是否根据Pod的security context和可用的PodSecurityPolicy对Pod的安全策略进行控制

10. Dashboard

之前在kubernetes中完成的所有操作都是通过命令行工具kubectl完成的。其实，为了提供更丰富的用户体验，kubernetes还开发了一个基于web的用户界面（Dashboard）。用户可以使用Dashboard部署容器化的应用，还可以监控应用的状态，执行故障排查以及管理kubernetes中各种资源。

10.1 部署Dashboard

1. 下载yaml，并运行Dashboard

```
# 下载yaml  
[root@k8s-master01 ~]# wget  
https://raw.githubusercontent.com/kubernetes/dashboard/v2.0.0/aio/deploy/recommen  
ded.yaml
```

修改kubernetes-dashboard的Service类型

```
kind: Service
apiVersion: v1
metadata:
  labels:
    k8s-app: kubernetes-dashboard
  name: kubernetes-dashboard
  namespace: kubernetes-dashboard
spec:
  type: NodePort # 新增
  ports:
    - port: 443
      targetPort: 8443
      nodePort: 30009 # 新增
  selector:
    k8s-app: kubernetes-dashboard
```

部署kubernetes-dashboard 不能安装在master, 需要手动指定 node1, 或者node2

```
[root@k8s-master01 ~]# kubectl create -f recommended.yaml
```

查看namespace下的kubernetes-dashboard下的资源

```
[root@k8s-master01 ~]# kubectl get pod,svc -n kubernetes-dashboard
```

NAME	READY	STATUS	RESTARTS	AGE
pod/dashboard-metrics-scraper-c79c65bb7-zwfvw	1/1	Running	0	111s
pod/kubernetes-dashboard-56484d4c5-z95z5	1/1	Running	0	111s

NAME	TYPE	CLUSTER-IP	EXTERNAL-IP
service/dashboard-metrics-scraper	ClusterIP	10.96.89.218	<none>
8000/TCP			
service/kubernetes-dashboard	NodePort	10.104.178.171	<none>
443:30009/TCP			

- 问题 - 安装出现的问题

```
[root@master ~]# kubectl get pod,svc -n kubernetes-dashboard
```

NAME	READY	STATUS	RESTARTS	AGE
pod/dashboard-metrics-scraper-65dbd6db9-f9k44	1/1	Running	0	44s
pod/kubernetes-dashboard-65d47bfff7-jrxqs	0/1	CrashLoopBackOff	1	44s

NAME	TYPE	CLUSTER-IP	EXTERNAL-IP
service/dashboard-metrics-scraper	ClusterIP	10.104.225.47	<none>
8000/TCP			
service/kubernetes-dashboard	NodePort	10.102.124.182	<none>
443:30009/TCP			

```
[root@master ~]# kubectl logs -f -n kubernetes-dashboard kubernetes-dashboard-65d47bfff7-jrxqs
```

```
2022/04/13 12:44:09 Starting overwatch
```

```
2022/04/13 12:44:09 Using namespace: kubernetes-dashboard
```

```
2022/04/13 12:44:09 Using in-cluster config to connect to apiserver
```

```
2022/04/13 12:44:09 Using secret token for csrf signing
2022/04/13 12:44:09 Initializing csrf token from kubernetes-dashboard-csrf secret
panic: Get https://10.96.0.1:443/api/v1/namespaces/kubernetes-
dashboard/secrets/kubernetes-dashboard-csrf: dial tcp 10.96.0.1:443: connect:
network is unreachable
```

```
goroutine 1 [running]:
github.com/kubernetes/dashboard/src/app/backend/client/csrf.
(*csrfTokenManager).init(0xc000461da0)

/home/travis/build/kubernetes/dashboard/src/app/backend/client/csrf/manager.go:41
+0x446
github.com/kubernetes/dashboard/src/app/backend/client/csrf.NewCsrfTokenManager(.
..)

/home/travis/build/kubernetes/dashboard/src/app/backend/client/csrf/manager.go:66
github.com/kubernetes/dashboard/src/app/backend/client.
(*clientManager).initCSRfKey(0xc00040cf80)

/home/travis/build/kubernetes/dashboard/src/app/backend/client/manager.go:501
+0xc6
github.com/kubernetes/dashboard/src/app/backend/client.
(*clientManager).init(0xc00040cf80)

/home/travis/build/kubernetes/dashboard/src/app/backend/client/manager.go:469
+0x47
github.com/kubernetes/dashboard/src/app/backend/client.NewClientManager(...)

/home/travis/build/kubernetes/dashboard/src/app/backend/client/manager.go:550
main.main()
    /home/travis/build/kubernetes/dashboard/src/app/backend/dashboard.go:105
+0x20d
```

```
# 三台节点 都 curl 10.96.0.1:443 均能访问，排除网络问题
# 查看 recommended.yaml 文件
# Comment the following tolerations if Dashboard must not be deployed on master
看到说有关master的信息
dashboard-metrics-scraper, kubernetes-dashboard不能安装，指定了个node1就能启动
```

2) 创建访问账户，获取token

```
#创建账号
[root@master ~]# kubectl create serviceaccount dashboard-admin -n kubernetes-
dashboard
serviceaccount/dashboard-admin created

#授权
[root@master ~]# kubectl create clusterrolebinding dashboard-admin-rb --
clusterrole=cluster-admin --serviceaccount=kubernetes-dashboard:dashboard-admin
clusterrolebinding.rbac.authorization.k8s.io/dashboard-admin-rb created
```

```
[root@master ~]# kubectl get secrets -n kubernetes-dashboard | grep dashboard-admin
```

#为上方密钥生成token

```
[root@master ~]# kubectl describe secrets dashboard-admin-token-d[cr]l -n
kubernetes-dashboard
```

```
Name:      dashboard-admin-token-d1cr1
Namespace:  kubernetes-dashboard
Labels:    <none>
Annotations:  kubernetes.io/service-account.name: dashboard-admin
              kubernetes.io/service-account.uid: 2b50e797-7aa7-4637-b43b-
fc2a231b27fc
```

Type: `kubernetes.io/service-account-token`

Data

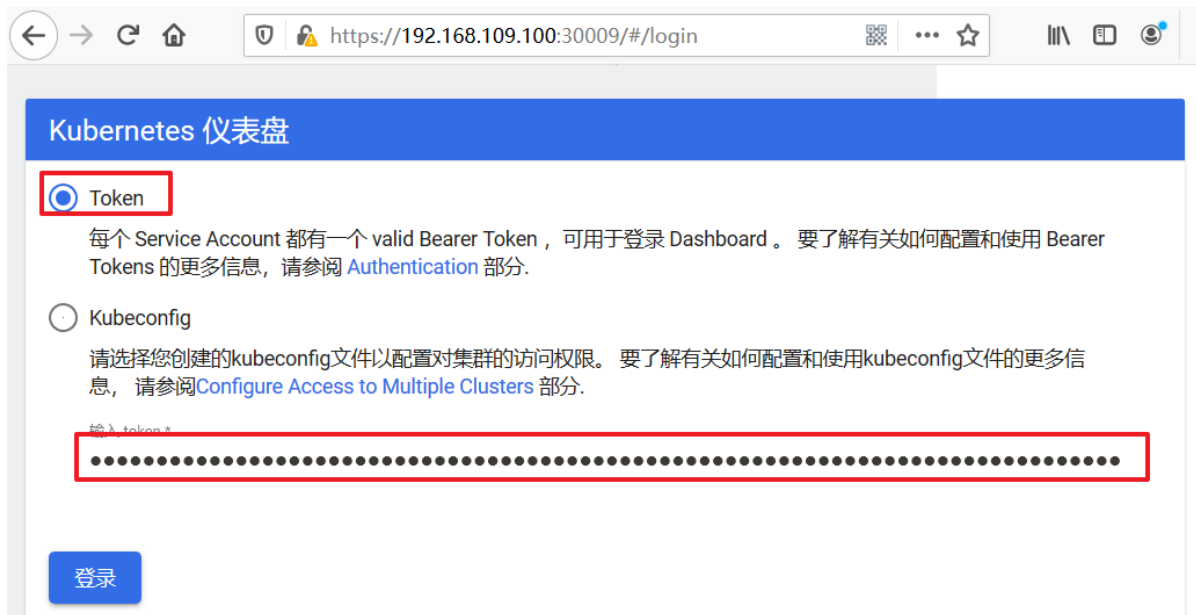
=====

token:

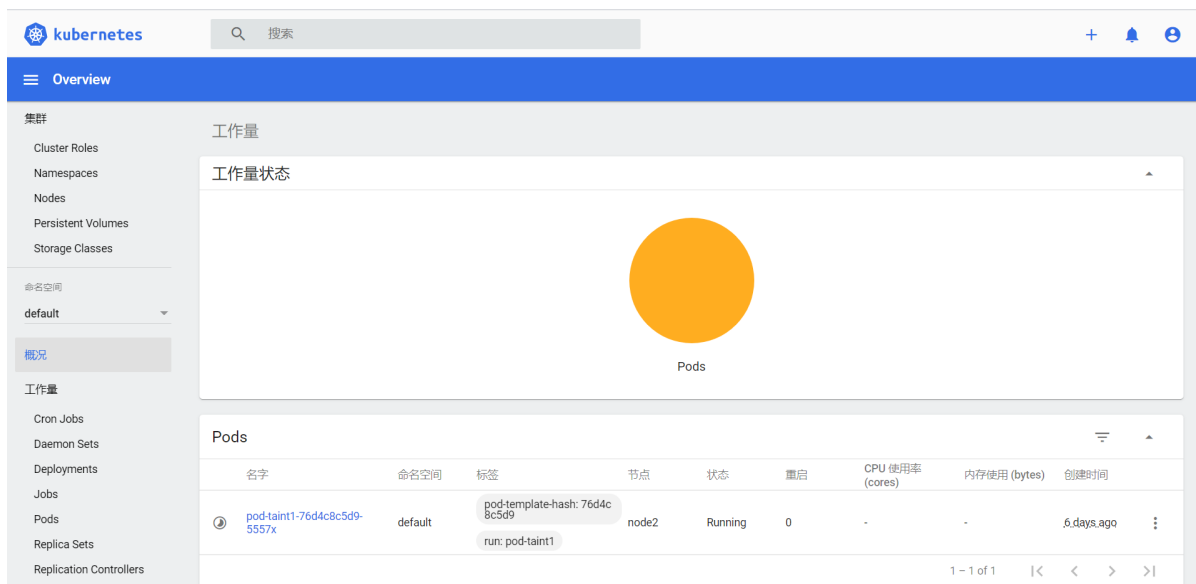
```
eyJhbGciOiJSUzI1NiIsImtpZCI6Iknxd2ZZNFpmjFDAjhMTG9xZjdtYXZKN2o1OUVtdkl7tu2JMU0RvT  
kNVV1EifQ.eyJpc3MiOiJrdWJlcm5ldGVzL3N1cnZpY2VhY2Nvdw50Iiwia3ViZXJuZXR1cy5pby9zZXZ  
2awNlYWnjb3VudC9uYwllc3BhY2UiOiJrdWJlcm5ldGVzLWRhc2hib2FyZCIsImt1YmVybmV0ZXMuaW8v  
c2VydmllZWJfY291bnQvc2VjcmV0Lm5lbWUioiJkYXNoYm9hcmQtYWRtaW4tdG9rZW4tZGxmcmwiLCJrd  
wJlcm5ldGVzLm1vL3N1cnZpY2VhY2Nvdw50L3N1cnZpY2UtYWnjb3VudC9uYwllIjoizGFzaGJvYXJkLW  
FkbWluIiwia3ViZXJuZXR1cy5pby9zZXZ2awNlYWnjb3VudC9zZXZ2awNlLWFfY291bnQudWlkIjoimMI  
1MGU3OTctN2FhNy00NjM3LWI0M2ItZmMyYTIZmWiYnZ2jiIiwic3ViIjoic3lzdGltOnN1cnZpY2VhY2Nv  
dw50Omt1YmVybmV0ZXMtZGFzaGJvYXJkomRhc2hib2FyZC1hZGlpbij9.s2gxdeysJLTRtcNmW-  
BfqW9iUnPU-_fm7Ar1ZrA9AL-2AK8HFDqd-JLYo_Zvihz1B8NqgjVXglrze4PfKnyjrPKMF9Q2dUN7m-  
U4cvZ70wwB2vsxdrVzQGpm1A9p8PUjv9cfMCihMMxF-  
F2SEV89RXULHPudnrDwawDs_y3uy8bBPTvkZBWnaqsnefwusQRns7s3ajgr0adFm6S11xisPhDZZqIS  
44xnpar1e7dv1-6fCPJxydv4qRCCgXR40SnntY-  
KD0KVNR0PSRp1JPYxs_Nuka4AVU12hpdybCurDIohbQEtx-se_C4Ex2CLA8iHeCzzP9m-QcubUGntba  
ca.crt:      1025 bytes  
namespace:   20 bytes  
[root@master ~]#
```

3) 通过浏览器访问Dashboard的UI

在登录页面上输入上面的token



出现下面的页面代表成功

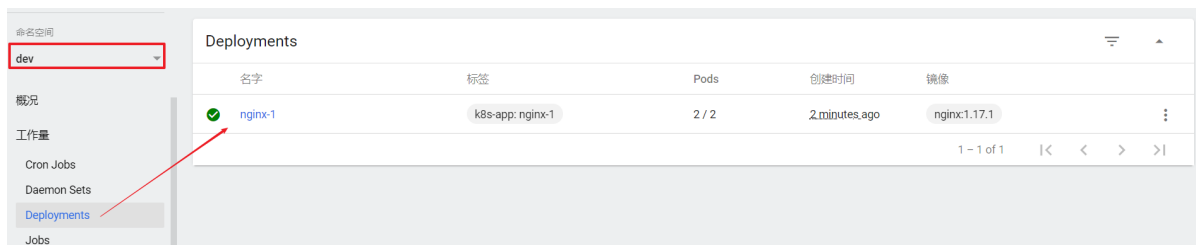


10.2 使用Dashboard

本章节以Deployment为例演示Dashboard的使用

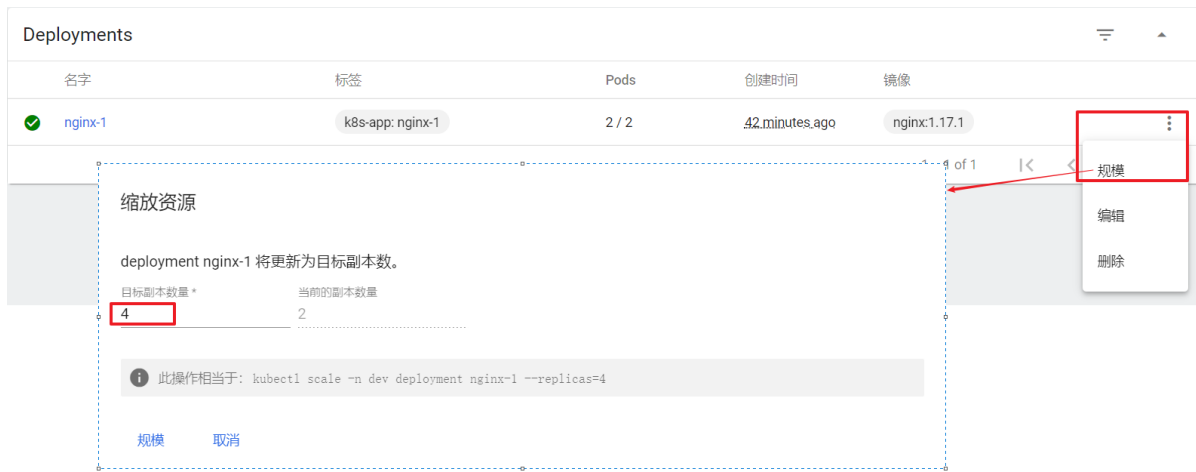
查看

选择指定的命名空间 `dev`，然后点击 `Deployments`，查看dev空间下的所有deployment



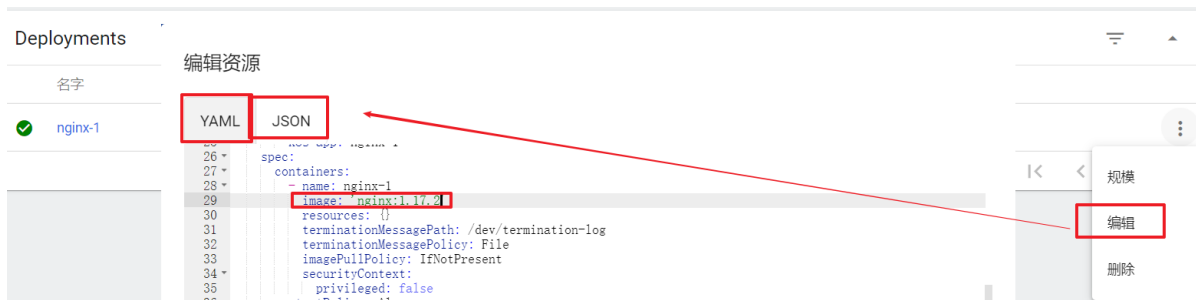
扩缩容

在 `Deployment` 上点击 `规模`，然后指定 `目标副本数量`，点击确定



编辑

在 Deployment 上点击 编辑，然后修改 yaml 文件，点击确定



查看Pod

点击 Pods，查看pods列表

Nodes		Pods							
Persistent Volumes		名字	标签	节点	状态	重启	CPU 使用率 (cores)	内存使用 (bytes)	创建时间
Storage Classes		nginx-1	k8s-app: nginx-1	node1	Running	0	-	-	2 minutes ago
命名空间		nginx-1-84b55dcd94-fh9gs	pod-template-hash: 84b55dcd94	node1	Running	0	-	-	2 minutes ago
概况		nginx-1-84b55dcd94-k7k5z	pod-template-hash: 84b55dcd94	node1	Running	0	-	-	2 minutes ago
工作量		nginx-1-84b55dcd94-kmlr9	pod-template-hash: 84b55dcd94	node1	Running	0	-	-	2 minutes ago
Cron Jobs		nginx-1-84b55dcd94-zznrk	pod-template-hash: 84b55dcd94	node1	Running	0	-	-	2 minutes ago
Daemon Sets									
Deployments									
Jobs									
Pods									

操作Pod

选中某个Pod，可以对其执行日志 (logs)、进入执行 (exec)、编辑、删除操作

nginx-1-84b55dcd94-fh9gs	k8s-app: nginx-1	node1	Running	0	-	-	5 minutes ago	...
nginx-1-84b55dcd94-k7k5z	k8s-app: nginx-1	node1	Running	0	-	-	5 minutes ago	...
nginx-1-84b55dcd94-kmlr9	k8s-app: nginx-1	node1	Running	0	-	-	5 minutes ago	...

Dashboard提供了kubectl的绝大部分功能，这里不再一一演示